

Master's Thesis

Deggendorf Institute of Technology – Campus Cham

Faculty of Applied Natural Sciences and Industrial Engineering

Study Programme: Master Artificial Intelligence for Smart Sensors and Actuators

Entwicklung eines SLAM und autonomen Navigationssystems für ein selbstfahrendes Auto

(translation in consultation with first examiner)

Development of a SLAM and Autonomous Navigation System for a Self-Driving Car

master's thesis to obtain the academic degree:

Master of Engineering (M.Eng.)

submitted by: Amit Sajeev

Cham

Matriculation Number: 22306894

first examiner: Prof. Dr. Dmitrii Dobriborsci

Cham, 14.09.2026

Declaration

name, student

Amit Sajeev

name, supervisor

Prof. Dr. Dmitrii Dobriborsci

title of the master's thesis

Development of a SLAM and Autonomous Navigation System for a Self-Driving Car

1. I hereby declare that this Master's thesis is my own independent work, that it has not been submitted in whole or in part for any other examination, that no sources or tools other than those acknowledged have been used, and that all direct and indirect citations have been properly credited.

Cham, 14.09.2026

signature, student

2. I give my consent for my Master's thesis to be made accessible to the wider public through the university library. To enable this, I am providing one additional bound copy of the Master's thesis.

☒ Yes

☐ No

Cham, 14.09.2026

signature, student

I declare and guarantee that I am the sole holder of all rights to the Master's thesis, including the right of disposal over templates for attached figures, plans, or similar materials, and that making it publicly accessible violates neither third-party rights and claims nor statutory provisions.

To be completed by the supervisor if the author consents to making the Master's thesis publicly accessible:

3. I declare that the inclusion of a copy of the Master's thesis in the library holdings and the lending of this copy is:

☐ approved

☐ not approved

Cham

signature, supervisor

Declaration Concerning Usage of AI

This thesis employed an AI assistant, Claude [1], for drafting and editing support: structuring prose, checking consistency across chapters, formatting and generic explanations of well-established background concepts. Every specific technical claim, number, formula and fact in this document is independently sourced to the citation given at the point it is made, or is this project's own directly measured result – none of it is attributed to the assistant itself.

Cham, 14.09.2026

Signature of student

Index of Abbreviations

AMCL	Adaptive Monte Carlo Localization
API	Application Programming Interface
BT	Behavior Tree
CPU	Central Processing Unit
DDS	Data Distribution Service
DHCP	Dynamic Host Configuration Protocol
DIT	Deggendorf Institute of Technology
FPGA	Field-Programmable Gate Array
GUI	Graphical User Interface
HAL	Hardware Abstraction Layer
IMU	Inertial Measurement Unit
IP	Internet Protocol
JSON	JavaScript Object Notation
LiDAR	Light Detection and Ranging
MPPI	Model Predictive Path Integral (control)
NTP	Network Time Protocol
PAL	Product Application Layer (Quanser)
PID	Proportional-Integral-Derivative (controller)
PWM	Pulse-Width Modulation
QoS	Quality of Service
RMW	ROS Middleware (interface)
ROS	Robot Operating System
RTPS	Real-Time Publish-Subscribe (wire protocol)
RViz	ROS Visualization (tool)
SDF	Simulation Description Format
SEDP	Simple Endpoint Discovery Protocol
SLAM	Simultaneous Localization and Mapping
SPDP	Simple Participant Discovery Protocol
SSH	Secure Shell
TCP	Transmission Control Protocol
TF	Transform (ROS coordinate-frame library)
UDP	User Datagram Protocol
URDF	Unified Robot Description Format
XML	Extensible Markup Language
YAML	YAML Ain't Markup Language

Abstract

Autonomous mobile robots are typically developed in simulation before deployment on physical hardware, yet the transfer to a real platform routinely exposes gaps that simulated testing alone does not predict. This thesis documents the migration of a ROS 2 Nav2 navigation stack for the Quanser QCar, an Ackermann-steered vehicle, from Gazebo simulation onto physical hardware. Limited onboard storage, together with a ROS 2 Dashing/Humble distribution incompatibility on the robot’s onboard compute, motivated a custom TCP/JSON bridge. The thesis presents the bridge’s design, the calibration of an open-loop throttle model against real actuator behaviour and the diagnosis of a hardware buffering defect misattributed at first to drivetrain coasting. The resulting system is validated through closed-loop Nav2 goal-reaching tests on the vehicle. The observed sim-to-real discrepancies are quantified.

Key words: ROS 2, Nav2, Ackermann steering, sim-to-real transfer, Gazebo, Model Predictive Path Integral control, mobile robot navigation, Quanser QCar

Contents

Index of Abbreviations	iii
Abstract	iv
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	1
1.3 State of the Art	2
1.4 Research Gap	3
1.5 Thesis Structure	4
2 Groundwork and Fundamental Principles	5
2.1 The ROS 2 Communication Model	5
2.1.1 Why DDS Discovery Incompatibilities Manifest as Silence	6
2.1.2 The Action Interface and Goal Lifecycle	8
2.2 Ackermann Steering Kinematics	8
2.2.1 The True Two-Wheel Ackermann Angle	9
2.3 Simulation with Gazebo	10
2.3.1 The Plugin’s PID Control Loop and the Physics Step	11
2.3.2 Contact Friction via the Coulomb Model and Friction Cone	12
2.4 Simultaneous Localization and Mapping with Cartographer	13
2.5 The Nav2 Navigation Stack	13
2.5.1 MPPI’s Sampling, Scoring and Softmax Update in Detail	14
2.5.2 AMCL’s Particle Filter and Recovery Mechanism in Detail	15
2.6 The Quanser QCar Platform	17
2.7 The Sim-to-Real Gap	18
3 Simulation Stack Development	20
3.1 Diagnostic Standard: Verification by Direct Measurement	20
3.2 Baseline Manual Bicycle-Model Inversion	20
3.3 First Migration to <code>ros2_control</code> ’s <code>AckermannSteeringController</code>	21
3.3.1 A Wheel-Axis Bug Exposed by the Migration	21
3.4 Second Migration to the Direct Drive Plugin	22
3.4.1 A Quiet Regression Where Odometry Reverts to Ground Truth	23
3.5 The “Robot Not Moving Forward” Bug Pattern	24
3.5.1 Diagnostic Method	26
3.6 A Behaviour-Tree Fix for Ackermann Recovery	26
3.7 Summary of the Simulation-Stack Architecture	28

4	Real-Hardware Bridge Architecture	29
4.1	Why Native ROS 2 Communication With the QCar Is Not Available	29
4.2	Bridge Architecture Overview	30
4.3	LiDAR Angle-Convention Correction	32
4.4	Wheel Odometry over the Bridge	34
4.5	Steering-Trim Calibration	35
4.5.1	Steering Gain as a Second, Independent Nonlinearity	36
4.5.2	A Trim-Placement Bug and the “Apply at the Actuator Boundary” Principle	38
4.6	Clock Synchronisation and the Receipt-vs-Capture Timestamp Bug	39
4.7	Safe Shutdown Procedure	41
4.8	Summary	41
5	System Identification and Calibration	42
5.1	Problem Statement	42
5.2	Ruling Out Simpler Explanations	42
5.3	Active Braking as a Control-Design Attempt and a Safety Incident	43
5.4	Sim-versus-Real Comparison	45
5.5	Throttle Calibration	45
5.5.1	The Uncalibrated Gain Hypothesis	45
5.5.2	A First Calibration Attempt That Discovered a Deadband, Then Reverted	45
5.5.3	Final Calibration via an Empirical Duty Sweep	46
5.5.4	A Stray-Process False Positive	48
5.6	The Hardware Abstraction-Layer Buffering Defect	48
5.6.1	An Initial, Incomplete Diagnosis	49
5.6.2	Disproving Two Hypotheses and the Argument That Found the Real Cause	49
5.6.3	The Root Cause, a Ring-Buffer I/O Mode	50
5.6.4	Validation	51
5.7	Summary	52
6	Navigation Integration and Validation	54
6.1	Porting the Nav2 Configuration to Hardware	54
6.2	The Custom MPPI Critic <code>EarlyCommitCritic</code>	54
6.3	Phased Hardware Integration	57
6.3.1	First Real-Hardware Goals in Left- and Right-Turn Trials	58
6.4	Diagnosing a K-Turn Freeze at Reversal Cusps with a Custom Smoother	61
6.4.1	Reproduction	61
6.4.2	A Diagnosis Where the Planner Is Correct and the Controller Is Not	61
6.4.3	Three Plausible Fixes, All Reverted	62
6.4.4	A Reverse-Only, Gradually Blended Smoother	63
6.4.5	Extending the Corner, a Further and Larger Improvement	64
6.4.6	Real-Hardware Validation	65
6.4.7	A Re-Extension Bug and a Curvature-Continuing Refinement	67
6.4.8	A Related Planner-Side Finding on Near-Wall Goals and a Tolerance Trade-off	69

6.4.9	Two Follow-On Attempts, Reverted	69
6.4.10	A Related Operational Caution on Goal Preemption	70
6.5	Diagnosing an Apparent Display-Localisation Artefact	70
6.5.1	Ruling Out Simpler Causes	70
6.5.2	Instrumentation via <code>monitor_goal_timing.py</code>	71
6.5.3	Measured Result	72
6.5.4	Interpretation and a Necessary Caveat	73
6.6	Final Validation	73
6.7	Summary	74
7	Results	75
7.1	Actuator Calibration Accuracy	75
7.2	Stop-Distance and Latency	76
7.3	Sim-versus-Real Overshoot	77
7.4	Steering Response Accuracy	77
7.5	Navigation Robustness in Tight Spaces	77
7.6	Navigation Goal-Reaching	78
7.6.1	Goal 1, a Straight-Line Approach	79
7.6.2	Goal 2, a Curved Approach	80
7.6.3	Goal 3, a Near-180° Reversal Goal	81
7.7	Map Quality and Localisation Accuracy	82
7.7.1	Map Quality	83
7.7.2	Localisation Accuracy Ground-Truth	83
7.7.3	A LiDAR Range Calibration Curve and a Shared-Bias Hypothesis	84
7.8	Limitations	85
7.9	Applicability of the Results	87
8	Summary and Prospects	88
8.1	Summary	88
8.1.1	Critical Reflection	89
8.2	Prospects	89
	Bibliography	93
A	Appendix	95
A.1	QCar Hardware Safety Limits	95
A.2	Diagnostic and Calibration Tooling Reference	95
A.3	Real-Hardware Bring-Up Command Reference	96
A.3.1	Safe Shutdown Sequence	96
A.4	Source Code and Data Availability	96

List of Figures

2.1	DDS discovery: SPDP and SEDP stages	7
2.2	Ackermann steering geometry	10
2.3	The drive plugin’s per-wheel PID control loop	12
2.4	MPPI’s sampling-based control update	15
2.5	The AMCL predict-update-resample cycle	16
2.6	The QCar’s front-mounted sensor stack	17
4.1	Side view of the QCar’s onboard compute stack	30
4.2	Command/sensor data path: Nav2 to physical actuators	32
4.3	Commanded vs. implied real steering angle	37
5.1	Ring-buffer production/consumption lag over time	51
6.1	Launch dependency graph of <code>qcar_nav2.launch.py</code> (hardware branch)	55
6.2	The Phase 2 occupancy grid from real LiDAR data	58
6.3	The Nav2 stack running end-to-end in simulation	59
6.4	RViz during the right-turn trial	59
6.5	The QCar at the end of the right-turn trial	60
6.6	RViz during the left-turn trial	60
6.7	The QCar at the end of the left-turn trial	60
6.8	The reproduction goal’s reversal cusp in RViz	62
6.9	The corner-extension mechanism	65
6.10	A reversal cusp during real-hardware validation	66
6.11	RViz during a real-hardware reversal-manoevre trial	66
7.1	Stop latency before/after the <code>readMode=0</code> fix	76
7.2	Mean K-turn manoeuvre duration by configuration	78
7.3	Goal 1: commanded vs. actual speed	79
7.4	The QCar at rest at the end of the Goal 1 trial	80
7.5	Goal 2: commanded vs. actual speed	80
7.6	The QCar at rest at the end of the Goal 2 trial	81
7.7	Goal 3: commanded vs. actual speed	82
7.8	The QCar at rest at the end of the Goal 3 trial	83

List of Tables

2.1	QoS compatibility rule for the two policies relevant to this thesis (strongest requestable value shown first).	7
2.2	QCar Ackermann geometry parameters used throughout this thesis.	10
3.1	The recurring “not moving forward” bug pattern found while testing this drive architecture.	24
4.1	Illustrative raw-to-corrected LiDAR angle mapping (Equation 4.1).	33
4.2	Commanded steering angle versus implied real wheel angle (<code>STEERING_TRIM=0</code>).	36
4.3	Diagnostic measurements underlying the clock-offset fix.	40
5.1	Fixed-duty sweep results (QCar on the ground, real load).	47
5.2	Throttle calibration validation (Equation 5.2).	48
5.3	Effect of the <code>readMode=0</code> fix on start-up delay, extra distance and stop latency.	52
6.1	K-turn manoeuvre duration and recovery count, corridor reproduction goal (simulation).	64
6.2	Timing trace of one Nav2 goal (<code>monitor_goal_timing.py</code>).	72
7.1	Results at a glance.	75
7.2	Commanded-versus-real cruise speed, before and after calibration.	75
7.3	Stop-latency and extra-distance results, before and after the <code>readMode=0</code> fix.	76
7.4	Sim-versus-real stop-distance overshoot (measured before the fixes of Ch. 5).	77
7.5	Real wheel deflection versus kinematically commanded deflection, before and after <code>STEERING_GAIN</code> correction.	77
7.6	K-turn manoeuvre duration and recovery count, before and after the cusp-straightening smoother (simulation, three trials each).	78
7.7	Goal 1: given goal vs. final AMCL-estimated pose (map frame).	79
7.8	Goal 2: given goal vs. final AMCL-estimated pose (map frame).	81
7.9	Goal 3: given goal vs. final AMCL-estimated pose (map frame).	82
7.10	Map-frame distance versus physical tape measurement, three reference spans.	83
7.11	AMCL-reported final pose versus physical ground truth, three trials of the same straight 2 m goal.	84
7.12	Raw LiDAR range versus tape-measured distance from a flat wall, five points.	84
A.1	QCar documented hardware safety limits.	95
A.2	Diagnostic and calibration scripts referenced in this thesis.	95

1 Introduction

1.1 Motivation

Autonomous, self-driving vehicles are one of the most visible and heavily pursued applications of modern robotics, promising to navigate complex, populated environments without human intervention. Realising that promise at full scale is costly and safety-critical, so a common alternative is to first develop and validate the same core competencies – mapping, localisation and navigation – on a small-scale, real-hardware platform.

This thesis uses the Quanser QCar for exactly that purpose: a 1/10th-scale, real-wheel-drive, Ackermann-steered research vehicle equipped with a 2D LiDAR, an inertial measurement unit (IMU) and an onboard NVIDIA Jetson TX2 compute module [17]. It is a physically real, if miniature, self-driving car. The practical challenges of autonomous navigation can be identified and solved on it directly, rather than only theorised about.

The project’s goal was to build a self-sufficient navigation system capable of taking the QCar from a given start pose to a given goal pose in a mapped indoor environment, reliably and without manual intervention along the way.

Reaching that goal required a complete stack: sensor-driven mapping, localisation within that map and closed-loop trajectory control. In contemporary robotics practice, building such a stack begins in simulation, where repeatable, safe iteration lets a planner be tuned against a thousand trial runs before it is ever allowed to steer a physical vehicle. The pattern runs deep enough in the field that entire open-source ecosystems are built around software that runs, largely unmodified, against either a simulated or a physical robot. Three such ecosystems anchor this thesis: the Robot Operating System (ROS) and its successor ROS 2 [19, 10], the Gazebo simulator [9] and the Nav2 navigation stack [11].

Simulation here was never treated as a separate or preliminary exercise. It was the first of two necessary phases toward one end goal: a navigation stack validated on the physical QCar. Every design decision made in simulation – the vehicle’s kinematic model, its sensor suite, the manoeuvres it was asked to perform – was made with the real QCar’s actual capabilities in mind, not any capability convenient only in simulation. The aim was for migrating the finished stack onto hardware to be a validation step, not a redesign.

The stack was accordingly built and validated end-to-end in a Gazebo simulation of the vehicle first, then migrated onto the physical robot for a second, hardware validation phase. This thesis reports on both in turn.

1.2 Problem Statement

At the outset of this project, no navigation stack had yet been developed for the QCar. The complete pipeline – sensor-driven mapping, localisation and navigation – had to be designed and built from scratch, not extended from an existing implementation. The *actual state* was “no working mapping, localisation, or navigation capability yet built for this project’s QCar setup, in any form”.

The *target state* was two-fold, matching the two phases described in Section 1.1: a complete navigation stack built and validated end-to-end in simulation, deliberately designed around the real QCar’s actual capabilities rather than any simulation-only convenience. This same stack was then to be migrated onto the physical QCar and validated there, reaching goal poses reliably. Any discrepancy between the two phases was to be identified and explained and, where practical, corrected or compensated for – not left as an unexplained residual error.

Reaching that target state required answering four sub-questions, each of which forms one methodology chapter of this thesis:

1. The navigation stack covers SLAM-based mapping, localisation and Nav2-based path planning and control. How should it be designed and validated end-to-end in simulation, built around the real QCar’s actual kinematics and sensor suite, so that it is a genuine first phase toward hardware operation rather than a simulation-only exercise? (Chapter 3)
2. The robot’s onboard compute cannot run the same ROS 2 distribution as the navigation stack. How should the two be connected so that the rest of the stack does not need to know the difference? (Chapter 4)
3. The vehicle exhibits both an unknown gain and a hard static-friction non-linearity. How should the mapping from a ROS 2 velocity command to a physical actuator signal be identified and calibrated? (Chapter 5)
4. Once bridged and calibrated, does the navigation stack actually reach goals reliably on hardware? What residual gap between the simulated and physical phases remains? (Chapters 6 and 7)

1.3 State of the Art

ROS 2 was designed to address shortcomings of the original ROS that made it unsuitable for production and safety-relevant use, chief among them a lack of real-time support and a single-point-of-failure master node. In their place, ROS 2 replaces ROS’s custom TCP transport with the Data Distribution Service (DDS) publish-subscribe middleware standard [14, 10].

Gazebo is the long-established open-source robot simulator most widely paired with ROS/ROS 2 [9], providing physics simulation and sensor models (LiDAR, cameras, IMUs) against the same message interfaces a real robot would publish and subscribe to. This project relied on that property to develop and validate its navigation stack in simulation before ever running it on hardware.

Nav2 is the de facto standard ROS 2 navigation stack, providing pluggable costmaps, global planners and local trajectory controllers behind a behaviour-tree-orchestrated action interface [11]. MPPI control, the local trajectory controller used in this project, samples large numbers of candidate control sequences and weights them by a differentiable cost function evaluated over a rollout horizon, which avoids the need for a convex or otherwise analytically tractable vehicle model [24]. This suits an Ackermann-steered vehicle well, since its real dynamics (addressed in Chapter 5) turned out to be considerably less clean than the textbook bicycle model in Chapter 2.

Simultaneous localisation and mapping (SLAM) and probabilistic localisation are similarly well-established techniques here, not novel contributions of this thesis. Mapping uses Cartographer, a real-time, submap-based 2D LiDAR SLAM implementation [8]; localisation during navigation uses Adaptive Monte Carlo Localisation (AMCL), a particle-filter approach to pose estimation within a known map [22, 23].

Sim-to-real transfer is most extensively studied in reinforcement-learning contexts, where a policy trained purely in simulation is deployed with no further real-world tuning [25]. This project’s navigation stack – SLAM, AMCL and MPPI – is classically engineered rather than learned and is configured identically in both simulation and reality. The challenges this thesis reports when migrating that stack to hardware (Chapters 4–6) are of a different, more concrete kind than the reality-gap problem addressed in that literature. They are specific, individually diagnosable mismatches between a simulated and a real implementation of the same explicit model, not a policy’s failure to generalise beyond its training distribution.

1.4 Research Gap

Section 1.3 named the tools this thesis builds on: ROS 2, Gazebo, Nav2, Cartographer and MPPI. None of them were written or substantially modified here. What none of them provide, out of the box, is a working answer for this platform’s specific combination of constraints.

No published Nav2 integration handles a vehicle whose onboard compute cannot run a matching ROS 2 distribution and is too storage-constrained to simply install a second one alongside it. Closing that gap meant designing a bespoke, non-ROS communication layer from scratch (Chapter 4), not configuring an existing one. Nav2’s stock MPPI critic set is documented and tuned against differential-drive and generic holonomic platforms; it has no critic addressing the late-turn-commitment behaviour this thesis found on an Ackermann-steered vehicle with a wide minimum turning radius (Section 6.2). Nav2 also ships no fix for the reversal-cusp K-turn freeze this thesis reproduced, diagnosed and resolved with a custom smoother plugin (Section 6.4). This failure mode is specific to tight-space Ackermann geometry, not a defect in Nav2 itself.

This thesis’s own contributions sit in five places:

- the TCP/JSON bridge architecture connecting the navigation stack to a storage-constrained, ROS-incompatible onboard compute (Chapter 4);
- **EarlyCommitCritic**, a custom MPPI critic plugin addressing Ackermann-specific late-turn commitment (Section 6.2);
- a custom path-smoother plugin resolving the reversal-cusp K-turn freeze (Section 6.4);
- root-cause diagnoses of defects specific to this platform: a hardware-abstraction-layer buffering artefact misattributed to physical coasting (Section 5.6), a LiDAR angle-convention bug (Section 4.3) and a display-localisation timing artefact (Section 6.5);
- an empirical actuator-calibration methodology, throttle deadband characterisation and a steering-gain sweep, built for this vehicle rather than taken from a paper.

Together, these close the distance between the general-purpose tools of Section 1.3 and a working, hardware-validated navigation system.

1.5 Thesis Structure

This thesis follows the two-phase methodology described in Section 1.1. It begins by laying the technical groundwork the navigation stack is built on, then develops and validates that stack in simulation. From there it migrates the stack to the physical QCar and works through the bridging, calibration and integration issues that migration surfaced, before reporting and reflecting on the results of both phases together. The remaining chapters are structured as follows:

- Introduces the technical foundations this thesis builds on: the ROS 2/DDS communication model, Ackermann steering kinematics, the Gazebo simulator and its ROS 2 integration, SLAM via Cartographer and the Nav2 stack’s AMCL localisation and MPPI control components.
- Describes the simulation-side development of the navigation stack, including a recurring vehicle-dynamics bug pattern uncovered during testing of this drive architecture.
- Presents the design of the TCP/JSON bridge that connects the ROS 2-based navigation stack to the robot’s onboard compute, which cannot run that stack natively, including the odometry, LiDAR angle-convention and steering-trim fixes required to make the bridged sensor and actuator data trustworthy.
- Covers the identification of the vehicle’s throttle response, including the static-friction deadband and the hardware abstraction-layer buffering defect that had been mistaken for physical coasting.
- Describes integrating and tuning the full Nav2 stack against the physical robot, including a custom MPPI critic plugin and AMCL parameter changes, culminating in closed-loop goal-reaching tests.
- Presents and quantifies the resulting gap between the simulated and physical phases across the project’s diagnostic measurements.
- Answers the research questions posed above and reflects on what generalises beyond this specific platform.

2 Groundwork and Fundamental Principles

This chapter introduces the concepts and software components that the remainder of the thesis builds on: the ROS 2 communication model (Section 2.1), Ackermann steering kinematics (Section 2.2), the Gazebo simulator (Section 2.3), SLAM via Cartographer (Section 2.4), the Nav2 navigation stack (Section 2.5), the physical Quanser QCar platform (Section 2.6) and the general concept of the sim-to-real gap (Section 2.7).

2.1 The ROS 2 Communication Model

The Robot Operating System (ROS) is not an operating system in the kernel sense. It is a middleware and tooling ecosystem for distributed robotics software, where independent processes (*nodes*) communicate by publishing and subscribing to named, typed *topics*, or by calling *services* for synchronous request/response interactions. This node/topic/service nomenclature comes from the original 2009 ROS paper [19]. *Actions*, for invoking long-running, cancellable, feedback-reporting tasks, were a later addition, formalised as a first-class ROS 2 concept alongside topics and services [10].

The original ROS (subsequently referred to as ROS 1) relied on a custom TCP-based transport and a single centralised `roscore` master process for node discovery. This design was later recognised as unsuitable for real-time, multi-robot, or safety-relevant deployments: the master is both a single point of failure and a source of unbounded discovery latency [10].

ROS 2 replaces this transport with the Data Distribution Service (DDS), an OMG-standardised, decentralised publish-subscribe protocol originally developed for defence and industrial real-time systems [14]. Each ROS 2 node embeds a DDS participant and nodes discover each other via a standardised discovery protocol (commonly Simple Discovery Protocol, SPDP/SEDP) rather than a central registry. Topic, service and action semantics are implemented on top of DDS’s publish-subscribe and request-reply patterns via the ROS Middleware interface (RMW) – an abstraction layer that lets multiple DDS implementations, such as Fast DDS and Cyclone DDS, serve as ROS 2’s transport interchangeably [10].

DDS communication is governed per-topic by Quality of Service (QoS) policies [14]. The two relevant here are reliability (`RELIABLE` vs. `BEST_EFFORT`) and durability (`VOLATILE` vs. `TRANSIENT_LOCAL`, the latter retaining and re-delivering the last sample(s) to late-joining subscribers), alongside history depth and others. A mismatch between a publisher’s and a subscriber’s QoS settings can silently prevent message delivery, even when both sides otherwise appear correctly configured. Section 6.5.2 describes one such mismatch, encountered directly during this project’s own diagnostic tooling.

This discovery and transport layer is directly relevant to this thesis’s central architectural constraint: DDS participants must share a compatible wire protocol version to discover and communicate with each other at all. Chapter 4 documents that the QCar’s factory-installed ROS 2 Dashing distribution (2019) and this project’s ROS 2 Humble development host (2022) do not, in practice, satisfy this requirement. This mismatch is one of two independent reasons

for the TCP/JSON bridge presented there (Section 4.1).

2.1.1 Why DDS Discovery Incompatibilities Manifest as Silence

Understanding *why* a protocol-version mismatch produces total silence, rather than a partial or degraded connection, requires a closer look at how two DDS participants find each other in the first place. This is the exact mechanism Chapter 4’s diagnosis exercises. This discovery mechanism and the wire protocol it runs over is defined in a separate OMG specification from the DCPS API specification cited above: the Real-time Publish-Subscribe (RTPS) DDS Interoperability Wire Protocol [13]. DDS discovery happens in two stages.

The first stage, the Simple Participant Discovery Protocol (SPDP), has every participant periodically multicast, by default, a small announcement of its own existence and the network endpoints it listens on, while listening for the equivalent announcements from others. This is how two DDS processes on the same network first become aware of each other, with no central broker involved.

The second stage begins once two participants know of each other via SPDP. The Simple Endpoint Discovery Protocol (SEDP) exchanges the finer-grained information each side needs to actually match publishers to subscribers: topic names, data types and QoS policies. Only after SEDP completes successfully does either side create the local proxy objects needed to actually send or receive user data.

A topic that exists on one side but was never successfully matched via SEDP behaves, from the application’s point of view, exactly as if the other process did not exist. No error is raised on either side, because DDS discovery is a best-effort background process, not a connection attempt that can fail loudly.

Both SPDP and SEDP messages are themselves serialised using the DDS wire protocol (RTPS – Real-Time Publish-Subscribe), which is versioned. A participant running an RTPS implementation that predates a given protocol revision does not necessarily reject a newer participant’s announcement outright. Depending on exactly which fields changed between versions, it can instead fail to parse the announcement correctly, or parse it into a form that never completes the handshake. From the outside, this looks identical to no announcement having arrived at all.

QoS policies add a second, independent way for two otherwise discoverable endpoints to fail to exchange data – this time by *design*, not by defect. DDS defines an explicit compatibility rule per policy, evaluated during SEDP matching: a subscriber may request a QoS *no stronger* than what a publisher offers (Table 2.1) [14]. A subscriber requesting a stronger policy than the publisher offers on that dimension is a declared incompatibility. DDS silently does not match that pair. Again, an application-level developer would learn nothing about this without independently comparing the two sides’ QoS settings.

By this rule, a `VOLATILE` subscriber against a `TRANSIENT_LOCAL` publisher is formally compatible. This is exactly the configuration Section 6.5.2 found, on real hardware, delivering zero messages in practice despite being spec-compliant.

Section 6.5.2 discusses the likely practical reason. Durability governs whether *historical*, already-published samples are replayed to a late-joining subscriber, not merely whether future

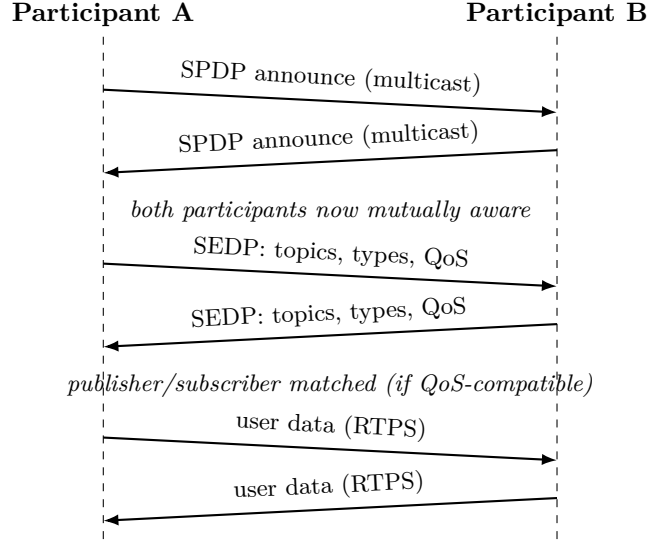


Figure 2.1: DDS discovery, in two stages, before any user data is exchanged: SPDP lets participants become mutually aware with no central broker; SEDP then matches individual publishers to subscribers by topic, type and QoS. Both stages are themselves messages on the versioned RTPS wire protocol – if either side fails to parse the other’s SPDP/SEDP messages (e.g. due to a protocol-version mismatch), the process stops here with no error raised on either side, exactly the “silent failure” Section 4.1 diagnosed on hardware.

Table 2.1: QoS compatibility rule for the two policies relevant to this thesis (strongest requestable value shown first).

Policy	Publisher offers	Subscriber may request
Reliability	RELIABLE	RELIABLE or BEST_EFFORT
Reliability	BEST_EFFORT	BEST_EFFORT only
Durability	TRANSIENT_LOCAL	TRANSIENT_LOCAL or VOLATILE
Durability	VOLATILE	VOLATILE only

ones are delivered. A `VOLATILE` subscriber that joins after a transient-local publisher’s relevant samples were already sent may receive none of the backlog and then simply waits for a next sample – one that, for a low-frequency topic like action status, may not arrive for a long time.

The point for this chapter is the general one: formal QoS compatibility, as defined by the DDS specification, is a necessary but not sufficient condition for an application to actually observe the message flow it expects.

2.1.2 The Action Interface and Goal Lifecycle

Nav2’s `navigate_to_pose` interface, used throughout Chapter 6, is a ROS 2 *action* [10]. Rather than being a distinct transport, an action is built on top of topics and services: it pairs a goal-request service call with two topics the server publishes on. A feedback topic carries periodic progress updates. A status topic reports each known goal’s current state as one of a fixed enumeration (`ACCEPTED`, `EXECUTING`, `CANCELING`, `SUCCEEDED`, `CANCELED`, `ABORTED`, among others).

The status topic uses `TRANSIENT_LOCAL` durability, the same property discussed above. This lets a subscriber that starts listening partway through a goal’s execution recover its current state from the retained history, rather than being blind until the next status change.

Status updates are only published when the server’s internal state machine actually transitions. The status topic’s history depth is finite. This means a fast state machine can advance through more than one transition between two publishes that an external observer actually receives. There is no guarantee that every individual state is observable – only that the *latest* state at any polling instant is correct.

Section 6.5.2 documents exactly this: a goal was observed to skip directly from absent to `EXECUTING` in one status message, with no separately observable `ACCEPTED` update in between. The server’s internal state machine does pass through `ACCEPTED` on its way to `EXECUTING` – the transition happened, but no status publish was ever emitted while it was current in isolation. This is a general property of polling a periodically published state summary, not a special case of this one topic.

2.2 Ackermann Steering Kinematics

The QCar is a rear-wheel-drive vehicle with front-wheel Ackermann steering [21]. The two front wheels pivot independently about near-vertical steering axes, such that for any commanded steering angle, both front wheels’ instantaneous velocity vectors are perpendicular to lines drawn through a single common point on the extended rear axle. This point is the vehicle’s instantaneous centre of rotation. This geometric constraint means the two front wheels must generally be steered to slightly different angles, with the inner wheel turning more sharply than the outer, so that all four wheels roll without slipping through a common turn centre.

For control and estimation purposes, this thesis follows the standard simplification to a single-track (“bicycle”) model [21], which replaces the two front wheels with a single virtual wheel at the centre of the front axle steered at angle δ and the two rear wheels with a single virtual wheel at the centre of the rear axle. With vehicle heading θ , wheelbase L (the longitudinal distance between the front and rear axles) and rear-axle speed v , the kinematic bicycle model is:

$$\dot{x} = v \cos \theta \quad (2.1)$$

$$\dot{y} = v \sin \theta \quad (2.2)$$

$$\dot{\theta} = \frac{v}{L} \tan \delta \quad (2.3)$$

and the corresponding instantaneous turning radius about the rear axle is

$$R = \frac{L}{\tan \delta}. \quad (2.4)$$

Equation 2.3 is the basis of the onboard bridge’s dead-reckoning odometry described in Chapter 4, which genuinely integrates wheel-encoder and commanded-steering readings forward through this relationship at every control cycle to estimate pose from scratch. A single-track model is a simplification: it assumes no lateral tyre slip and treats both front wheels as a single steering angle rather than the true Ackermann-differentiated pair. Chapter 5 presents direct evidence of where this simplification’s limits were reached in practice on physical hardware (Section 5.5.2).

2.2.1 The True Two-Wheel Ackermann Angle

The single-track model above is a control- and estimation-time convenience. It is not what the Gazebo drive plugin actually commands at the level of the two physical front wheels.

Because the two front wheels steer about laterally separated axes, a single-track model’s one steering angle δ must generally be converted into two slightly different angles, with the inner wheel (on the side the vehicle is turning toward) steered more sharply than the outer. This ensures both front wheels’ rolling directions remain tangent to circles sharing the same centre on the extended rear axle. This is the condition for rolling without slipping introduced in Section 2.2’s opening paragraph. With track width T (the lateral distance between the two front steering axes) and wheelbase L , the exact left- and right-wheel steering angles corresponding to a single-track angle δ are:

$$\delta_{\text{left}} = \text{atan2}\left(\tan \delta, 1 - \frac{T}{2L} \tan \delta\right) \quad (2.5)$$

$$\delta_{\text{right}} = \text{atan2}\left(\tan \delta, 1 + \frac{T}{2L} \tan \delta\right) \quad (2.6)$$

(signed so that a positive δ , a left turn, gives $\delta_{\text{left}} > \delta_{\text{right}}$ – the inside wheel of the turn steers further). This is precisely the pair of relationships the `gazebo_ros_ackermann_drive` plugin evaluates every simulation step from a single commanded angular velocity. Chapter 3 also refers to this same formula when describing the plugin auto-computing Ackermann geometry from URDF joint positions: T and L in Equations 2.5–2.6 are measured directly from the URDF’s own front-hub joint origins at spawn time, never stated as separate configuration values.

As $T \rightarrow 0$ (a vanishingly narrow front track), both angles converge to the single-track δ . This confirms Equations 2.5–2.6 correctly generalise, rather than contradict, the simpler model of

Equations 2.1–2.3.

Table 2.2: QCar Ackermann geometry parameters used throughout this thesis.

Parameter	Value	Source
Wheelbase L	0.257 25 m	URDF joint geometry (measured)
Front track width	0.1118 m	URDF joint geometry (measured)
Rear track width	0.1122 m	URDF joint geometry (measured)
Wheel radius	0.033 m	Wheel mesh bounding box (measured)
Max. steering angle δ_{max}	0.5236 rad (30°)	Vehicle/plugin limit
Min. turning radius (from Eq. 2.4)	0.446 m	Derived

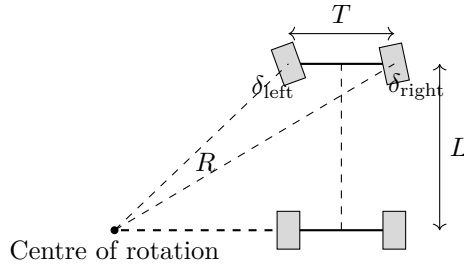


Figure 2.2: Ackermann steering geometry (top-down view, not to scale): the two front wheels are steered to different angles $\delta_{left} > \delta_{right}$ (left turn shown) so that lines through both front wheels’ rolling directions and the rear axle all intersect at a single centre of rotation. Wheelbase L and track width T are the parameters of Equations 2.5–2.6.

2.3 Simulation with Gazebo

Gazebo is a physics-based 3D robot simulator, originally built around a rigid-body dynamics engine, a small set of sensor models (odometry, ray-proximity/laser range-finding and camera) and a modular client/interface layer for injecting control logic into simulated models without coupling that logic to the physics engine itself [9].

Two later generations of Gazebo are relevant to this project’s history. Both are architecturally distinct from each other and both extend the original model with a formal SDF-based plugin system and a substantially wider sensor suite, including LiDAR and IMU sensor models. Only the LiDAR model is actually used in this project’s simulation stack (see the IMU note in Section 2.6). *Gazebo Classic* (versions up to 11) integrates with ROS 2 through the separate `gazebo_ros` / `gazebo_plugins` packages, while the newer *Gazebo Sim* (formerly Ignition Gazebo) integrates through `ros_gz_sim` / `ros_gz_bridge`.

The two are not drop-in replacements for each other. Plugin filenames, topic remapping mechanisms and world-file (SDF) syntax for built-in physics/sensor systems all differ between them. A package written against one will not load under the other without a systematic conversion.

This project uses Gazebo Classic 11 with the `gazebo_ros` family of plugins, matching the ROS 2 Humble / Ubuntu 22.04 host environment, in which Gazebo Sim’s binaries are not available at all.

The robot itself is described in the Unified Robot Description Format (URDF), an XML dialect specifying the vehicle’s kinematic tree (links and joints), visual meshes and collision geometry. A `<gazebo>` block within the URDF attaches simulation-only elements: sensor definitions and the drive plugin (`libgazebo_ros_ackermann_drive.so`) that subscribes to `/cmd_vel` and converts commanded linear/angular velocity into per-wheel drive and steering-hub joint efforts. It uses the Ackermann geometry of Section 2.2 for this, auto-computed from the URDF’s own joint and collision geometry at spawn time rather than restated in a separate configuration file.

A recurring practical property of this plugin, directly relevant to Chapter 3, is that its wheel-radius auto-detection only supports primitive collision shapes (cylinders, spheres). A mesh-shaped wheel collision silently yields a wheel radius of zero, which propagates as a division-by-zero into the plugin’s velocity PID controller.

Simulated friction is likewise not automatic: both the wheel links and the ground plane must be assigned explicit Coulomb friction coefficients (SDF’s `mu1/mu2`), or the simulated vehicle exhibits perfect wheel slip with no resulting translation, regardless of commanded wheel speed. Neither failure mode raises an error. Both look, from the outside, like “the robot is not moving.” Both are addressed in detail in Section 3.5.

2.3.1 The Plugin’s PID Control Loop and the Physics Step

The drive plugin does not run on its own timer. It registers a callback on Gazebo’s `WorldUpdateBegin` event, which fires once per physics step of the rigid-body solver – by default 1 ms of simulated time per step, i.e. 1000 Hz, independent of and generally much faster than any ROS 2 topic rate in the system.

On every physics step, regardless of whether a new `/cmd_vel` message has arrived, the callback re-reads the rear wheel’s current angular velocity from the physics engine. It compares that reading against the most recently *commanded* linear velocity, converted to an equivalent target angular velocity (`target_linear/wheel_radius`) and passes the resulting error through a discrete PID controller. The controller’s output is applied directly as a joint *force* (torque), not a velocity: this is a torque-controlled velocity-tracking loop, not a kinematic teleport of the wheel to a commanded speed. The controller update takes the classical form [2]

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}, \quad e(t) = \text{measured}(t) - \text{target}(t), \quad (2.7)$$

evaluated once per physics step, with the integral and derivative terms accumulated or estimated over the actual, possibly non-uniform, time elapsed since the previous step. The integral term can optionally be clamped to a configured $[i_{\min}, i_{\max}]$ range, to prevent unbounded windup while the system is far from its target.

Three independent instances of this controller run inside the plugin: one for the rear-axle drive velocity and one each for the left and right steering hubs, tracking the two Ackermann angles of Equations 2.5–2.6 independently rather than as a single shared steering command. All three sets of gains (K_p , K_i , K_d) are declared per-vehicle in the URDF’s `<gazebo>` plugin block. Section 3.5 documents in detail what happens when these gains are tuned for one physical configuration (e.g. frictionless wheels) and then reused unchanged after that configuration changes.

Equation 2.7 also explains precisely why the wheel-radius defect above is destructive

rather than merely inaccurate. With `wheel_radius = 0`, the target angular velocity `target_linear/wheel_radius` is not merely wrong – it is unbounded for any nonzero commanded linear velocity. That makes $e(t)$ itself unbounded, driving the PID controller to its output saturation limit (or to a floating-point infinity/NaN) regardless of how the three gains are tuned. No amount of gain retuning can compensate for an error signal that is not a finite number in the first place, which is why this class of bug survives naive “just tune the PID harder” troubleshooting.

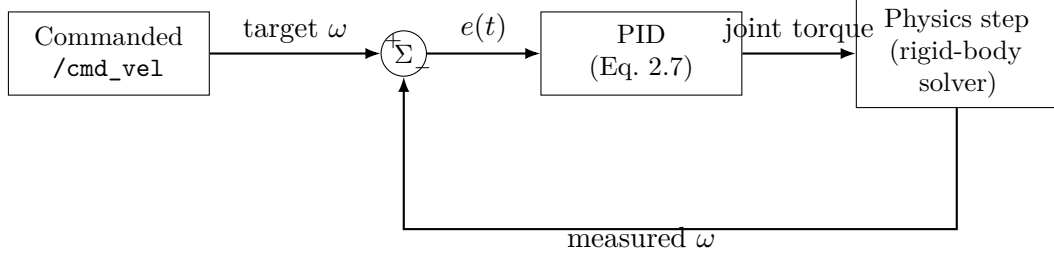


Figure 2.3: The drive plugin’s per-wheel torque control loop, re-evaluated every Gazebo physics step (1 kHz): a commanded velocity is converted to a target angular velocity, compared against the physics engine’s own measured wheel angular velocity to form an error $e(t)$ and passed through Equation 2.7’s PID controller, whose output is applied directly as a joint torque – three independent instances of this loop run per vehicle (drive, left steering, right steering).

2.3.2 Contact Friction via the Coulomb Model and Friction Cone

Gazebo Classic’s default physics engine, the Open Dynamics Engine (ODE), models contact friction at each collision point using a Coulomb friction approximation [20]: the maximum tangential (sliding) force a contact can sustain before slipping is proportional to the normal (compressive) force pressing the two surfaces together, with the constant of proportionality being the friction coefficient μ . ODE further allows this coefficient to be specified separately along two orthogonal directions in the contact plane (`mu1/mu2` in the URDF/SDF). A tyre could in principle grip differently along its rolling direction than sideways across it, though this project sets both equal – isotropic friction. ODE itself approximates the true (circular) Coulomb friction cone in one of two ways: a default “constant-force-limit” mode, where μ is treated as an absolute force limit independent of the actual normal force, or a more physically accurate “friction pyramid” approximation, enabled via the `dContactApprox1` contact flag [20]. This project does not override either setting. The URDF’s `<gazebo>` friction blocks (Listing 3.2) set only `mu1/mu2`, leaving Gazebo Classic’s own default friction-approximation mode in effect. The isotropic $\mu = 1.0$ value chosen is what actually mattered for fixing the zero-friction bug below, not which of the two approximation modes was active.

Critically, **ODE applies no friction at all to a contact whose μ_1/μ_2 default to zero**. There is no implicit “reasonable default” friction coefficient assumed for an unconfigured surface, unlike, say, a default coefficient of restitution.

This is precisely why Section 3.5’s bug 2 (zero wheel friction) produces literal, total 100% slip rather than merely reduced traction. With $\mu = 0$, the Coulomb limit on tangential force is zero regardless of how hard the wheel presses into the ground. No sliding friction force can develop at all, no matter how fast the wheel itself spins, so no forward thrust on the chassis can

develop either.

2.4 Simultaneous Localization and Mapping with Cartographer

Simultaneous Localization and Mapping (SLAM) is the problem of building a map of an unknown environment while concurrently estimating the robot’s pose within that same map, using only the robot’s own sensor stream and (optionally) odometry [3]. This thesis uses Google’s Cartographer [8], a real-time 2D/3D LiDAR SLAM system that performs local scan matching to build submaps and a global pose-graph optimisation to detect and correct loop closures. It is exposed to ROS 2 via the `cartographer_ros` package.

Cartographer can either fuse external odometry into its scan-matching prediction, or perform scan-matching-only pose estimation (`use_odometry: false`). In the latter case, it also provides the full `map` $\rightarrow$ `odom` $\rightarrow$ `base` transform (TF) chain itself (`provide_odom_frame: true`), rather than expecting an external odometry source to supply the `odom` $\rightarrow$ `base` portion.

This configuration choice was made for this project’s mapping pipeline and is detailed in Chapter 4. It turned out to remove an entire class of TF-ownership conflict between Cartographer and the vehicle’s own drive-plugin odometry that would otherwise arise, since ROS 2’s TF system permits only one broadcaster per transform edge at a time without producing ambiguous, jittering results.

2.5 The Nav2 Navigation Stack

Nav2 is the ROS 2 successor to the ROS 1 navigation stack, providing a modular, plugin-based pipeline for autonomous point-to-point navigation against a known (or concurrently built) map [11]. Its major components, all used in this thesis, are:

Costmaps Layered 2D occupancy grids [7] (static map layer, obstacle/LiDAR layer, inflation layer) that represent both the known static environment and dynamically observed obstacles as traversability cost, separately for global path planning and local trajectory control.

AMCL (Adaptive Monte Carlo Localization) A particle-filter localisation algorithm that maintains a probability distribution over the robot’s pose as a weighted set of discrete pose hypotheses (particles), resampled according to how well each particle’s predicted LiDAR scan matches the actual scan against the static map. This follows the Monte Carlo localisation method [5], presented here in the probabilistic framework used throughout this section [22]. AMCL’s motion-model noise parameters (commonly denoted α_1 – α_5) control how much particle spread is injected per unit of commanded motion. Chapter 6 documents a real localisation failure mode and its correction, tied directly to these parameters.

Global planning Nav2’s `SmacPlannerHybrid` plugin, used in this project, plans a kinematically feasible path using a hybrid-state A* search with a Reeds-Shepp motion primitive set [6], respecting the vehicle’s minimum turning radius (Table 2.2) rather than planning paths the physical vehicle cannot actually follow.

Local trajectory control: MPPI Model Predictive Path Integral control [24] is a sampling-based model-predictive control method. At each control cycle, a large number ($\mathcal{O}(1000)$) of candidate control sequences are simulated forward through a vehicle model over a fixed horizon. Each rollout is scored by a weighted sum of cost terms – *critics* in Nav2’s terminology, covering path alignment, path following, goal distance, obstacle cost and others. The executed control is then an importance-weighted average across all sampled rollouts, not the single lowest-cost trajectory. This avoids the need for a convex, analytically differentiable cost function or vehicle model, at the cost of requiring many parallel forward simulations per control cycle. Nav2’s MPPI controller plugin supports custom critic plugins loaded alongside the stock set. Section 6.2 describes a custom critic developed as part of this project’s navigation stack.

Behavior Trees Nav2 orchestrates recovery behaviours (costmap clearing, spinning in place, waiting, backing up) and overall navigation logic via an externally configurable behaviour tree (BT) [4], rather than hard-coded state machine logic. This allows per-robot customisation of, for instance, which recovery actions are physically valid for a given platform. It is directly relevant to Section 3.6, where the stock recovery tree’s default in-place **Spin** action is shown to be physically unachievable for an Ackermann-steered vehicle.

2.5.1 MPPI’s Sampling, Scoring and Softmax Update in Detail

MPPI’s central idea is to replace the search for a single optimal control sequence with a weighted average over many randomly perturbed candidates, where the weighting itself is derived from an information-theoretic optimality argument rather than chosen heuristically [24]. Concretely, given a current nominal control sequence $\mathbf{u} = (u_0, \dots, u_{T-1})$ over a horizon of T steps, the controller draws K independent perturbed sequences

$$u_t^{(k)} = u_t + \varepsilon_t^{(k)}, \quad \varepsilon_t^{(k)} \sim \mathcal{N}(0, \Sigma), \quad k = 1, \dots, K, \quad (2.8)$$

rolls each one forward through the vehicle model (the Ackermann bicycle model of Section 2.2, in this project’s configuration) to obtain a full predicted trajectory $\tau^{(k)}$ and evaluates a total cost $S(\tau^{(k)})$ for each. In Nav2’s implementation, this total cost is exactly the running sum that every enabled critic plugin adds into via a single shared per-trajectory accumulator, including the custom **EarlyCommitCritic** of Section 6.2. Section 6.2’s critic therefore contributes one additive term among several to $S(\tau^{(k)})$, not a separate decision stage. Trajectories are then combined into an updated control sequence via an importance-sampling weighted average:

$$w^{(k)} = \frac{\exp\left(-\frac{1}{\lambda} \left(S(\tau^{(k)}) - \rho\right)\right)}{\sum_{j=1}^K \exp\left(-\frac{1}{\lambda} \left(S(\tau^{(j)}) - \rho\right)\right)}, \quad u_t^{\text{new}} = \sum_{k=1}^K w^{(k)} u_t^{(k)}, \quad (2.9)$$

where $\rho = \min_k S(\tau^{(k)})$ is subtracted purely for numerical stability, so the largest exponent evaluated is always zero. $\lambda > 0$ is a temperature parameter controlling how sharply the weighting favours the very lowest-cost rollouts: as $\lambda \rightarrow 0$, Equation 2.9 approaches a hard arg-min over the sampled trajectories, while larger λ approaches a uniform average across all of them, largely ignoring cost.

Only the first control u_0^{new} of the updated sequence is actually sent to the vehicle each control cycle. The whole horizon is then shifted by one step and the process repeats. This is the standard receding-horizon pattern shared with other model-predictive control methods, but arrived at here via random sampling and a softmax-like weighted average, rather than a gradient- or search-based optimisation over a differentiable cost surface.

This is precisely why MPPI tolerates the sharply non-smooth, hand-crafted critic terms used throughout Nav2, including a critic like `EarlyCommitCritic` that is not differentiable in any convenient closed form, without requiring them to be reformulated as a smooth, gradient-friendly cost function first.

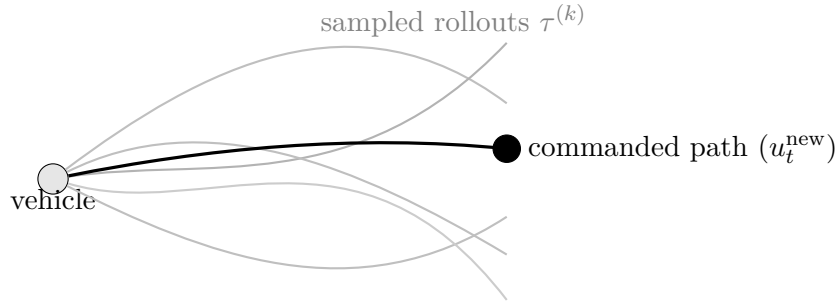


Figure 2.4: MPPI’s sampling-based control update: K randomly perturbed control sequences are each rolled forward through the vehicle model into a candidate trajectory $\tau^{(k)}$ (grey), each scored by the total critic cost $S(\tau^{(k)})$; Equation 2.9’s softmax weighting then combines all K trajectories into one commanded control sequence (black) – lower-cost rollouts contribute more weight, but every sampled trajectory contributes something, unlike a hard arg-min over a discrete set of candidates.

2.5.2 AMCL’s Particle Filter and Recovery Mechanism in Detail

AMCL maintains its belief about the robot’s pose as a set of N weighted particles $\{(x^{(i)}, w^{(i)})\}_{i=1}^N$, each a concrete pose hypothesis, updated on every new odometry/LiDAR pair through the standard Monte Carlo localisation prediction-update-resample cycle [22]:

1. **Prediction (motion model).** Each particle is propagated forward using the commanded/odometric motion since the last update, corrupted by noise whose magnitude is controlled by the α_1 – α_4 parameters mentioned in the component list above (roughly: α_1/α_2 scale rotational noise from the odometry’s rotational and translational components respectively, α_3/α_4 scale translational noise the same way). Larger values inject more particle spread per unit of commanded motion, modelling greater distrust in odometry.
2. **Update (measurement model).** Each particle’s weight is set proportional to the likelihood of the actual LiDAR scan given that particle’s hypothesised pose and the static map, typically via a likelihood-field model that scores each scan endpoint by its proximity to the nearest mapped obstacle from that hypothesised pose.
3. **Resampling.** A new particle set is drawn, with replacement, with probability proportional to weight. Particles consistent with the actual scan multiply, while inconsistent ones are discarded. The particle cloud then statistically converges toward the true pose over repeated cycles. AMCL’s “adaptive” element (KLD-sampling) additionally varies N itself between

update cycles, using fewer particles once the distribution is already tightly converged and more while it remains spread out.

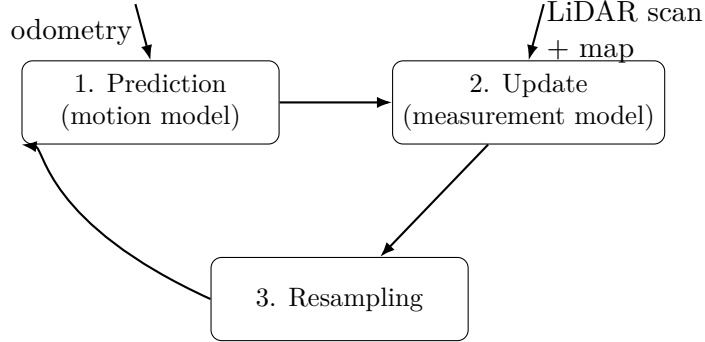


Figure 2.5: The AMCL particle filter’s prediction-update-resampling cycle, repeated on every new odometry/LiDAR pair: particles are propagated by the motion model, reweighted by how well each hypothesised pose explains the actual LiDAR scan, then resampled proportional to weight so the particle cloud statistically converges toward the true pose. The augmented-MCL recovery mechanism of Equation 2.10 modifies step 3 only, occasionally drawing a fresh, uniformly random particle instead of resampling from the existing weighted set.

The parameters directly responsible for the freeze failure mode documented in Section 5.2 are a separate pair from the four motion-model α s above. `recovery_alpha_slow/recovery_alpha_fast` govern an *augmented* MCL recovery mechanism: injecting randomly sampled particles when recent localisation quality drops. This general concept was evaluated directly against the alternative of plain MCL, for exactly this kidnapped-robot/recovery scenario, in [23].

AMCL’s specific implementation of the concept, read directly from its own parameter documentation and used as configured throughout this thesis, tracks two exponential moving averages of the particle set’s average weight: a slow long-run average $\bar{w}_{\text{slow}}$ and a fast short-run average $\bar{w}_{\text{fast}}$. Both are updated after every measurement step, as

$$\bar{w}_{\text{slow}} += \alpha_{\text{slow}} (\bar{w} - \bar{w}_{\text{slow}}), \quad \bar{w}_{\text{fast}} += \alpha_{\text{fast}} (\bar{w} - \bar{w}_{\text{fast}}), \quad (2.10)$$

where $\bar{w}$ is the current cycle’s average particle weight.

During resampling, instead of drawing every new particle from the existing weighted set, each new particle is drawn *uniformly at random from the entire map* with probability $\max(0, 1 - \bar{w}_{\text{fast}}/\bar{w}_{\text{slow}})$. Intuitively: if recent particle quality ($\bar{w}_{\text{fast}}$) has dropped noticeably below the longer-run baseline ($\bar{w}_{\text{slow}}$), the filter injects fresh, unbiased hypotheses to help it recover from a bad convergence or a “kidnapped robot” scenario, rather than continuing to resample only from a particle set that may have already converged to the wrong pose.

This project’s simulation-tuned configuration originally inherited $\alpha_{\text{slow}} = \alpha_{\text{fast}} = 0.0$. At that setting, neither running average is ever updated from its initial value, so the random-injection probability is permanently fixed rather than responsive to real degradation in particle quality. A clean simulation rarely needs this recovery path at all, but on real, non-ideal sensor noise, the particle set can converge overconfidently to an incorrect pose with no mechanism available to reintroduce diversity. This is the permanent freeze Section 5.2 reports and fixes, by raising these two parameters off zero.

2.6 The Quanser QCar Platform

The physical platform underlying this thesis is the Quanser QCar: the original platform, not the newer, distinct QCar 2 that Quanser’s documentation set also covers. It is a 1/10th-scale rear-wheel-drive, Ackermann-steered research vehicle [17].

Its onboard compute is an NVIDIA Jetson TX2 system-on-module, running Ubuntu 18.04 LTS with 32 GB eMMC storage and Python 3.6.9. It ships with both ROS 1 Melodic and ROS 2 Dashing pre-installed side by side, though neither is auto-sourced in the default shell environment.

The platform additionally ships with an Intel RealSense D435 RGBD camera, four 360° CSI cameras and an onboard IMU as standard hardware. This thesis’s navigation pipeline uses none of these. It relies solely on the 2D LiDAR for mapping, localisation and obstacle sensing, with pose estimation between LiDAR updates coming from wheel-encoder-based odometry rather than the IMU.

The IMU is electrically present and exposed by the vendor’s HAL/PAL library (gyroscope/accelerometer fields), but nothing in this project’s software stack reads it. Cartographer’s `use_imu_data` is left `false`. No IMU sensor plugin is declared in the simulation’s URDF either, so only the LiDAR is discussed further.

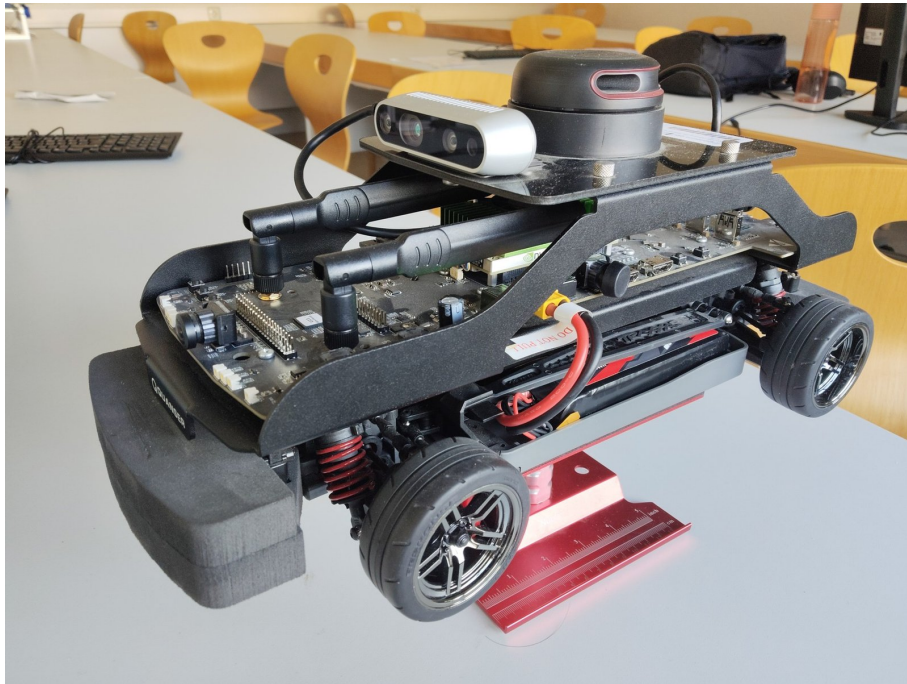


Figure 2.6: The QCar’s front-mounted sensor stack: the Intel RealSense D435 RGBD camera and the RPLiDAR unit sit above the carrier board, with the LiPo battery pack (labelled “DO NOT PULL”) and Ackermann front suspension visible below. This thesis’s navigation pipeline uses only the LiDAR, as discussed below.

Drive and steering actuation are exposed through Quanser’s own Hardware Abstraction Layer / Product Application Layer (HAL/PAL) Python library, which wraps the vehicle’s motor, servo, encoder and battery-monitoring hardware behind a `QCar` object exposing `read()`/`write()` calls. Two of its limits are enforced at the field-programmable gate array (FPGA) level regardless

of what software commands [17]. The steering command range is ± 0.5 rad against a physical limit of ± 0.5236 rad ($\pm 30^\circ$). An overcurrent protection state, triggered at 5 A/8 s, 10 A/2 s, or 15 A/0.5 s, forces the vehicle into a neutral mode until the underlying hardware-in-the-loop device is restarted. Both limits are respected throughout every piece of drive-control code presented in this thesis.

A third figure, commonly listed alongside these two, is not a hardware-enforced limit at all. Quanser’s own troubleshooting documentation *recommends* saturating commanded PWM duty cycle to $\pm 30\%$ with a 100%/s rate limiter, specifically to avoid a battery-brownout-induced shutdown [18] – not something the HAL itself clamps. This project’s own drive-control code adopts that recommendation (Section 5.5.3’s `THROTTLE_LIMIT`), but it is a software policy choice, not a property of the vehicle the way the other two limits are.

A property of the HAL library specific to this platform and central to one of this thesis’s most significant findings (Section 5.6) is its two supported I/O modes. In *immediate I/O* (`readMode=0`), each call to `read()` triggers a fresh hardware sample. In *task-based I/O* (`readMode=1`), a background acquisition task begins sampling into a fixed-depth ring buffer at object construction time, independently of when (or whether) the caller ever calls `read()`. This distinction is not merely an implementation detail. Chapter 5 documents how it produced a measurement artefact large enough and stable enough to be repeatedly and reasonably misattributed to genuine vehicle physics before its true cause was found.

2.7 The Sim-to-Real Gap

The term *sim-to-real gap* (or *reality gap*) is most extensively studied in the deep-reinforcement-learning literature, where it refers to the degradation of a policy’s performance when a controller trained purely in simulation is transferred to a physical robot [25]. This thesis borrows the term for a broader, classically-engineered sense not specific to any learned policy: any systematic discrepancy between a robot’s simulated and physical behaviour under nominally identical software and commands. Sources of this gap are commonly grouped into a few broad categories, all of which recur, in concrete and specific form, across this thesis’s methodology chapters:

- **Unmodelled or under-modelled physical effects** – e.g. static friction / stiction, actuator response lag, or non-linear actuator-to-output mappings that a simulator either omits entirely or models only in idealised form. Chapter 5’s throttle calibration and friction-deadband findings are a direct instance of this category.
- **Sensor and estimation differences** – simulated sensors are frequently noiseless or near-noiseless and perfectly time-synchronised with the simulation clock, whereas physical sensors carry real noise, real latency and, in a distributed system, real inter-process clock skew. Chapter 4’s clock-offset and LiDAR angle-convention fixes fall into this category.
- **Software/communication-layer differences** – a simulated robot typically runs its entire software stack as co-located ROS 2 processes on one machine. A physical robot may be split across incompatible middleware versions, unreliable wireless links, or, as found directly in this project, firmware- or driver-level buffering that a simulation has no equivalent of at all. Chapters 4 and 5 both document findings in this category.

A methodological point recurs throughout this thesis and is made explicit here because it shapes every later chapter’s approach: these categories are not always distinguishable from the symptom alone. A fixed-duration, speed-independent delay between a software command and its physical effect is, from a black-box measurement standpoint, equally consistent with several explanations. It could be drivetrain coasting, a physical-effect explanation, or a command-delivery lag in the bridge software, a communication-layer explanation. Or, as this thesis’s own investigation ultimately found, it could be a stale-data artefact in a sensor read path – a sensor/estimation-layer explanation that happens to mimic a physical one almost exactly.

Distinguishing between these required, in every case documented in this thesis, going beyond the symptom to a direct measurement or source-level inspection of the specific layer under suspicion. This is a theme Chapter 5 returns to explicitly.

3 Simulation Stack Development

This chapter documents the development of the simulation-side navigation stack that Chapters 4 through 6 later migrate onto physical hardware. The stack went through two full drive-architecture migrations before reaching the form used for hardware bridging (Sections 3.2–3.4). Along the way, it surfaced a four-part vehicle-dynamics bug pattern that recurred during testing of this drive architecture (Section 3.5). The chapter closes with a behaviour-tree fix specific to Ackermann-steered vehicles (Section 3.6).

3.1 Diagnostic Standard: Verification by Direct Measurement

The empirical standard applied to every discrepancy in this thesis was established in this chapter. Section 3.5 documents a four-part bug pattern encountered during testing of this drive architecture. None of its components were distinguishable by static inspection of the URDF or launch configuration alone. A simulated vehicle in freefall, one frozen inside a crashed physics server and one driving its wheels at complete slip with zero chassis motion all look superficially identical.

Diagnosing each required directly comparing `/odom` position over several seconds of sustained commanded motion against the Gazebo server log, rather than reasoning from configuration alone. This became the project’s standard verification method for any subsequent change to the simulation stack (Section 3.5.1). The same standard carried into the hardware validation phase of Chapters 4–6.

3.2 Baseline Manual Bicycle-Model Inversion

Early experimentation approached the QCar as a generic four-wheeled robot with front-wheel steering, controllable in much the same way as a differential-drive platform. This was the simplest model to get the vehicle moving at all. It became clear soon after that this simplification did not hold: the QCar is genuinely Ackermann-steered, and a differential-drive-style controller has no way to express the coordinated left/right hub-angle split that Ackermann steering requires. This is what motivated computing a proper Ackermann inversion by hand, described next, as this chapter’s first documented drive architecture.

The navigation stack’s earliest drive architecture used two independent `ros2_control JointGroup` controllers (`drive_controller` for wheel velocity, `steering_controller` for hub position) driven by a hand-written node, `cmd_vel_to_drive.py`, that manually inverted the bicycle-model relationship of Equations 2.1–2.3 to compute a per-wheel velocity and steering-hub angle from an incoming `geometry_msgs/Twist` on `/cmd_vel`. This architecture worked, but it duplicated the Ackermann inverse-kinematics logic that a purpose-built controller would otherwise provide. More importantly for a navigation stack meant to reach physical hardware, it could not support Nav2’s recovery behaviours that require reversing, since the manual node had no reverse-driving logic of its own.

3.3 First Migration to `ros2_control`'s `AckermannSteeringController`

The first architectural migration replaced both custom `JointGroup` controllers and `cmd_vel_to_drive.py` with `ros2_control`'s own `ackermann_steering_controller` / `AckermannSteeringController` plugin, configured via `config/qcar_controllers.yaml`. This controller performs the same Ackermann inverse kinematics internally and additionally publishes its own wheel-based odometry. This project deliberately adopted that odometry as the vehicle's `/odom` source, in place of Gazebo's `gazebo_ros_p3d` ground-truth plugin (the latter was kept running, but remapped to `/ground_truth/odom` for debugging comparison only). This was a considered choice, not a default: ground-truth odometry is trivially available in simulation but has no equivalent on the physical robot, whose odometry can only ever be a real, drift-prone dead-reckoning estimate (Section 4.4). Developing and tuning the navigation stack against the same *kind* of odometry it would have to work with on hardware, from the start, avoided deferring that discrepancy to the eventual hardware migration.

`Nav2`, `AMCL` and `bt_navigator` all hard-code an expectation that odometry arrives on `/odom` and driving commands are consumed from `/cmd_vel`. The new controller instead exposes these under its own namespaced topics (`~/odometry`, `~/reference_unstamped`), so two `topic_tools relay` nodes were added to the launch files to bridge the naming mismatch without modifying `Nav2` itself.

3.3.1 A Wheel-Axis Bug Exposed by the Migration

The migration exposed a latent URDF defect that had been silently compensated for by the old manual node and had therefore never produced a visible symptom before. The rear-left drive wheel joint (`base_wheelrl_joint`) had its `<origin>` yawed by 180° for correct mesh mirroring, but retained the *same* `<axis>` direction as the corresponding right-side joint – which, combined with the yaw flip, silently inverted that wheel's effective rotation sense relative to a positive commanded velocity. The old `cmd_vel_to_drive.py` had manually compensated for this by commanding `[-speed, +speed]` to the left/right wheel velocities respectively. The new controller had no such compensation and computed wheel odometry that did not match the vehicle's real motion direction.

The fix corrected the `<axis>` declaration on the affected joint directly, rather than perpetuating a sign-flip workaround downstream in every consumer of that joint. It illustrates a pattern worth stating explicitly: a URDF geometry defect that is silently absorbed by one consumer's ad hoc compensation will resurface, often with a confusing symptom, the moment that consumer is replaced.

Listing 3.1 shows the two rear drive-wheel joints as they stand in the corrected URDF: the left joint's mesh-mirroring yaw (`rpY="0 0 3.14"`) is paired with a negated `<axis>` so that a positive commanded wheel velocity turns both wheels in the same physical sense, whereas the right joint needs no such compensation since its origin carries no mirroring rotation.

```
1 <joint name="base_wheelrl_joint" type="continuous">
2   <parent link="base" />
3   <child link="wheelrl" />
4   <origin rpy="0 0 3.14" xyz="-0.12765 0.05610 0.03338" />
```

```

5      <!-- axis negated to compensate for the 180deg yaw above (mesh mirror) -
6           without this the left wheel spins opposite the right wheel for the
7           same signed velocity command -->
8      <axis xyz="0 -1 0" />
9  </joint>
10
11 <joint name="base_wheelrr_joint" type="continuous">
12   <parent link="base" />
13   <child link="wheelrr" />
14   <origin rpy="0 0 0" xyz="-0.12765 -0.05610 0.03338" />
15   <axis xyz="0 1 0" />
16 </joint>

```

Listing 3.1: Rear-left and rear-right drive-wheel joints (`urdf/qcar_model.xacro`) – the axis on the mirrored left joint is negated to compensate for its 180° origin yaw.

Why the sign flip happens: axes live in the joint’s own, rotated frame. The mechanism behind this bug is a direct consequence of one URDF convention that is easy to overlook: a joint’s `<axis>` vector is expressed in the joint’s *own* local frame, not directly in the parent link’s (here, `base`’s) frame. That local frame is produced by applying the joint’s own `<origin>` rotation to its parent link’s frame. The rear-right joint’s origin carries no rotation (`rpy="0 0 0"`), so its joint frame is identical to `base`’s frame and `axis="0 1 0"` means exactly $+Y$ in the vehicle’s own reference frame. The rear-left joint’s origin, in contrast, is yawed 180° about Z (`rpy="0 0 3.14"`) so its mesh renders mirrored on the correct side of the chassis. A 180° yaw about Z maps a frame’s own X and Y axes to their negatives while leaving Z unchanged ($X \rightarrow -X$, $Y \rightarrow -Y$), so that joint’s *local* $+Y$ direction already points along `base`’s $-Y$ direction before the `<axis>` element is even considered. Declaring `axis="0 1 0"` on the rear-left joint looks, by simple inspection, like “the same axis as the right wheel.” It therefore actually commands rotation about `base`’s $-Y$ – the opposite physical sense from the right wheel’s `base`-frame $+Y$ – for the identical signed joint velocity.

Negating it to `axis="0 -1 0"` (Listing 3.1) cancels the already-applied yaw, restoring both wheels to rotating about the same `base`-frame axis for the same commanded sign. The fix is not “pick whichever sign makes it drive forward” by trial and error – it is a direct, derivable consequence of the joint’s own origin rotation, which is why it generalises immediately to any other mirrored-mesh joint pair in this or a similar URDF. This particular compensation was specific to the URDF as it stood at this stage of the project; it was later reverted once the project reached its final, direct-drive architecture (Section 3.4).

3.4 Second Migration to the Direct Drive Plugin

The navigation stack’s final drive architecture is the one this thesis’s hardware bridge (Chapter 4) targets. It is also the configuration this thesis’s own results are ultimately built on. It drops `ros2_control` entirely, driving the robot instead through `libgazebo_ros_ackermann_drive.so` directly, declared in a `<gazebo>` block in the URDF rather than through a separate controller-manager configuration file. This is the architecture that both preceding migrations were, in

hindsight, steps toward. The manual bicycle-model inversion of Section 3.2 proved the Ackermann kinematics were necessary at all. The first migration of Section 3.3 proved that a purpose-built controller could own that kinematics and its own odometry. This migration removes the last piece of custom controller-manager configuration standing between the URDF and the vehicle’s actual behaviour in simulation.

This plugin subscribes to `/cmd_vel` and publishes `/odom` and the `odom` $\rightarrow$ `base` TF transform directly, again auto-computing wheelbase, track width and wheel radius from URDF joint and collision geometry at spawn time. Doing so removes an entire category of configuration drift risk, where a controller’s stated geometry parameters could silently fall out of sync with the URDF’s actual geometry – something that had been possible, if not observed, under the first migration’s `ros2_control` YAML-based configuration. This is the architecture described in Section 2.3’s account of the drive plugin and the one whose real-world calibration Chapter 5 presents.

3.4.1 A Quiet Regression Where Odometry Reverts to Ground Truth

This migration has one consequence worth stating explicitly, because it is easy to overlook and was not caught at the time: it silently reversed the odometry-source decision made in Section 3.3. Reading `gazebo_ros_ackermann_drive`’s own `UpdateOdometryWorld()` implementation directly shows that its `/odom` is computed from `model_->WorldPose()` and `model_->WorldLinearVel()` – the physics engine’s own exact, noise-free simulated pose and velocity for the whole vehicle body – not an integration of individual wheel-joint velocities or steering angles at all. In other words, the final simulation-side drive architecture publishes `/odom` that is, mechanically, no different from what `gazebo_ros_p3d` (Section 3.3) would have provided: perfect, driftless ground truth, despite carrying the same topic name and message type the first migration deliberately chose *instead of* ground truth.

This is not a bug in the sense the rest of this chapter uses the word. The plugin behaves exactly as its own source dictates. Nothing in this project’s testing depended on `/odom` actually drifting in simulation.

It is, however, a genuine and previously unrecorded discrepancy between this thesis’s stated intent – training against the same *kind* of odometry the physical robot will have – and what the final simulation configuration actually exercises. It is worth stating plainly rather than leaving the earlier claim uncorrected: **only the physical QCar’s own odometry (Chapter 4, Section 4.4) is genuine encoder-based dead reckoning in this project; the simulation’s final `/odom` is ground truth, exactly like the `gazebo_ros_p3d` source it was originally meant to replace.**

Framed in terms of Section 2.7’s sim-to-real gap taxonomy, this means the localisation/odometry accuracy gap between simulation and hardware in this project is not purely an empirically discovered discrepancy. It is, in part, structurally guaranteed by this architecture, since the simulated AMCL/Nav2 stack is given a perfect motion estimate that its real-hardware counterpart can never have. Any future comparison of localisation accuracy between simulation and hardware should account for this, rather than treating the two `/odom` streams as like for like.

3.5 The “Robot Not Moving Forward” Bug Pattern

Testing this drive architecture (Section 3.4) surfaced a recurring symptom: the simulated vehicle did not drive forward under a sustained `/cmd_vel` command. This single symptom turned out to be caused by a stack of independent bugs, each one masking the next until the previous was fixed. The pattern recurred closely enough, across separate testing on this plugin/vehicle combination, to warrant treating it as a general defect signature rather than an isolated incident. Table 3.1 summarises the bugs found.

Table 3.1: The recurring “not moving forward” bug pattern found while testing this drive architecture.

#	Bug	Symptom / mechanism
1	Wheel <code><collision></code> was the visual STL mesh, not a primitive shape	The plugin’s radius auto-detection only supports cylinder/sphere collisions; a mesh collision silently returns radius 0, so <code>target_linear / wheel_radius</code> becomes infinite, poisoning the velocity PID regardless of gain. Fixed by keeping the STL for <code><visual></code> but using an explicit <code><cylinder radius="0.036" length="0.0245"/></code> for <code><collision></code> .
2	Zero Coulomb friction on the wheel links	Even with a correct wheel radius, the wheel spins at exactly the commanded angular velocity forever while the chassis never translates – 100% slip. Fixed with <code><mu1>1.0</mu1><mu2>1.0</mu2></code> on every wheel link.
3	Absurd ground-plane friction (<code>mu=100, mu2=50</code>)	Once wheel friction is fixed, this combination produces a real physics instability/NaN blow-up on contact. Fixed by lowering to a realistic <code>mu=1, mu2=1</code> .
4	Razor-thin wheel/ground clearance (only ~ 0.38 mm geometric overlap at the original 0.033 m collision radius)	Enough overlap to rest on the ground without falling through, but not enough consistent contact-normal force to generate real traction even at correct friction – chassis twist stayed near zero despite full wheel spin. Fixed by padding the collision cylinder radius to 0.036 m (+3 mm), which alone flipped the vehicle from “spins in place” to sustained forward motion.

Listing 3.2 shows the corrected form of one rear wheel link and its friction declaration, combining the fixes for bugs 1 and 2 from Table 3.1: an explicit cylinder `<collision>` geometry (the `<visual>` mesh, omitted here, is unaffected and keeps using the STL) and a separate `<gazebo reference="...">` block assigning Coulomb friction coefficients to that same link. URDF has no wheel-friction concept of its own, so this Gazebo-specific extension is the only place that value is set at all.

```

1 <link name="wheelr1">
2   <collision>
3     <origin rpy="1.5708 0 0" xyz="0 0 0" />
4     <geometry>
5       <cylinder radius="0.036" length="0.0245" />
6     </geometry>

```

```

7     </collision>
8     <!-- <visual> (STL mesh) and <inertial> omitted here, unaffected -->
9 </link>
10
11 <!-- wheel-ground friction - without this the wheels spin freely with 100%
12     slip and the chassis never translates, regardless of PID gains -->
13 <gazebo reference="wheelrl">
14     <mu1>1.0</mu1>
15     <mu2>1.0</mu2>
16 </gazebo>

```

Listing 3.2: Wheel collision cylinder and friction (`urdf/qcar_model.xacro`) – fixes for bugs 1 and 2 in Table 3.1.

Mechanism: how bugs 1 and 2 poison the control loop and physics, in numbers.

Section 2.3.1 and Section 2.3.2 give the general theory bugs 1 and 2 exercise [2]. Concretely, for this vehicle, a commanded 0.3 m/s with a mesh-shaped (bug 1, radius $\rightarrow 0$) rear-wheel collision computes a target angular velocity of $0.3/0 \rightarrow \infty$ rad/s in Equation 2.7’s error term. This is not a large but finite number that a differently-tuned PID could still track – it is a value with no finite representation at all, driving the linear-velocity PID’s output straight to its saturation limit on the very first physics step, regardless of gain. With the corrected 0.036 m radius (Listing 3.2) that same command instead computes a finite target of $0.3/0.036 \approx 8.33$ rad/s, a well-posed, trackable error signal.

Bug 2 (zero friction) then governs whether that correctly-spinning wheel produces any chassis motion at all. With $\mu_1 = \mu_2 = 0$, Section 2.3.2’s Coulomb limit permits zero tangential contact force, regardless of how much normal force the vehicle’s weight presses the wheel into the ground with. The wheel can therefore spin at exactly the commanded 8.33 rad/s indefinitely while contributing zero net force to the chassis.

The two bugs fail at two different, independently necessary conditions: a well-posed control error and a nonzero force limit to actually convert correct wheel spin into chassis thrust. This is why fixing either one alone still left the vehicle stationary. Only fixing both together produced motion.

Testing also surfaced a preceding, more basic defect: mesh URIs (`package://.../models/qcar/QCarBody` style paths) pointed at the wrong directory layout, rather than the correct `meshes/` layout and file names. As a result, the robot had *no* collision geometry at all and fell through the simulated ground plane – confirmed directly via `/odom` tracking a *z*-position reaching approximately $-38,000$ m.

This is worth recording as a separate, more fundamental lesson: Gazebo Classic’s URDF-to-SDF loader silently rewrites `package://` mesh URIs to `model://`, which only resolves via the `GAZEBO_MODEL_PATH` environment variable. Since the package’s `package.xml` did not export the `<gazebo_ros gazebo_model_path=.../>` tag required for that resolution, correcting the path string alone would not have been sufficient. The fix instead used xacro’s `$(find)` substitution to build `file://` URIs directly, which bypass package-path resolution entirely.

Further testing on this drive architecture also required a numerical-stability fix not initially

anticipated: wheel rotational inertia values that were physically correct for a solid disk of the wheel’s real mass and radius, but small enough in absolute terms to interact badly with the plugin’s 100 Hz PID control loop, producing an `Ogre::AxisAlignedBox::setExtents` assertion crash a few seconds into driving. Bumping wheel inertia roughly $20\times$ (`ixx=izz=0.0012`, `iyy=0.0022`), together with lowering the steering and linear-velocity PID gains, resolved the crash and remained stable throughout subsequent testing.

A further gain-tuning finding is worth stating as a general caution for reusing PID values across otherwise-identical vehicle configurations. A steering PID gain verified stable and accurate on a *frictionless* wheel configuration produced a real turning radius roughly $6\times$ larger than commanded, once realistic wheel friction (bug 2 above) was added. The same low gain that had previously been sufficient, with no resistive torque to overcome, could no longer turn the front hubs against a now-gripping, rolling tyre.

The fix raised `left/right_steering_pid_gain` from 0.02 to 3.0. This was verified directly via TF lookups on the steering-hub joints during a commanded 1.0 m-radius turn, confirming hub angles converging to the correct $\sim 15^\circ$ – 18° Ackermann inner/outer split. A circle fit to the resulting odometry trace recovered a turning radius of 0.87 m against the 1.0 m commanded – a reasonable margin, given the real-vs-ideal gap between the single-track model of Section 2.2 and the vehicle’s true four-wheel geometry.

3.5.1 Diagnostic Method

A methodological point generalises from this bug pattern: none of these four bugs are visible from `ros2 topic list` or a static inspection of the URDF alone. A simulated robot that appears correctly configured can simultaneously be in freefall, frozen inside a crashed-but-still-running `gzserver` process, or driving its wheels at 100% slip with zero real chassis motion. None of these states are distinguishable without directly comparing `/odom` position over several seconds of sustained commanded motion, cross-checked against the Gazebo server log for silent `[Errr]`/assertion lines. This became this project’s standard verification method for any subsequent URDF or physics change to the simulation stack.

3.6 A Behaviour-Tree Fix for Ackermann Recovery

How a behaviour tree actually executes. A behaviour tree is evaluated by repeatedly “ticking” it from the root [4]. Each tick propagates down the tree. Every node it reaches returns one of `SUCCESS`, `FAILURE`, or `RUNNING` (still in progress, re-tick next cycle) back up to its parent. How a parent node interprets its children’s results depends entirely on that parent’s type.

A `RoundRobin` node – the one this section’s fix concerns – ticks exactly one of its children per activation, advancing to the next child and wrapping back to the first, only once the currently-selected child itself returns `SUCCESS` or `FAILURE` rather than `RUNNING`. It cycles through its full child list one at a time across repeated recovery attempts, rather than trying all of them on a single activation.

A `ReactiveFallback` (the parent of `RecoveryActions` in Listing 3.3) instead re-ticks its children from the first one every single tick, moving on to the next only if the current one fails. The two control-flow types are not interchangeable. Nav2’s stock recovery subtree pairs

them deliberately: the outer `ReactiveFallback` re-checks whether a goal update has arrived, via the `GoalUpdated` condition node, before committing to whichever recovery action the inner `RoundRobin` has advanced to.

A separate, functional (rather than purely physics-level) issue appeared once the vehicle was reliably driving: Nav2’s default recovery behaviour tree [11] round-robins through `[ClearCostmaps, Spin(1.57 rad), Wait(5 s), BackUp(0.3 m)]` whenever the active controller reports it cannot make further progress. The `Spin` recovery commands a pure in-place rotation. This is physically impossible for an Ackermann-steered vehicle: its steering angle has no effect while linear velocity is zero, so the vehicle simply cannot rotate without also translating. Roughly one in three recovery attempts was therefore a guaranteed no-op that burned its full timeout without attempting any corrective motion.

The fix was to remove `<Spin>` from the recovery action list in local copies of Nav2’s stock behaviour-tree XML files (`navigate_to_pose_w_replanning_and_recovery.xml` and its through-poses equivalent), wired into the launch configuration via a `RewrittenYaml` substitution. This required one non-obvious step: `RewrittenYaml` only overwrites *existing* leaf keys in a YAML file, it does not add new ones, so the parameter keys pointing at these custom BT file paths (`default_nav_to_pose_bt_xml` / `default_nav_through_poses_bt_xml`) had to already exist as placeholder entries in `nav2_params.yaml` before the launch-time substitution could take effect.

Listing 3.3 shows the resulting recovery fallback from the local behaviour-tree copy: a `RoundRobin` of clearing both costmaps, waiting and backing up – with no `<Spin>` node present at all, compared to the stock tree’s four-way rotation through `[ClearCostmaps, Spin, Wait, BackUp]`.

```

1 <ReactiveFallback name="RecoveryFallback">
2   <GoalUpdated/>
3   <RoundRobin name="RecoveryActions">
4     <Sequence name="ClearingActions">
5       <ClearEntireCostmap name="ClearLocalCostmap-Subtree"
6         service_name="local_costmap/clear_entirely_local_costmap"/>
7       <ClearEntireCostmap name="ClearGlobalCostmap-Subtree"
8         service_name="global_costmap/clear_entirely_global_costmap"/>
9     </Sequence>
10    <Wait wait_duration="5"/>
11    <BackUp backup_dist="0.30" backup_speed="0.05"/>
12  </RoundRobin>
13 </ReactiveFallback>

```

Listing 3.3: Recovery actions in the local behaviour-tree copy (`config/nav2/behavior_trees/navigate_to_pose_w_replanning_and_recovery.xml`) – no `<Spin>` node, unlike the Nav2 stock tree.

Because `RoundRobin` advances exactly one position per activation, dropping `Spin` from a four-child list to this three-child list is not merely “the useless action happens less often.” It removes the guaranteed no-op activation entirely, so every single recovery activation now attempts a physically achievable action. The cycle back to any one specific action – e.g. back to `BackUp` after it was last tried – also completes one activation sooner than it did with four children in the

rotation.

In practice, this project’s own navigation calls exclusively issue Nav2’s `NavigateToPose` action for a single goal pose (Chapter 6). This makes `navigate_to_pose_w_replanning_and_recovery.xml` the behaviour tree actually exercised at runtime. The through-poses equivalent was patched identically for consistency, though it was never itself driven by this project’s own goal-sending code.

The patched `RecoveryFallback` subtree fires whenever the active controller reports it cannot make further progress toward the current goal, for example when the local planner fails to find a usable path around an obstacle. At that point the vehicle clears both costmaps, waits and backs up, in that order. The outer `NavigateToPose` tree then retries planning.

3.7 Summary of the Simulation-Stack Architecture

By the end of this chapter’s work, the simulation-side navigation stack has taken its final form. A URDF-described vehicle is driven directly by `libgazebo_ros_ackermann_drive.so` (Section 3.4), publishing ground-truth-derived `/odom` – a quiet regression from the genuine dead-reckoning odometry the first migration (Section 3.3) had deliberately adopted (Section 3.4.1). Wheel collision geometry, friction and inertia have all been corrected (Section 3.5), PID gains are tuned for both drive and steering and a custom Ackermann-aware recovery behaviour tree is in place (Section 3.6). This is the configuration Chapter 4 treats as the reference to be matched, as closely as the physical robot’s own constraints allow, when bridging to hardware.

4 Real-Hardware Bridge Architecture

This chapter presents the architecture built to connect the simulation-validated navigation stack of Chapter 3 to the physical QCar. Section 4.1 establishes why native ROS 2 communication with the robot’s onboard compute is not available, which motivates the TCP/JSON bridge architecture of Section 4.2. The remaining sections document the sensor and actuator correctness issues found while bringing that bridge up to the same fidelity as the simulation stack: LiDAR angle convention (Section 4.3), odometry (Section 4.4), steering trim (Section 4.5) and inter-machine clock synchronisation (Section 4.6).

4.1 Why Native ROS 2 Communication With the QCar Is Not Available

Two separate constraints rule out installing ROS 2 Humble directly on the QCar’s own compute. Either one alone would have forced the bridge architecture of Section 4.2. Together, they leave no realistic native alternative.

Storage. As described in Section 2.6, the Jetson TX2’s 32 GB eMMC [17] already carries Ubuntu 18.04, ROS 1 Melodic, ROS 2 Dashing and Quanser’s own HAL/PAL software. A second, full ROS 2 Humble installation needs its own client libraries, its own DDS implementation and driver code to bridge ROS 2 to the HAL/PAL layer. Fitting all of that into what remained of the 32 GB was judged unlikely, based on the typical size of a ROS 2 Humble install. This constraint does not depend on whether Dashing and Humble can otherwise talk to each other. Fixing DDS compatibility would not, by itself, free up disk space.

The ROS 2 Dashing / Humble DDS Incompatibility. Independently of storage, this incompatibility was tested directly rather than assumed. The QCar’s onboard Jetson TX2 ships with ROS 2 Dashing (released 2019) pre-installed, while this project’s navigation stack targets ROS 2 Humble (released 2022). Section 2.1 established that ROS 2 nodes communicate via DDS participants that must share a compatible discovery and wire protocol to find each other at all. The working hypothesis at the start of this thesis’s hardware phase was that this would be a soft compatibility problem: degraded performance, perhaps, but not a hard failure.

A minimal ROS 2 publisher (`demo_nodes_cpp talker`) on the QCar and a minimal subscriber (`demo_nodes_py listener`) on the development PC, both configured with the same `ROS_DOMAIN_ID`, never established communication across three repeated attempts. The talker was independently confirmed to be actively publishing each time. The network layer itself was systematically ruled out before concluding this was a middleware-level problem. Raw UDP unicast passed cleanly in both directions, including on the exact Fast DDS discovery port for the domain in use. No firewall was active on either host. An explicit unicast-peers Fast DDS XML profile, which bypasses multicast-based discovery entirely, still failed to achieve discovery. This isolates the failure to the DDS protocol implementation itself: a genuine Fast DDS protocol-version incompatibility between the versions bundled with ROS 2 Dashing and ROS 2 Humble respectively. Each RTPS message identifies the protocol version its sender implements [13]. Two

peers implementing different versions can fail to complete discovery even when the underlying network path is sound. This is not a network, firewall, or configuration problem that could be worked around within ROS 2 itself.

This result rules out one further option that the storage constraint alone would not: running a Humble-compatible container on the QCar instead of a second native install. A Humble Docker container was considered early on. It only partially sidesteps the storage question: a container image adds its own storage overhead rather than removing it. Nor does it address the protocol-version incompatibility at the DDS transport layer at all. The Dashing side of the link is fixed by what physically runs on the vehicle's ROS 2 partition and cannot itself be upgraded without departing from the vendor-supported software image.

4.2 Bridge Architecture Overview



Figure 4.1: A side view of the physical QCar showing the onboard compute and sensing stack that this bridge architecture runs on: the RPLiDAR unit, the Jetson TX2 carrier board (with its wireless antennas and USB peripherals) and the wireless router used for the development PC's network link, all mounted above the chassis and drivetrain.

The architecture built in response uses no ROS 2 at all on the QCar's own compute. Two plain Python processes run directly on the Jetson TX2 (no `colcon` build, invoked as `python3 <script>.py`):

- `qcar_bridge.py` – a TCP server (port 5555) wrapping the Quanser HAL/PAL QCar object (Section 2.6): receives newline-delimited JSON drive/steering commands and responds with odometry JSON over the same full-duplex connection.
- `qcar_lidar_node.py` – a second TCP server (port 5556) wrapping the LiDAR driver, streaming scan data as newline-delimited JSON.

On the development PC, a single node, `qcar_relay_node.py`, is the sole point of contact between ROS 2 and this TCP/JSON protocol. It subscribes to `/cmd_vel` and forwards commands into the motor socket. The same node also publishes `sensor_msgs/LaserScan` on `/scan` and `nav_msgs/Odometry` plus the `odom` $\rightarrow$ `base` TF on `/odom`, from data received back over both sockets. Every other node in the navigation stack – Nav2, AMCL, Cartographer, RViz – interacts with this relay node exactly as it would with the simulation’s own drive plugin. This requires no awareness that the underlying vehicle is real hardware reached over a bespoke protocol rather than a co-located Gazebo simulation.

Listing 4.1 shows the command-receiving side of this protocol inside `qcar_bridge.py`’s main loop: incoming TCP bytes are accumulated into a buffer and split on newlines. Each complete line is then parsed as one JSON object, while a malformed line is logged and discarded rather than allowed to crash the control loop. This is deliberately fail-soft, since this loop is also driving the physical motor and must keep running even if one message is corrupt.

```

1 chunk = conn.recv(4096)
2 if chunk == b'':
3     print('dev PC relay disconnected')
4     break
5 buf += chunk
6 while b'\n' in buf:
7     line, buf = buf.split(b'\n', 1)
8     if not line.strip():
9         continue
10    try:
11        cmd = json.loads(line)
12        target_linear = float(cmd['linear_x'])
13        angular_z = float(cmd['angular_z'])
14        target_steering = compute_steering(target_linear, angular_z)
15        last_cmd_time = time.time()
16    except (ValueError, KeyError, TypeError):
17        print('bad command line, ignoring:', line)

```

Listing 4.1: Newline-delimited JSON command parsing inside `qcar_bridge.py`’s main loop.

Why TCP, why newline-delimited JSON and why one socket per direction. Three protocol design choices underlie Listing 4.1 and are worth making explicit, since each addresses a specific failure mode rather than being an arbitrary convenience. **TCP rather than UDP:** TCP guarantees in-order, complete delivery of every byte sent (retransmitting silently on packet loss) at the cost of unbounded, variable latency under loss, whereas UDP offers low, predictable latency but silently drops or reorders packets under loss with no delivery guarantee at all [16, 15]. For velocity and steering commands, a stale duplicate or a dropped byte silently corrupting the JSON stream is a worse failure than an occasional extra tens of milliseconds of latency – latency that the `CMD_TIMEOUT` watchdog in Section 4.4 already treats as a safety condition regardless of its cause. TCP’s reliability guarantee won out over UDP’s lower-latency-but-lossy profile.

Newline-delimited JSON rather than a fixed-length or length-prefixed binary framing: TCP is a byte stream with no built-in concept of message boundaries, so *some* framing

scheme is unavoidable. A length-prefixed binary format would be marginally more compact, but a plain ASCII delimiter is trivially inspectable with a bare `nc/telnet` session during debugging. This was a real, exercised advantage throughout this project’s development, not a theoretical one. A plain delimiter also requires no shared binary schema to keep in sync between the QCar-side Python and dev-PC-side Python.

This is also why `buf` accumulates across `recv()` calls, rather than assuming one `recv()` returns exactly one message. TCP’s byte-stream semantics make no guarantee that message boundaries align with any particular `recv()` call’s return: a message can legitimately arrive split across two calls, or two messages can arrive concatenated in one call. Listing 4.1’s `while b'\n' in buf` loop handles both cases uniformly, by always operating on whatever has accumulated so far.

One full-duplex socket carrying both directions, rather than two separate one-way sockets for commands and odometry: a single TCP connection is inherently bidirectional, since each endpoint can read and write independently on the same socket. Section 4.4’s later addition of odometry sent back to the relay required no new listening port or connection – only using a direction of the existing socket that had, until then, been opened but left unread.

Figure 4.2 shows the complete command and sensor data path across both machines and both TCP sockets, matching exactly the components and topics named above.

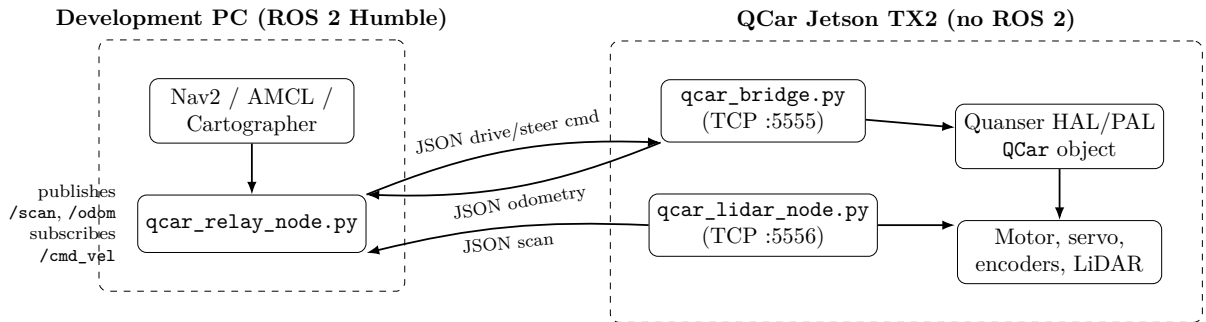


Figure 4.2: Command and sensor data path from the Nav2 stack to the physical actuators and back. Both cross-machine links are newline-delimited JSON over TCP (ports 5555 and 5556); no ROS 2 process runs on the QCar’s own compute at any point in this path.

Both onboard TCP servers were built to handle `SIGTERM` identically to `SIGINT` – a deliberate correctness fix, not a default. Python does not automatically translate a plain `pkill`’s default `SIGTERM` into a caught, clean-exit signal the way it does for `SIGINT` (Ctrl-C). An earlier plain `pkill` against the LiDAR process left the physical LiDAR unit spinning after the owning process had already exited – both a battery drain and a safety concern for anyone near the vehicle. Section 4.7 presents the resulting documented shutdown procedure in full.

4.3 LiDAR Angle-Convention Correction

The first sensor-correctness issue found while bringing up visualisation of the bridged data (`robot_state_publisher` + `joint_state_publisher` + RViz, no Gazebo) was a LiDAR angle-binning defect in `qcar_lidar_node.py`: obstacles rendered in RViz did not appear at their true bearing relative to the rendered robot model. This required *two* stacked, independent corrections,

not one – a clockwise-to-counter-clockwise direction flip *and* a $+90^\circ$ rotational offset between the LiDAR’s zero-reference bearing and the vehicle’s own forward direction. The first attempted fix (direction only) produced a result that was itself the tell that a second error remained. It exactly mirrored the original error about the vehicle’s forward axis, rather than correcting it – the expected signature of fixing only a sign error while a separate rotational offset error is still present. The final, verified-correct transform is

$$\theta_{\text{target}} = \text{wrap}_{(-\pi, \pi]} \left(-\theta_{\text{lidar}} + \frac{\pi}{2} \right), \quad (4.1)$$

This transform was verified against a real obstacle at a known bearing (the vehicle’s physical right side). The check used the rendered robot model’s own orientation in RViz, not the screen’s raw left/right, since screen left/right depends on the RViz camera’s own facing direction and is not a reliable reference on its own.

Listing 4.2 shows Equation 4.1 as implemented in `qcar_lidar_node.py`: each raw (`angle`, `distance`) sample from the HAL is transformed, wrapped into $(-\pi, \pi]$ via `atan2` and binned onto the fixed 360-sample grid a ROS 2 `LaserScan` message requires. The HAL does not guarantee its raw samples already arrive pre-gridded in order, so this binning step is necessary independently of the angle correction itself.

```

1 increment = (2.0 * math.pi) / NUM_MEASUREMENTS
2 ranges = [float('inf')] * NUM_MEASUREMENTS
3 for angle, dist in zip(lidar.angles, lidar.distances):
4     target = -angle + math.pi / 2
5     normalized = math.atan2(math.sin(target), math.cos(target))
6     idx = int(round((normalized + math.pi) / increment)) % NUM_MEASUREMENTS
7     if RANGE_MIN <= dist <= RANGE_MAX:
8         ranges[idx] = float(dist)

```

Listing 4.2: LiDAR angle correction and grid binning (`qcar_lidar_node.py`).

Worked example: why a direction flip alone is not enough. Equation 4.1’s two-part correction is easiest to see by tracing a few canonical raw LiDAR angles θ_{lidar} through the transform $\theta_{\text{target}} = \text{wrap}(-\theta_{\text{lidar}} + \pi/2)$ in Table 4.1. A *direction-flip-only* correction ($\theta_{\text{target}} = -\theta_{\text{lidar}}$, no $+\pi/2$ term) would map the same four raw angles to $0, -\pi/2, \pi, \pi/2$ respectively – every value rotated exactly 90° short of the fully corrected column. This is precisely the signature Section 4.3 describes: a direction-only fix mirrors the error about the vehicle’s forward axis rather than removing it. Negating the angle alone corrects the sense of increasing angle (clockwise vs. counter-clockwise), without also correcting *where zero itself points* – two logically independent properties of an angular convention. This is exactly why this bug needed two independent corrections, not one.

Table 4.1: Illustrative raw-to-corrected LiDAR angle mapping (Equation 4.1).

θ_{lidar} (raw)	0	$\pi/2$	π	$-\pi/2$
θ_{target} (corrected)	$\pi/2$	0	$-\pi/2$	π

4.4 Wheel Odometry over the Bridge

Mapping (Cartographer, Section 2.4) was successfully brought up on real hardware using scan-matching alone, with `use_odometry: false / provide_odom_frame: true` in the Cartographer configuration – meaning the initial bridge implementation required no odometry extension at all for mapping to work end-to-end. Localisation (AMCL) does not share this property. Unlike Cartographer’s self-contained scan matching, AMCL looks up a live `odom → base` TF at every scan callback as its motion-model input. As a result, the wheel-odometry extension originally scoped only for the mapping phase became a hard requirement once navigation (Nav2/AMCL) was brought up.

`qcar_bridge.py` was extended with an `Odometry` class implementing the same bicycle-model dead-reckoning integration as Equations 2.1–2.3. The resulting odometry is sent back to the relay over the same already-full-duplex TCP connection the drive commands arrive on; the reverse direction of this connection had previously been opened but its data discarded. `qcar_relay_node.py` parses this stream and publishes both `/odom` and the corresponding TF. Tested in isolation before layering Nav2 on top, the resulting odometry published at approximately 45 Hz. A 3-second commanded straight-line drive at 0.3 m/s produced $x \approx 1.25$ m, $v \approx 0.37$ m/s, $y \approx 0$ – consistent with straight-line motion at close to the commanded speed. This confirmed the odometry was working before the full navigation stack was brought up on top of it.

Listing 4.3 shows the `Odometry` class in full. `update()` is called once per control-loop cycle with the raw encoder count and the current commanded steering angle and directly implements the bicycle-model relationship of Equations 2.1–2.3. Encoder-count deltas become a travelled distance via the fixed `METERS_PER_COUNT` conversion, from which velocity and yaw rate follow; x , y and yaw are then integrated forward exactly as in the continuous-time model.

```
1 class Odometry:
2     def __init__(self):
3         self.x = 0.0
4         self.y = 0.0
5         self.yaw = 0.0
6         self.last_encoder = None
7
8     def update(self, encoder_counts, steering, dt):
9         if self.last_encoder is None:
10             self.last_encoder = encoder_counts
11         delta_counts = encoder_counts - self.last_encoder
12         self.last_encoder = encoder_counts
13
14         distance = delta_counts * METERS_PER_COUNT
15         v = distance / dt
16         yaw_rate = v * math.tan(steering) / WHEELBASE
17
18         self.yaw += yaw_rate * dt
19         self.x += distance * math.cos(self.yaw)
20         self.y += distance * math.sin(self.yaw)
21
```

```

22     return {'x': self.x, 'y': self.y, 'yaw': self.yaw,
23            'v': v, 'yaw_rate': yaw_rate}

```

Listing 4.3: Bicycle-model dead-reckoning odometry (`qcar_bridge.py`).

From raw encoder counts to metres. `METERS_PER_COUNT` (documented in `qcar_bridge.py` as $\frac{0.01977}{2880}$) is itself a small derived constant, not a directly measured one. 2880 is the number of raw encoder counts per one full revolution of the motor shaft, as documented in the Quanser HAL manual [17] for this drive motor/encoder pairing. 0.01977 m is the linear distance one rear-wheel revolution advances the vehicle – the wheel’s rolling circumference, folding in the fixed gear ratio between motor shaft and wheel. Their ratio converts a raw encoder delta directly into a travelled distance in one multiplication, with no separate gear-ratio or wheel-circumference term needed at the call site.

The integration is forward Euler, not an exact arc. Line-by-line, `update()` performs a standard forward-Euler discretisation of Equations 2.1–2.3. It advances `self.yaw` by `yaw_rate * dt` *first*, then immediately uses that already-updated yaw to project the travelled `distance` onto *x* and *y* via $\cos(\text{yaw})/\sin(\text{yaw})$. This effectively assumes the vehicle’s heading was already at its new value for the entirety of this cycle’s straight-line displacement, rather than integrating the true curved arc the vehicle followed while turning during the interval.

This is a standard, deliberate simplification, matching how Chapter 3’s simulation-side plugin also integrates its own motion incrementally rather than analytically per step. Its error shrinks toward zero as the control loop’s period `dt` shrinks, but it is not exact for any nonzero `yaw_rate * dt`. At this project’s 50 Hz control rate (Section 4.2), `dt` is small enough that the resulting error is negligible next to this thesis’s other, much larger sources of odometry inaccuracy, such as steering trim and throttle calibration. It is still worth naming explicitly as a non-zero, if small, contributor to dead-reckoning drift over a long enough drive.

4.5 Steering-Trim Calibration

A small but consistent steering bias was present on this specific chassis: the vehicle drifted laterally under a nominally straight, zero-steering command. The physical steering linkage could not be adjusted to correct it mechanically. The fix was implemented in software instead, as a constant offset (`STEERING_TRIM`, an early estimate of -0.08 rad, later refined to -0.087 rad in Section 4.5.1 below) applied only at the point the steering command is written to the servo. Calibration proceeded in two stages: an initial static test (motor off, holding candidate angles and judging visually against a reference, preserved as `calibrate_steering.py` for future re-calibration) narrowed the value to roughly the right range. A series of real straight-line driving tests (drive a known distance, measure the resulting lateral offset, compute $\delta_{\text{trim}} = \arctan(\text{offset}/\text{distance})$) refined it further. This data carried real measurement noise. Lateral offset depends on the vehicle’s initial aim at the start of each test run, not purely on trim, so the calibration deliberately did not over-fit a precise value from only two data points. A consistent directional nudge toward zero observed drift converged faster in practice than attempting a precise interpolation.

This first-pass value was accepted once a 4.4 m test drive showed no reported drift. It is described as “almost accurate,” a deliberate acceptance of a small residual rather than a claim of exact zero. As the next section shows, it was in fact only half of the real correction this vehicle needed.

4.5.1 Steering Gain as a Second, Independent Nonlinearity

The trim calibration above corrects only the steering command’s *offset* – what the vehicle does at a commanded angle of zero. It says nothing about whether a nonzero commanded angle produces the correct amount of real wheel deflection. For a long period of this project nobody had verified that separately. The gap surfaced directly during Nav2 integration (Chapter 6): I reported that autonomous navigation on real hardware was “not turning as well as sim,” a complaint that a pure offset error cannot explain, since an offset only shifts the zero-point and leaves proportionality untouched.

Steering-gain test. With a fresh battery fitted and the test speed raised to 0.4 m/s, confirmed via logged velocity to sit safely above the stiction floor, a seven-point sweep of commanded steering angles was driven, each a fresh 1 m arc from a consistent physical start pose. `STEERING_TRIM` was held at zero throughout, so the commanded value was exactly what reached `car.write()`. The real wheel angle implied by each measured lateral deviation was back-solved from the same circular-arc geometry as the trim calibration above ($R = L / \tan \delta$, $y = R(1 - \cos(s/R))$, wheelbase $L = 0.257\,25$ m), using bisection rather than a closed-form inverse since the forward relationship is transcendental in δ . Table 4.2 presents the results.

Table 4.2: Commanded steering angle versus implied real wheel angle (`STEERING_TRIM=0`).

Commanded angle (rad)	Implied real angle (rad)
0.00	+0.112
−0.05	+0.052
−0.08	+0.018
−0.085	+0.005
−0.09	0.000
−0.10	−0.021
−0.12	−0.062

The relationship is clean, monotonic and shows no sign flips or stalls. A least-squares linear fit, $\delta_{\text{real}} = m \delta_{\text{cmd}} + c$, gives $m = 1.39$, $c = 0.119$ rad, $R^2 = 0.977$ (the two largest-magnitude commands sit slightly above the fitted line, a mild hint of additional curvature at large angles not investigated further). Two conclusions follow directly. The true zero-point trim is $\delta_{\text{cmd}} = -0.09$, not the -0.08 found by the coarser static test above, pinned by direct interpolation between the -0.085 (implied $+0.005$) and -0.09 (implied 0.000) rows. Independently of the trim value entirely, the real wheel also turns approximately 39% *further* than the kinematic model’s commanded angle implies for any non-zero command. This is a genuine multiplicative gain error, not an offset and one that a pure trim correction cannot fix. This gain error is a strong candidate explanation for the original complaint that motivated the investigation. Nav2’s planner and MPPI controller

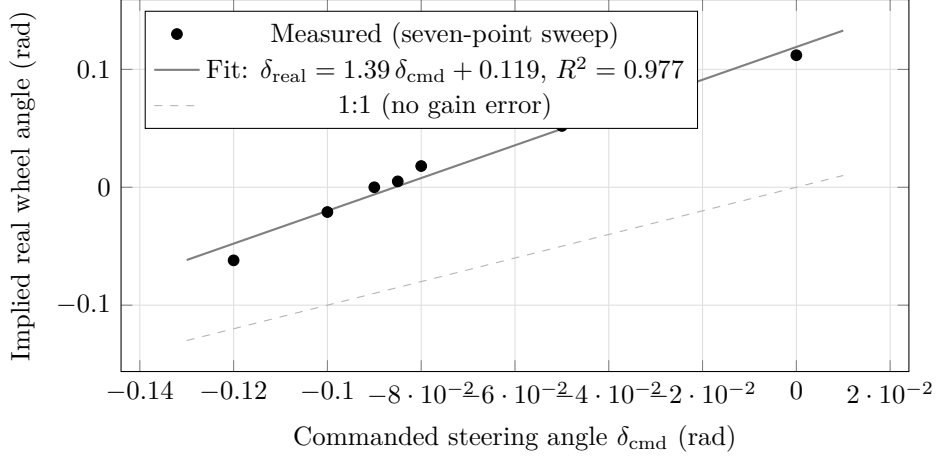


Figure 4.3: Commanded versus implied real steering angle (Table 4.2), with the fitted regression line against the 1:1 relationship a gain-free kinematic model would predict. The fitted slope of 1.39 is the multiplicative gain error itself; the vertical offset between the two lines at $\delta_{\text{cmd}} = 0$ separately reflects the zero-point trim before it was corrected.

both assume the kinematic model of Section 2.2.1 maps 1:1 onto real wheel angle. A vehicle that over-rotates by 39% for every commanded turn would visibly overshoot turns and cut corners tighter than planned, exactly the symptom reported.

Fix and validation. A documentation check first confirmed that no factory steering calibration exists to consult. The Quanser hardware manual [17] documents only the raw servo command range and the physical maximum wheel angle (Table A.1). The HAL’s own `steeringBias` construction parameter was confirmed left at its default of zero in `qcar_bridge.py` – nothing at the firmware level was fighting or duplicating the software correction. `STEERING_GAIN = 1.39` (Listing 4.4’s `apply_trim()`, which pre-divides the kinematic angle by this constant before adding `STEERING_TRIM`) was set directly from this regression. The trim itself was then refined further, from three repeated trials each at commanded -0.08 , -0.085 and -0.09 (to average out human placement and measurement noise): the zero-crossing of the three averaged measurements gave a final `STEERING_TRIM = -0.087`. A dedicated validation drive commanded a real kinematic turn of 0.15 rad over a 1 m arc at 0.4 m/s, predicting 28.5 cm of lateral deflection. The measured deflection was 27 cm, a residual error of $\approx 5.6\%$, down from $\approx 39\%$ with no gain correction at all. The fix was subsequently confirmed in genuine closed-loop use. After re-running autonomous Nav2 navigation on the physical vehicle post-deployment, my direct assessment was that turning behaviour had visibly improved. This is consistent with the gain error having been the dominant cause of the original complaint rather than merely a contributing factor.

As with the throttle calibration constants of Section 5.5.3, both `STEERING_GAIN` and `STEERING_TRIM` are per-unit values, expected to differ on a different physical chassis. The empirical sweep-and-regression *method* used to find them is the transferable result, not the specific numbers. One operational caution from this investigation is also worth recording: testing a previously untested steering-angle magnitude can produce more real curvature than expected. Available clearance should be confirmed in the new command’s actual curve direction before testing it, not only in directions already validated at smaller magnitudes.

4.5.2 A Trim-Placement Bug and the “Apply at the Actuator Boundary” Principle

The first implementation of this trim contained a real bug worth recording as a general principle for any per-unit hardware calibration constant, not only this one. It folded the trim into the *same* steering value used both for the real servo command *and* for the odometry dead-reckoning calculation in Section 4.4. The entire purpose of the trim is to make the real wheels read approximately zero deflection despite a nonzero servo command. Feeding that same trimmed value into odometry made the dead-reckoning model believe the wheels were deflected by the trim amount, even while the vehicle was in reality driving dead straight. A live test briefly exposed this directly: a straight-line drive of approximately 2.8 m produced a fictitious $y \approx -1.72$ m in the odometry output, a spurious $\sim 30^\circ$ curve that did not correspond to any real vehicle motion.

The fix separated `compute_steering()` (returns the untrimmed, kinematic steering angle, used for odometry) from a new `apply_trim()` (computes the real servo command, called only immediately before the HAL write), shown together in Listing 4.4. `target_steering` is `compute_steering()`’s untrimmed return value. It is also what Listing 4.3’s `odom.update()` call is given. `apply_trim()` is invoked only once, at the `car.write()` call site itself.

```
1 def compute_steering(linear_x, angular_z):
2     # Returns the KINEMATIC (untrimmed) angle - used for odometry.
3     # STEERING_TRIM must NOT be folded in here (see prose above).
4     if abs(linear_x) < 1e-2:
5         steer = 0.0
6     else:
7         steer = math.atan(WHEELBASE * angular_z / linear_x)
8     return max(-MAX_STEER_CMD, min(MAX_STEER_CMD, steer))
9
10
11 def apply_trim(kinematic_steering):
12     # Converts a real/kinematic steering angle into the servo command
13     # that achieves it on this vehicle - call only right before
14     # car.write(), never upstream of odometry or anything else that
15     # treats its input as the true physical wheel angle.
16     return max(-MAX_STEER_CMD,
17               min(MAX_STEER_CMD,
18                   kinematic_steering / STEERING_GAIN + STEERING_TRIM))
```

Listing 4.4: The kinematic-steering / servo-trim split (`qcar_bridge.py`).

Re-verified with the same drive command, the odometry then correctly read $y \approx 0$. The general principle this illustrates is stated here explicitly, because it is expected to generalise to any future per-unit hardware calibration constant on this or a similar platform: a calibration constant compensating for a specific actuator’s physical imperfection should be applied *as late as possible, immediately at the actuator boundary*. It should never be applied upstream of any computation, such as odometry, that treats its input as an accurate, ground-truth kinematic quantity.

4.6 Clock Synchronisation and the Receipt-vs-Capture Timestamp Bug

The first real navigation-goal test I ran, once mapping, localisation and the fixes above were all in place, surfaced a new symptom: sudden AMCL localisation jumps and visible path deviation on both straight and curved segments. The description of what was actually observed was distinctive: LiDAR scan shape stays internally coherent, but the entire scan rigidly shifts and appears to track the robot’s motion, “like the skeleton of the map is moving.” That, together with an explicit instinct that this might be a transform-related issue rather than a tuning problem, pointed toward a *timing* rather than a spatial or gain-tuning cause – and this proved correct.

`/scan` (port 5556) and `/odom` (port 5555) are two independent TCP streams, each subject to its own, uncorrelated network jitter – this link had already been measured showing 87 ms to 300 ms of variable latency. `qcar_relay_node.py` had been stamping both incoming streams with `self.get_clock().now()` at the moment of TCP *receipt* on the development PC, rather than at the moment of sensor *capture* on the QCar. Because the two streams’ receipt times are decorrelated, a late-arriving scan could be paired against an odometry sample purely by receipt-time coincidence. In real physical time, that odometry sample could already be stale by a different, independent amount. This produces exactly the observed symptom: a LiDAR scan that is internally self-consistent but rigidly offset relative to the map, tracking the robot’s own recent motion.

The fix required both machines’ clocks to be related, not merely each stream’s own internal timing corrected. A round-trip-corrected NTP-style offset measurement (five clustered trials of $\text{offset} = t_{\text{remote}} - (t_1 + t_2)/2$ around an SSH `date` call) found a genuine ~ 0.50 s clock offset between the two machines – a naive sequential comparison without the round-trip correction would have overstated this by also counting SSH connection-setup time as clock skew.

Why the round-trip correction is needed. A naive measurement works like this: record a local timestamp t_1 , SSH into the QCar and read its clock to get t_{remote} , then compute $t_{\text{remote}} - t_1$. This conflates two effects that a single one-way measurement cannot separate. One is the genuine clock offset between the two machines. The other is the real, nonzero time the SSH round trip itself takes to establish a connection and execute the remote command, which inflates the apparent gap in one direction only.

The correction used here follows the same symmetric-delay assumption underlying NTP itself [12]: record a second local timestamp t_2 immediately after the remote reply arrives. Treat the remote reading as having been taken at the network’s best available estimate of the corresponding local instant – the midpoint $(t_1 + t_2)/2$. This assumes the outbound and return legs of the round trip take approximately equal time, which is reasonable for a short, otherwise idle local-network SSH round trip, but would not be for a highly asymmetric link. Averaging five such trials, rather than trusting a single one, further reduces the influence of any one trial’s round-trip time deviating unusually from that symmetric-delay assumption.

This offset was deliberately *not* corrected via a system-level NTP/chrony configuration, since the QCar is shared laboratory hardware. A persistent system configuration pointing it at one researcher’s personal development machine as a time reference is not appropriate to leave on shared infrastructure. Instead, `qcar_bridge.py` and `qcar_lidar_node.py` were

changed to stamp every message with their own local `time.time()` at the moment of capture, added as a 't' field in the JSON payload. `qcar_relay_node.py` gained a `qcar_clock_offset` parameter and a `qcar_time_to_stamp()` helper that converts a QCar-local capture timestamp into its development-PC-clock equivalent, applied consistently to both the `/odom+TF` and `/scan` publication paths. Listing 4.5 shows the conversion.

```

1 def qcar_time_to_stamp(qcar_time, qcar_clock_offset):
2     '''Converts a QCar-clock time.time() reading into a
3     builtin_interfaces/Time on this machine's clock, correcting for
4     the measured offset between the two machines' independent,
5     unsynchronized clocks.'''
6     corrected = qcar_time - qcar_clock_offset
7     sec = int(corrected)
8     nanosec = int(round((corrected - sec) * 1e9))
9     return Time(sec=sec, nanosec=nanosec)

```

Listing 4.5: Converting a QCar-local capture timestamp to the development PC's clock (`qcar_relay_node.py`).

Table 4.3: Diagnostic measurements underlying the clock-offset fix.

Quantity	Value
Network latency (TCP link, both ports)	87 ms to 300 ms (variable)
Measured clock offset (round-trip corrected, initial session)	≈ 0.50 s
Measured clock offset (persistent-connection method, final session)	≈ 0.019 s
<code>qcar_clock_offset</code> launch parameter (final verified session)	0.019

A measurement artifact found in a later session. A subsequent hardware session revisited this offset and found the round-trip correction above still carries a hidden bias when each of the five trials opens its own fresh SSH connection. Repeating the measurement that day gave a wide, noisy spread (mean 0.93 s, range 0.28 s–1.58 s) rather than a tight, repeatable figure. Clock drift accumulates gradually and could not explain a spread this wide within one sitting. The actual cause was SSH's own per-connection handshake: opening a brand-new connection for every trial front-loads that handshake delay onto the outbound leg only, which breaks the equal-outbound/return-delay assumption the round-trip correction itself depends on.

Re-measuring over a single, persistent SSH connection (**ControlMaster** multiplexing, no fresh handshake per sample) removed this asymmetry directly. The result converged tightly to 0.019 s across eleven of twelve samples, roughly two orders of magnitude smaller than the 0.50 s figure originally measured. The diagnosis and fix earlier in this section remain correct in substance: an uncorrected receipt-timestamp mismatch of this kind genuinely produces the display artefact described here. Only the specific magnitude measured at the time was inflated by the fresh-connection method's own hidden asymmetry, not by a flaw in the receipt-vs-capture diagnosis itself. `qcar_clock_offset = 0.019` is the value used in this project's final hardware configuration.

4.7 Safe Shutdown Procedure

Because two of the four processes in this bridge run directly on physical hardware with no supervising ROS 2 lifecycle to fall back on, this project established and documented a fixed shutdown sequence, using `SIGINT` exclusively rather than a plain `kill` (per the signal-handling fix in Section 4.2): (1) stop `qcar_relay_node.py` on the development PC; (2) stop the visualisation launch process group (RViz may itself segfault on exit, a known, harmless Qt/OpenGL quirk on this system – worth confirming it has actually exited rather than assuming); (3) stop `qcar_lidar_node.py` on the QCar; (4) stop `qcar_bridge.py` on the QCar (requires `sudo`). This project’s own operational experience found it is not sufficient to trust that the sequence *was run*. On one occasion, the motor bridge was still listening on its TCP port after I believed it had been stopped. The documented procedure therefore explicitly includes confirming both TCP ports (5555, 5556) are released on the QCar (`ss -tlnp`) before considering the vehicle idle.

4.8 Summary

This chapter’s bridge architecture makes the physical QCar appear, from the perspective of every ROS 2 node in the navigation stack, as an ordinary `/cmd_vel`-in, `/odom//scan`-out robot – despite the underlying reality of two incompatible ROS 2 distributions connected by a bespoke TCP/JSON protocol. Getting the data flowing across that bridge to a fidelity usable by AMCL and Nav2, however, required four further, independently discovered corrections (LiDAR angle convention, wheel odometry, steering trim and inter-machine clock synchronisation) beyond simply moving bytes across the link. Chapter 5 continues directly from here, addressing the one remaining discrepancy this chapter’s fixes do not touch: the relationship between a commanded velocity and the vehicle’s actual physical speed.

5 System Identification and Calibration

This chapter documents the longest and most consequential investigation of the project. A real-hardware symptom emerged in which the vehicle, commanded to travel a fixed distance and then stop, consistently travelled substantially further before actually coming to rest, in the worst observed case roughly three times as far. What began as a search for a physical explanation (drivetrain coasting) ended in the discovery of a stale-data artefact in the vendor hardware abstraction layer described in Section 2.6. The true physical residual turned out to be small once that artefact was removed. The chapter also documents a control-design attempt that was tested on physical hardware without adequate prior validation and produced a genuine, dangerous oscillation. It is included here in full because it is as methodologically important as the investigation’s eventual successful resolution.

5.1 Problem Statement

With the bridge of Chapter 4 operational, a simple closed-loop test was used throughout this investigation: drive the vehicle at a fixed commanded speed toward a target distance, using odometry to detect when the target is crossed and stop. Repeated trials showed the vehicle travelling substantially further than the commanded target before settling, with the overshoot growing, not shrinking, across repeated sessions under otherwise similar conditions. This was an early indication that the effect was systematic and reproducible rather than simple measurement noise.

5.2 Ruling Out Simpler Explanations

Several plausible causes were tested directly and eliminated before the investigation’s eventual resolution:

- **AMCL/localisation.** The original suspect. Investigating it produced one genuine, separately valid fix. This raised AMCL’s `recovery_alpha_fast/recovery_alpha_slow` from 0.0 to 0.1/0.001, since a value of 0.0 allows the particle filter to freeze permanently once it (incorrectly) converges to zero uncertainty under real sensor noise. This failure mode is not exposed by a clean simulation environment. This fix was kept, but it does not explain the stop-distance symptom, which persisted after it was applied.
- **Command-delivery delay.** Measured directly on the QCar’s own local clock (eliminating any network-latency contribution) at approximately 0.2s from decision to applied throttle. This is normal and not large enough to explain metre-scale overshoot at the tested speeds.
- **A backlog of stale /cmd_vel commands.** Hypothesised as queuing behind a blocking socket `send` call in the relay’s single-threaded executor. Two increasingly careful fixes were built and deployed: a staleness timestamp check, then a full non-blocking sender-thread architecture. Both were tested directly on hardware. **Neither made any measurable difference**, ruling this out as the cause.

- **Logging/print buffering artefacts.** The timing of QCar-local debug samples during the “coast” period was a steady 20 ms to 35 ms throughout, not a rapid burst consistent with a draining backlog – ruled out.

5.3 Active Braking as a Control-Design Attempt and a Safety Incident

With the above ruled out, the leading hypothesis shifted to genuine drivetrain (motor and gearbox) rotational inertia keeping the wheels turning for a period after commanded throttle reached zero. This was a physically plausible mechanism and one apparently consistent with QCar-local encoder readings that showed real velocity persisting for several seconds after a stop was commanded.

A corrective control law was implemented directly in `qcar_bridge.py`: a reverse-throttle brake pulse proportional to the residual measured velocity, intended to actively cancel the coasting motion. **This was deployed directly to physical hardware without first being traced through by hand or tested in simulation.** On its first test, it produced a genuine, dangerous oscillation: the vehicle lurched forward and backward repeatedly, reaching -0.88 m/s in reverse and still accelerating. It had to be stopped manually via an SSH-issued kill command mid-test. The root cause, in hindsight, is a textbook control failure mode. A continuous proportional reaction to *any* nonzero residual velocity, including reverse velocity, with no deadband and no explicit disarm condition, means every correction’s own overshoot looks to the controller like a fresh error requiring correction. The result is an undamped hunting loop with no mechanism to settle.

Since both attempts were reverted, no implementation of this control law survives in the repository to quote directly. Listing 5.1 instead reproduces the comment left in its place in `qcar_bridge.py`, at the site the code once occupied. This is the project’s own permanent record of what was tried, why it failed and why it must not be blindly re-attempted.

```

1 # Tried active braking (brief reverse-throttle pulse proportional to
2 # residual speed when a stop is commanded) to counter drivetrain coast -
3 # REJECTED, not just untuned. It's a proportional loop on velocity sign
4 # with no deadband and one-cycle-stale (~20ms) feedback: if the pulse
5 # overshoots past zero into real reverse motion, the next cycle reads a
6 # negative v and "corrects" with a forward push, which can itself
7 # overshoot - a genuine undamped hunting oscillation, not a bad gain
8 # value. Confirmed on hardware (twice - it was reverted once already
9 # before this comment, then re-tried and reproduced the same result):
10 # car visibly lurched forward/back repeatedly and fast, had to be
11 # killed manually. Do not re-add this without a real deadband/
12 # hysteresis or a one-shot (non-continuous) brake pulse design -
13 # retuning BRAKE_GAIN alone will not fix it.
```

Listing 5.1: Comment documenting the rejected active-braking design, left in place of the reverted code (`qcar_bridge.py`).

A minimal model of why this design cannot be stable. The instability the comment above describes can be shown formally with a deliberately simplified discrete-time model that

keeps only the mechanism responsible, not the full nonlinear vehicle dynamics. Let v_k be the residual measured velocity at control cycle k . Let the brake law command a throttle proportional to that residual with gain K : $u_k = -Kv_k$. If the vehicle’s velocity responds to one cycle’s throttle command with an effective per-cycle gain a (lumping motor response, control period and vehicle mass into a single constant and ignoring natural friction/drag for this minimal model), the resulting closed-loop update is

$$v_{k+1} = v_k + a u_k = v_k(1 - aK). \quad (5.1)$$

This is a first-order linear recurrence. Its behaviour is governed entirely by the magnitude of $(1 - aK)$. The residual velocity decays toward zero only if $|1 - aK| < 1$, i.e. $0 < aK < 2$; it oscillates in sign every cycle if $aK > 1$. Critically, that oscillation *grows* in magnitude, without bound, whenever $aK > 2$, since each successive v_k is then multiplied by a factor more negative than -1 . This single inequality is the formal statement of exactly what the source comment describes qualitatively: a correction that overshoots past zero into real reverse motion, prompting a next-cycle correction in the opposite direction that itself overshoots by more. It also shows directly why *no* amount of retuning K alone guarantees safety here. a , the vehicle’s true, real per-cycle response to a given throttle command, was never independently measured before this control law was deployed, so there was no way to have verified $aK < 2$ in advance from the gain value alone.

A one-shot design avoids re-arming after the first correction, but Equation 5.1’s single-step overshoot condition does not require re-arming to manifest. If the very first correction alone satisfies $aK > 2$, it already produces a reverse velocity larger in magnitude than the original residual it was meant to cancel. This is consistent with the one-shot retest reproducing the same oscillation, rather than being protected by its disarm condition. It is precisely the kind of hand-traceable instability check Section 3.1’s standing rule asks for *before* hardware deployment, not after.

A revised, “one-shot” design was implemented: braking fires exactly once per drive-to-stop transition and permanently disarms the instant measured velocity crosses zero, rather than re-arming from residual drift. It **reproduced the same oscillation on retest**. Given the repeated safety failure, this control approach was abandoned entirely rather than iterated on further. Both modified files were reverted to their exact pre-investigation state, verified byte-identical to the known-good version by hash comparison. The failure was documented directly in a source-code comment so that it is not blindly re-attempted in a future session.

This incident is the clearest instance in this project of the general principle stated in Section 3.1. A reactive feedback controller acting on a live sensor reading, on a system already confirmed to have real actuator lag and inertia, should be traced through by hand for an overshoot scenario or tested in simulation first, before it is ever sent to physical hardware. “The control theory seems sound” was not, on its own, sufficient justification here. This project treats that as a standing rule for any future reactive control logic on this platform, not merely a lesson learned in retrospect.

5.4 Sim-versus-Real Comparison

Following a methodology suggested by my academic supervisor, the identical closed-loop stop-distance test (drive N metres at a fixed speed, stop the instant odometry crosses the target) was reproduced in both the physical robot and the Gazebo simulation of Chapter 3, using the same launch stack with only the drive target switched from simulated to real hardware.

A six-point sweep across three target distances (1.0 m, 1.5 m and 2.0 m) and three commanded speeds (0.20 m/s, 0.25 m/s and 0.30 m/s) found real hardware overshoot in the range 0.58 m to 2.12 m across two sessions roughly one hour apart. *Every* tested condition was worse in the second session, by a broadly consistent margin. This is evidence of real, reproducible drift, plausibly battery voltage or floor/tyre condition, rather than measurement noise.

The matched simulation sweep, over the same six conditions, showed overshoot of only 0.044 m to 0.067 m – 10 to 30 $\times$ smaller and only weakly sensitive to speed. This is in contrast to the real hardware overshoot, which scaled roughly with the cube of speed. A dedicated, isolated comparison at 2 m / 0.3 m/s sharpened this further. The simulated vehicle settled at 2.067 m (an extra 0.067 m), while the real vehicle settled at 3.91 m (an extra 1.90 m). This is a $\sim 28\times$ difference for an identical software command. This result strongly supports a real-hardware-specific controller/dynamics cause, rather than something inherent to the stop-command approach itself. It directly motivated the calibration work in the remainder of this chapter.

5.5 Throttle Calibration

5.5.1 The Uncalibrated Gain Hypothesis

`qcar_bridge.py`'s mapping from `cmd_vel`'s `linear.x` (in m/s) to the HAL's PWM duty cycle had, since early in the project, used a placeholder gain (`THROTTLE_GAIN = 1/3`), explicitly documented in its own source comment as uncalibrated. Direct measurement from the sweep data above confirmed this mattered. Commanding 0.3 m/s produced a real, QCar-local encoder-measured cruise velocity averaging ≈ 0.8 m/s, roughly 2.6–2.7 $\times$ the commanded speed, consistent across two independent measurements. Since coasting distance from momentum scales with the square of velocity, this alone predicts roughly a 6.9 $\times$ larger coast distance than a genuine 0.3 m/s stop would produce. This is a substantial fraction of the sim-vs-real gap found above and a reminder that the sim-vs-real comparison in Section 5.4 was not, at that point, comparing truly matched real speeds.

5.5.2 A First Calibration Attempt That Discovered a Deadband, Then Reverted

A dedicated measurement tool, `check_stop_latency.py`, was built to measure cruise velocity and stop latency in a single run. An initial correction lowered `THROTTLE_GAIN` by dividing by the previously measured 2.67 $\times$ ratio. Deployed to hardware, it immediately surfaced a second, independent, hard non-linearity. At the old gain, a commanded 0.25 m/s ($\sim 8\%$ duty) sat completely motionless for ≈ 2.1 s before suddenly starting to move. This delay was later traced (Section 5.6) to a HAL buffering artefact rather than a friction effect. 0.20 m/s and 0.15 m/s ($\sim 6.7\%/5\%$ duty) never moved at all across 8 s to 10 s of commanding.

This second behaviour is zero motion regardless of how long the duty is held. It is a genuine, independent physical deadband property of the vehicle, not a linear gain error. It interacts badly with a naive linear-gain correction: the lowered gain restored accurate cruise-speed tracking at higher commanded speeds, but pushed the duty cycle for normal teleoperation speeds below the deadband threshold entirely, making the vehicle stop responding to real driving commands. The gain correction was reverted the same day to restore usable teleoperation.

This is a different friction phenomenon from Section 2.3.2’s – and, in part, not friction at all. It is worth being precise about which effects are actually responsible here, since this thesis initially conflated two genuinely distinct physical phenomena with a third, unrelated software artefact, all under similar friction-flavoured language. Section 2.3.2’s Coulomb tyre-ground friction (the simulation’s μ_1/μ_2) governs whether a spinning wheel’s rotation converts into forward chassis thrust at all and is essentially instantaneous. It has no notion of a wheel being “stuck” and is unrelated to everything below. The deadband characterised here is a different phenomenon internal to the real drivetrain itself (motor brushes, gearbox mesh, bearing seats): *static* friction (often called stiction) resists the onset of relative motion between two surfaces up to some breakaway force threshold, while *kinetic* friction, which applies once motion has actually begun, is often measurably lower than that static threshold for the same surface pair. A commanded duty cycle below the static threshold produces zero motion no matter how long it is held, since the applied torque never exceeds the force needed to break static friction in the first place. This zero/non-zero threshold is genuinely real and remains present after every fix in this chapter (Section 5.6.4).

What is *not* a friction effect, despite initially being described as one here, is the multi-second delay observed *before* that breakaway. Section 5.6 shows this ≈ 2 s figure is fixed regardless of duty, speed, or how much torque is applied, something a genuine breakaway-force process would not produce. It traces the delay instead to a buffered sensor read in the vendor HAL.

Once duty is raised past the real static threshold, the *lower* kinetic friction then applies to an already-moving system, so the same duty cycle that only barely broke static friction can accelerate substantially further than expected once moving. This asymmetry, unlike the delay, is a genuine, still-valid physical effect. It is the stick-slip signature Section 5.5.3 identifies directly in the duty-sweep data. Table 5.1’s 0.085 row *runs away* to a sustained higher cruise speed rather than settling, a continuing behavioural difference rather than a one-off timing offset.

This reframes the calibration problem: gain and deadband are not independent. Any solution must jointly (a) track commanded speed accurately at normal operating speeds and (b) keep every one of those same commands above the static-friction threshold. A single linear multiplier applied to the whole range cannot do both: scaling the gain down to fix (a) simultaneously worsens (b), for exactly the commands that were already only barely clearing the threshold.

5.5.3 Final Calibration via an Empirical Duty Sweep

Given that finding, the calibration approach was reframed entirely: rather than extrapolating a linear gain from one or two data points, THROTTLE_GAIN was forced to zero and a dedicated, lighter-weight tool (`drive_and_log.py`) was used to sweep fixed PWM duty values directly,

with the vehicle on the ground under real load throughout. Table 5.1 summarises the resulting duty-response curve.

Table 5.1: Fixed-duty sweep results (QCar on the ground, real load).

Duty	Observed behaviour
0.050	No motion at all (confirmed twice, up to 27.5 s of commanding)
0.060–0.061	Reliable breakaway after an initial ≈ 2.0 s to 2.1 s start-up delay (later traced to a HAL buffering artefact, Section 5.6, not friction); settles to a stable, non-runaway cruise of ≈ 0.24 m/s to 0.33 m/s
0.065	≈ 0.39 m/s
0.085	Runs away to ≈ 0.85 m/s to 0.9 m/s – a stick-slip signature (static friction exceeds kinetic friction, so once moving, the same duty produces far more net accelerating force), confirmed by watching the full velocity trace climb continuously rather than plateau

A caution surfaced during this sweep and worth recording explicitly: the motion/no-motion threshold near duty ≈ 0.06 is genuinely noisy, not a hard deterministic cut-off. A re-test with duty adjusted by only +0.0013 (0.0533 vs. 0.052) flipped a command from “27 s of zero motion” to reliably covering the full target distance. This floor appears to drift somewhat both within and across sessions (plausibly floor contact, resting wheel position, temperature, or battery voltage). No single threshold measurement from this sweep should be treated as an exact, permanent constant.

The deployed calibration combines a deadband floor with a linear gain above it, fit from the duty=0.061/0.33 m/s and duty=0.065/0.39 m/s points and validated separately:

$$\text{duty} = \text{THROTTLE_DEADBAND} + \text{THROTTLE_GAIN} \cdot |v_{\text{cmd}}|,$$

$$\text{THROTTLE_DEADBAND} = 0.04, \quad \text{THROTTLE_GAIN} = 0.0667. \quad (5.2)$$

Listing 5.2 shows Equation 5.2 as implemented in `qcar_bridge.py`’s control loop, together with the rate limiter that caps how fast `current_throttle` (the value actually written to the HAL) may change per cycle. This is a separate safeguard from the deadband/gain calibration itself, present throughout this chapter’s testing to keep duty-cycle steps within the documented 100 %/s hardware limit (Appendix A.1).

```

1 if abs(target_linear) < 1e-3:
2     desired_throttle = 0.0
3 else:
4     desired_throttle = math.copysign(
5         THROTTLE_DEADBAND + THROTTLE_GAIN * abs(target_linear), target_linear)
6 desired_throttle = max(-THROTTLE_LIMIT, min(THROTTLE_LIMIT, desired_throttle))
7 max_step = THROTTLE_RATE_LIMIT * dt
8 delta = max(-max_step, min(max_step, desired_throttle - current_throttle))
9 current_throttle += delta

```

Listing 5.2: Duty-cycle command computation and rate limiting (`qcar_bridge.py`).

Table 5.2 presents the resulting end-to-end validation across four commanded speeds, each from a dedicated 2 m drive-and-stop test with cruise velocity measured directly from `/odom`.

Table 5.2: Throttle calibration validation (Equation 5.2).

Commanded speed	Measured cruise speed	Ratio	Note
0.2 m/s	unreliable (≈ 0.24 or no motion)	–	At the deadband edge
0.3 m/s	0.31 m/s	1.05	
0.4 m/s	0.42 m/s	1.05	
0.5 m/s	0.50 m/s	1.00	

This is a substantial improvement over the original uncalibrated gain’s 2.6–2.7 $\times$ overshoot: commanded speeds of 0.3 m/s to 0.5 m/s now track within $\approx 5\%$. Below ≈ 0.25 m/s to 0.3 m/s, open-loop duty control against real floor/battery/temperature-dependent friction is inherently unreliable. This is treated as a hard physical limit of the open-loop approach at the tested operating conditions, not a remaining tuning gap. Reliable sub-0.25 m/s creep would require closed-loop velocity feedback from the encoder, rather than a fixed duty mapping.

5.5.4 A Stray-Process False Positive

While iterating on this calibration, a separate, purely diagnostic false alarm is worth recording as a general debugging caution. A prior `check_stop_latency.py` test process, left running in the background rather than exiting automatically once its measurement completed, continued publishing `linear.x=0.0` on `/cmd_vel` at 20 Hz, silently racing against a separately launched teleoperation node publishing on the same topic. The resulting symptom was the vehicle briefly registering a throttle command and then immediately reverting to zero within one control cycle. It looked exactly like a gain, deadband, or logic bug and was initially investigated as one. ROS 2 does not warn about multiple publishers on the same topic; it simply interleaves messages, with whichever publisher’s message arrives last in a given control cycle taking effect. The actual fix was operational, not code: kill the stray process. This project subsequently adopted two standing practices as a result: checking `pgrep -af` for topic-related script names (or `ros2 topic info /cmd_vel -verbose` for publisher count) before deep-diving into a control-logic theory whenever real behaviour contradicts an issued command; and designing one-shot diagnostic nodes to exit automatically once their measurement is complete, rather than spinning indefinitely.

5.6 The Hardware Abstraction-Layer Buffering Defect

The original stop-distance symptom this chapter opened with is a multi-second coast after a stop command, with substantial extra distance. Even after the throttle calibration of Section 5.5.3 made commanded and real cruise speed match closely, that symptom was still present, apparently unaffected by the gain fix. This section documents how that residual was ultimately traced to a defect in the vendor hardware abstraction layer itself, not the vehicle’s physical dynamics at all. This is the project’s single most significant finding, both for its magnitude and for the methodological lesson in how long an incorrect physical explanation can survive against evidence that, in hindsight, directly contradicted it.

5.6.1 An Initial, Incomplete Diagnosis

A closer look at the raw coast-phase velocity trace from one of the calibration validation runs (0.5 m/s) showed the coast was *not* a smooth decay. Velocity remained essentially flat, if anything slightly *elevated* relative to the preceding cruise average, for the first ≈ 2.0 s after “stop” was commanded. It then crashed sharply to zero in the final 0.3 s to 0.4 s. This flat-plateau-then-cliff shape is inconsistent with passive drivetrain momentum/friction coasting (which predicts continuous, gradual decay starting immediately once power is cut) and instead resembles the real motor continuing to be *actively driven* for roughly two seconds after the stop command was issued. A review of the Quanser HAL documentation [17] found no mention of any drive-motor ramp, filter, or time constant that would explain a two-second lag of this kind – only a separate, documented 0.16 s steering-servo time constant, unrelated to the drive motor. This shifted the leading hypothesis toward a command-delivery lag in the project’s own bridge software, rather than physical coasting. This was a genuine change of theory, captured here because it turned out to still be one step short of the actual cause. A diagnostic instrumentation pass (logging commanded velocity, the bridge’s own internally-ramped throttle value and real encoder velocity together, all on the QCar’s local clock to eliminate network timing) was begun but interrupted by the test vehicle’s battery running out mid-session and resumed in a later session on a charged battery.

5.6.2 Disproving Two Hypotheses and the Argument That Found the Real Cause

The resumed diagnostic trace showed the bridge’s own internal throttle value reaching exactly zero within 52 ms of the stop command, while the encoder-reported velocity stayed elevated for roughly two more seconds. This correctly ruled out the project’s own throttle-ramping logic as the cause. As understood in hindsight, though, it did *not* actually rule out a measurement or reporting defect in the encoder-velocity reading itself, since the two values are produced through entirely separate code paths. This distinction was missed on the first pass through the data.

At this point the person operating the vehicle directly and repeatedly disputed the emerging conclusion. Teleoperating the QCar from a genuine standstill, the vehicle visibly starts and stops moving *instantly* in response to a command. This flatly contradicts the ≈ 2 -second delay the automated test script was measuring. Two further hypotheses were tested and disproven with direct hardware data before the actual cause was found:

- **Duty-magnitude dependence** (a higher commanded speed produces more torque and a faster breakaway): tested at 0.7 m/s (duty 0.087, well above the normally tested 0.052 r to 0.075 range) – the start-up delay was still 2.092 s, statistically identical to the low-speed tests. Disproven.
- **“v is a stale reading, not real motion.”** Disproven by the structure of the `Odometry` class’s own update code. The reported velocity `v` and the accumulated position `x` are computed from the exact same `distance` variable within the same loop iteration. If `v` were falsely indicating motion, the accumulated position would have to freeze at the same instant. It demonstrably did not: distance kept growing smoothly and matched the vehicle’s real, physically measured settled position.

The argument that ultimately identified the true cause was not a further measurement, but a piece of physical reasoning I raised directly: *an encoder is a real-time sensor – it cannot have a built-in two-second lag by itself*. This was correct. It redirected the investigation to a question the prior analysis had not actually asked: not whether the `Odometry` class was internally self-consistent (it was), but whether the value it was consuming as its own input, the raw `car.motorEncoder[0]` read, was itself stale before the `Odometry` class ever saw it. This prompted reading the vendor HAL’s own Python source directly, rather than continuing to reason about it indirectly from documentation or from the `Odometry` class’s own behaviour.

5.6.3 The Root Cause, a Ring-Buffer I/O Mode

The root cause, found in the vendor HAL source (`pal/products/qcar.py`) on the QCar itself, is exactly the `readMode` distinction introduced in Section 2.6. `qcar_bridge.py` constructed its `QCar` object with `readMode=1` (task-based I/O). This starts a background acquisition task *at object construction time*, before the bridge’s listening TCP socket has even opened. It fills a ring buffer of `frequency×2` samples (at this project’s 50 Hz read rate, exactly 100 samples, i.e. 2.0 s) with overwrite-on-overflow behaviour. Each call to `car.read()` dequeues one buffered sample. Because `car.read()` is not actually called until a relay client connects, which can take many seconds after the bridge process starts, the buffer fills and cycles during that idle gap. Once reads begin at a consumption rate matching the production rate exactly, the read stream is permanently stuck approximately 100 samples (2.0 s) behind real time, because the backlog never has an opportunity to drain.

Formalising the lag: why it is exactly the buffer depth, forever. Figure 5.1 illustrates the mechanism derived below. Let the background task write into the ring buffer at a fixed production index p , incrementing by one every $1/f$ seconds ($f = 50$ Hz here). Let `car.read()` calls advance a separate consumption index c , each call returning the sample at position $c \bmod B$ (buffer capacity $B = 100$) and then incrementing c . Production begins at object construction, $t = 0$; consumption begins only once a relay client connects, at $t = T_{\text{idle}}$. During $[0, T_{\text{idle}}]$, p advances alone (production with no consumption at all), so by the time consumption starts, $p = f \cdot T_{\text{idle}}$ while $c = 0$. This is a gap of $f \cdot T_{\text{idle}}$ samples between them. If $T_{\text{idle}} > B/f$ (true here, since a relay connecting takes several seconds while $B/f = 2.0$ s), the overwrite-on-overflow buffer has already wrapped around at least once by T_{idle} . This means the oldest sample still available to read is not the very first one produced, but the one produced exactly B samples before p ’s current value. The very first `car.read()` call, at $t = T_{\text{idle}}$, therefore necessarily returns a sample captured at $t = T_{\text{idle}} - B/f$, a lag of exactly $B/f = 2.0$ s, regardless of how much larger than B/f the idle gap T_{idle} itself was. Production and consumption both advance at the identical rate f from that point onward (one `car.read()` per control-loop cycle, matching the background task’s own sampling rate). Because of that, the gap between p and c (and therefore the lag between “now” and whatever sample is being read) stays fixed at exactly B samples indefinitely. Neither index ever gains on the other, so the backlog that accumulated during the idle gap is carried forward forever rather than draining over time. This is precisely why the lag proved so exactly reproducible (always $B/f = 2.0$ s, never more and never less, regardless of how

long the QCar sat idle before a relay connected) and why it never improved on its own the longer the system ran.

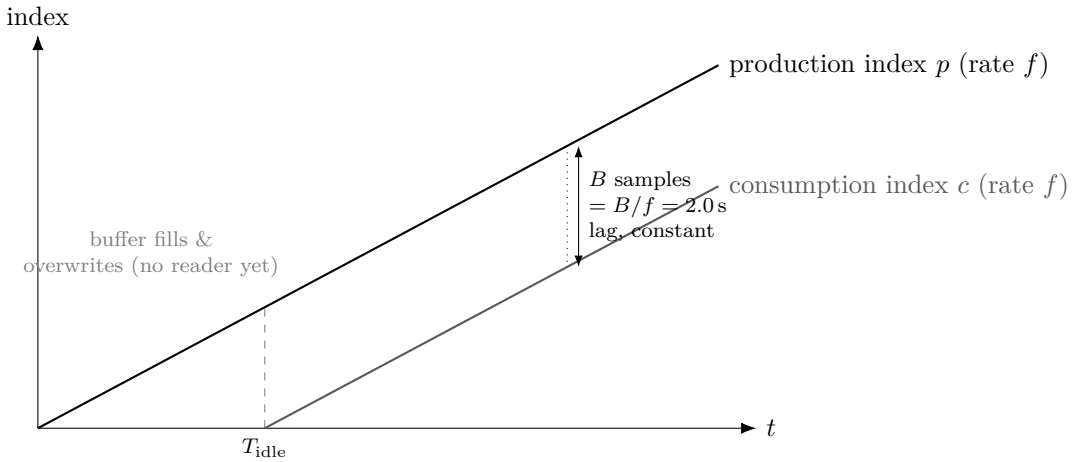


Figure 5.1: Production index p (background acquisition task, running from $t = 0$) versus consumption index c (`car.read()` calls, starting only once a relay connects at $t = T_{idle}$). Both advance at the same rate f once consumption begins, so the gap present at $t = T_{idle}$, exactly the buffer depth B once the buffer has wrapped, never closes.

This single mechanism explains every property of the symptom observed across this entire chapter: the near-exact 2.0s to 2.1s figure (matching the buffer’s mathematically exact 2.0s capacity), its remarkable stability across sessions, vehicle speeds and physical conditions (a fixed buffer depth is indifferent to any of these) and its symmetric appearance on both the start side (the start-up delay of Section 5.5.2) and the stop side (the “coast”), since both are read through the same buffered path.

Fix: `qcar_bridge.py` now constructs the `QCar` object as shown in Listing 5.3 – immediate I/O, reading current hardware state directly with no background task or buffer involved at all.

```

1 # readMode=0 (immediate I/O) - was 1 (task-based I/O), which runs a
2 # background acquisition task into a 100-sample ring buffer
3 # (frequency*2) from the moment this object is constructed,
4 # independent of when car.read() actually starts being called. Since
5 # this object is created before the listening socket even opens, and
6 # car.read() isn't called until a relay connects, the buffer fills and
7 # starts overwriting during that gap - once reads begin at the matched
8 # 50Hz rate, they get permanently stuck ~100 samples (~2.0s) behind
9 # real-time, since the backlog never drains. Immediate I/O reads
10 # current hardware state directly, no task/buffer involved.
11 car = QCar(readMode=0, frequency=READ_RATE)

```

Listing 5.3: The one-line HAL construction change that fixed the buffering defect (`qcar_bridge.py`).

5.6.4 Validation

The full calibration test suite from Section 5.5.3 was re-run in its entirety on a freshly charged battery, with the duty formula (Equation 5.2) unchanged, to isolate the effect of this single fix. Table 5.3 presents the results.

Table 5.3: Effect of the `readMode=0` fix on start-up delay, extra distance and stop latency.

Test	Start-up delay (before → after)	Extra distance (before → after)	Stop latency (before → after)
2 m @ 0.3 m/s	2.09 s to 0.117 s	0.520 m to 0.114 m	2.37 s to 0.598 s
2 m @ 0.4 m/s	2.14 s to 0.001 s	0.790 m to 0.114 m	2.40 s to 0.501 s
2 m @ 0.5 m/s	2.14 s to 0.116 s	0.923 m to 0.154 m	2.55 s to 0.550 s
1 m @ 0.3 m/s	2.14 s to 0.120 s	0.641 m to 0.118 m	2.44 s to 0.572 s
3 m @ 0.3 m/s	2.08 s to 0.088 s	0.583 m to 0.104 m	2.35 s to 0.509 s

Start-up delay collapses to 0.001 s to 0.12 s – an essentially instantaneous breakaway response, matching direct observation of the vehicle. Extra distance becomes small and roughly constant, ≈ 0.10 m to 0.15 m, regardless of speed or target distance, rather than scaling with cruise speed as previously found. This confirms that the earlier “extra distance scales with speed” relationship was itself an artefact of a fixed-*duration* buffering delay applied to different real velocities, not a genuine vehicle property.

Both the extra-distance and stop-latency figures in Table 5.3 are encoder-derived, from wheel rotation counts via `/odom`, as Section 5.6.2 establishes. At each of the tested distances, the encoder-reported travel was independently cross-checked against tape-measured floor markings and manual stopwatch timing of the commanded-stop-to-physical-stop interval. The two agreed. This confirms the encoder as an accurate proxy for real physical distance and timing, rather than a further source of measurement error.

The small residual that remains after this fix is real and is fully accounted for by ordinary physics, rather than being a further open mystery. Dividing extra distance by stop latency, an implied average coast velocity, yields 54 % to 61 % of cruise velocity in four of the five tests. This is consistent with an ordinary, roughly linear velocity decay to zero over the stop-latency window (a perfectly linear decay predicts exactly 50%). The one outlier (2 m @ 0.3 m/s, 108%) is attributable to a data-analysis artefact (the “cruise average” window for that specific test likely had not reached a true steady state, now that the run has much less runway once start-up delay is near-instant) rather than a physical inconsistency.

0.2 m/s remains genuinely unreliable even after this fix, sometimes failing to move the vehicle at all. This is the real, independent static-friction deadband characterised in Section 5.5.2 and is unrelated to the buffering defect resolved here.

5.7 Summary

This chapter’s investigation began with a symptom: travelling roughly three times the commanded stop distance. This symptom had at least three superficially plausible explanations: localisation error, command-delivery delay and physical drivetrain coasting. It ended by identifying two genuinely independent, compounding causes. One was an uncalibrated, non-linear throttle-to-speed mapping (Section 5.5). The other was a fixed-duration sensor-read buffering artefact in the vendor hardware abstraction layer (Section 5.6) that had, for a period of this project, been reasonably but incorrectly attributed to genuine vehicle physics.

Neither cause was found by reasoning from documentation or first principles alone. Both

required direct, targeted measurement on the physical robot. The second was found only after I raised a direct physical argument that prompted reading the vendor’s own source code, rather than continuing to trust an internally self-consistent, but incompletely verified, software abstraction. Chapter 6 builds on this chapter’s calibrated, buffering-free bridge to bring up and validate full closed-loop Nav2 navigation on the physical vehicle.

6 Navigation Integration and Validation

This chapter describes bringing the full Nav2 navigation stack up against the physical QCar, building on the bridge of Chapter 4 and the calibration of Chapter 5. Section 6.1 covers porting the simulation-tuned Nav2 configuration to hardware. Section 6.2 describes a custom MPPI critic plugin developed for this vehicle. Section 6.3 reports the phased bring-up used to de-risk the integration. Section 6.5 presents a detailed timing investigation into an apparent localisation-display artefact. Section 6.6 reports the resulting end-to-end validation.

6.1 Porting the Nav2 Configuration to Hardware

The Nav2 parameter set developed and tuned in simulation (Chapter 3) required a small, mechanical set of changes to run against physical hardware rather than Gazebo: every occurrence of `use_sim_time` (13 in total, across the costmap, planner, controller and AMCL parameter blocks) was switched from `true` to `false`, since the physical robot has no simulation clock to synchronise against. Both mapping and navigation launch files were given a real-hardware branch, gated on `launch_sim`, alongside the existing Gazebo branch. Real hardware is now the default, with simulation as an opt-in (`launch_sim:=true`) – a deliberate inversion made once hardware became the project’s active development target. The simulation path was kept fully intact for future regression testing rather than removed. Figure 6.1 traces the resulting launch sequence for this now-default hardware branch. It runs from the manual, pre-launch bring-up of the onboard bridge and relay (Chapter 4), through the top-level `ros2 launch` invocation, to the localisation and navigation lifecycle managers reaching `active`. Three processes must already be running before this launch file is even invoked: `qcar_bridge.py` and `qcar_lidar_node.py` on the QCar itself and `qcar_relay_node.py` on the development PC. None of these are started by `qcar_nav2.launch.py`. The simulation branch (`launch_sim:=true`) instead starts Gazebo, `robot_state_publisher` and `spawn_entity.py` directly from within the same launch file and needs no such manual step – at the cost of every sensor/odometry topic coming from a simulated plugin rather than the real hardware bridge.

The AMCL `recovery_alpha_fast/recovery_alpha_slow` fix from Section 5.2 was carried forward proactively into the hardware configuration at this point, rather than left to be independently rediscovered against hardware. The underlying cause is a property of AMCL’s own resampling algorithm, not specific to any one deployment target: Section 2.5.2’s augmented-MCL random-particle-injection mechanism is permanently disabled at $\alpha_{\text{slow}} = \alpha_{\text{fast}} = 0$. This leaves the filter with no way to recover from an overconfident convergence once real sensor noise (which a clean simulation rarely exercises) triggers one.

6.2 The Custom MPPI Critic `EarlyCommitCritic`

Early navigation testing showed a specific weakness in the stock MPPI critic set (Section 2.5) for this vehicle. The standard path-following critics evaluate a sampled trajectory’s alignment and progress against the path as a whole, which can leave the controller driving further into

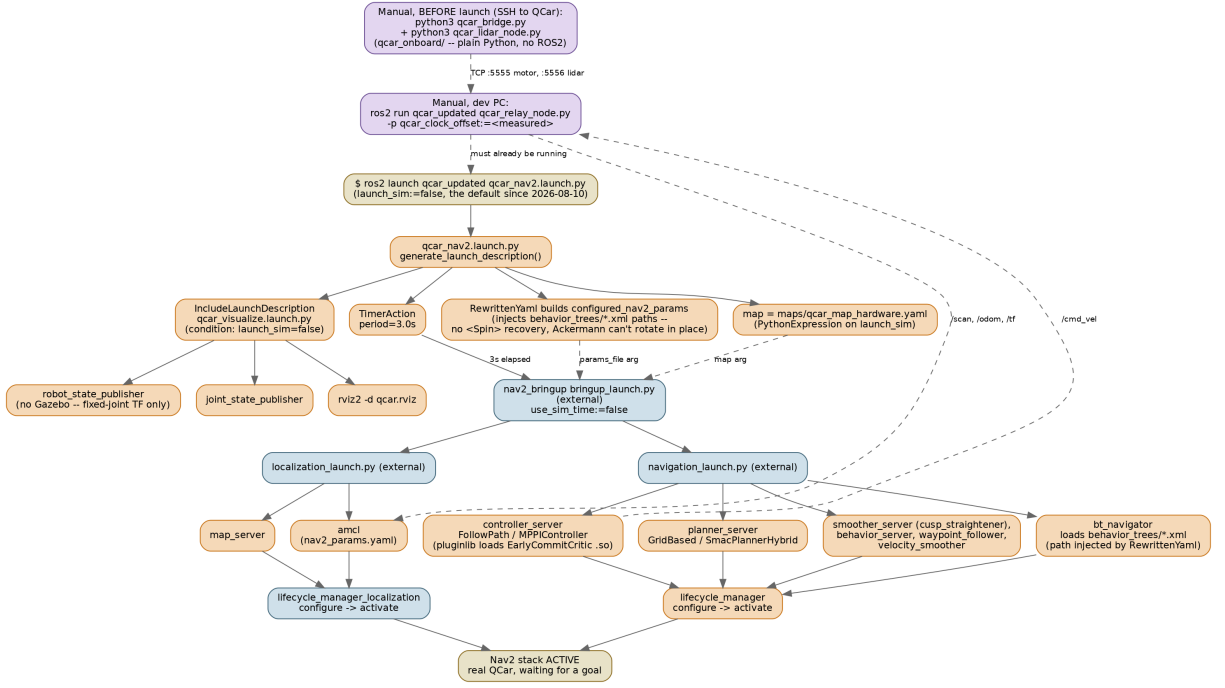


Figure 6.1: The launch dependency graph of `qcar_nav2.launch.py`'s hardware branch (`launch_sim:=false`, the default since 2026-08-10): the manual, pre-launch bring-up of the onboard bridge and relay (top), `qcar_visualize.launch.py`'s TF/RViz bring-up with no Gazebo involved (left), the timed `nav2_bringup` include (centre) and the `RewrittenYaml`-configured parameter file and hardware map selection (right) converge on `nav2_bringup`'s own `localization_launch.py`/`navigation_launch.py`, each bringing its lifecycle manager to `active` before the stack is ready to accept a goal.

an approaching curve before it begins reacting to it, rather than beginning the turn as early as physically sensible. This behaviour is more forgiving for a differential-drive robot's tight turning radius than for an Ackermann-steered vehicle with the finite minimum turning radius of Table 2.2.

A custom MPPI critic plugin, `EarlyCommitCritic`, was developed and built into the navigation package to address this. It scores only the first `early_time_steps` of each sampled trajectory rollout against the bearing to a near-term path point, with no distance or angle gating, so that MPPI is pushed to begin turning toward the path immediately rather than waiting until deeper into the curve. It is gated by an `active_path_points` parameter so that it stops influencing the controller once the vehicle has made genuine progress along the route, avoiding interference with the stock critics' normal path-tracking behaviour during straight segments. A later direction-awareness refinement extended the critic to handle reversing and K-turn path segments correctly, where a naive "always turn toward the next point" bearing calculation would otherwise fight the vehicle's intended direction of travel.

Building this plugin required matching `xtensor`/`xsimd` compile definitions exactly against Nav2's own MPPI controller build. A mismatch here does not produce a compile-time error. Instead it causes a silent ABI incompatibility that crashes the navigation container on the first control cycle that touches vectorised critic math, making it a notably unforgiving class of build-configuration bug to diagnose from its symptom alone.

Listing 6.1 shows the critic's `score()` method in full. It computes the bearing from each sampled trajectory's own starting position to a near-term path target (`offset_from_furthest` points

ahead of the furthest point the robot has already reached). For only the first `early_time_steps` of that trajectory, it then accumulates the angular distance between the trajectory’s actual yaw and that bearing. This scores every sampled rollout independently, not just the robot’s single current pose. The early return on `active_path_points` is what confines this to the start of a path. The `forward_preference_-`gated minimum of the forward- and reverse-bearing angular distances is the direction-awareness refinement mentioned above, letting a trajectory be scored against whichever of the bearing or its reverse it is actually closer to, rather than penalising a legitimate reversing manoeuvre.

```

1 void EarlyCommitCritic::score(CriticData & data)
2 {
3     if (!enabled_ || data.path.x.shape(0) < 2) { return; }
4     utils::setPathFurthestPointIfNotSet(data);
5
6     // Only relevant at the start of a path - once the robot has made
7     // real progress, let the normal path/goal critics take over.
8     if (*data.furthest_reached_path_point >= active_path_points_) { return; }
9
10    const size_t path_size = data.path.x.shape(0) - 1;
11    const size_t offseted_idx = std::min(
12        *data.furthest_reached_path_point + offset_from_furthest_, path_size);
13    const float target_x = data.path.x(offseted_idx);
14    const float target_y = data.path.y(offseted_idx);
15
16    const size_t time_steps = data.trajectories.x.shape(1);
17    const size_t early_steps = std::min(early_time_steps_, time_steps);
18    if (early_steps == 0) { return; }
19
20    const xt::xtensor<float, 1> x0 = xt::view(data.trajectories.x, xt::all(), 0);
21    const xt::xtensor<float, 1> y0 = xt::view(data.trajectories.y, xt::all(), 0);
22    const xt::xtensor<float, 1> bearing = xt::atan2(target_y - y0, target_x - x0);
23    const xt::xtensor<float, 1> reverse_bearing = bearing + static_cast<float>(M_PI);
24
25    xt::xtensor<float, 1> sum_diff = xt::zeros<float>({data.trajectories.x.shape(0)});
26    for (size_t t = 0; t < early_steps; ++t) {
27        const xt::xtensor<float, 1> yaw_t = xt::view(data.trajectories.yaws, xt::all(), t);
28        const xt::xtensor<float, 1> fwd_diff =
29            xt::abs(utils::shortest_angular_distance(yaw_t, bearing));
30        if (forward_preference_) {
31            sum_diff += fwd_diff;
32        } else {
33            const xt::xtensor<float, 1> rev_diff =
34                xt::abs(utils::shortest_angular_distance(yaw_t, reverse_bearing));
35            sum_diff += xt::minimum(fwd_diff, rev_diff);
36        }
37    }
38    const xt::xtensor<float, 1> cost = sum_diff / static_cast<float>(early_steps);
39    data.costs += xt::pow(cost * weight_, power_);

```

Listing 6.1: `EarlyCommitCritic::score()` (`src/critics/early_commit_critic.cpp`).

How this cost actually changes which trajectory wins. Connecting this directly to Section 2.5.1’s softmax weighting (Equation 2.9): `data.costs` is the same per-trajectory accumulator every enabled critic adds into. This critic’s contribution $(\overline{\Delta\theta} \cdot \text{weight_})^{\text{power_}}$ (the early-steps-averaged angular difference $\overline{\Delta\theta}$ from the listing’s `cost` variable, scaled and exponentiated by its own configured weight and power) is simply summed alongside the stock critics’ own contributions into $S(\tau^{(k)})$ before Equation 2.9’s exponential weighting is applied. Concretely: of two sampled trajectories approaching the same curve, one that begins turning toward the near-term path point immediately accrues a small $\overline{\Delta\theta}$ over its first `early_time_steps`, while one that continues driving straight before reacting accrues a much larger one. This inflates that second trajectory’s total cost $S(\tau^{(k)})$, which Equation 2.9 translates into an *exponentially* smaller importance weight $w^{(k)}$ relative to the early-turning trajectory. Because the executed control is the weight-averaged sum over all K sampled trajectories, what actually shifts the controller’s averaged output toward beginning the turn earlier is the systematic suppression of every late-turning sample’s weight. This is not the same as marking one specific trajectory as disqualified; no single “best” trajectory ever needs to be identified or selected explicitly.

6.3 Phased Hardware Integration

Given the number of independent subsystems involved (TCP bridge, odometry, SLAM, localisation, planning, control), integration proceeded in three deliberately staged phases rather than attempting the full stack at once. This way, a failure at any stage could be attributed to a bounded set of newly-added components rather than the entire system:

Phase 1 – Teleoperation. Bridge (Chapter 4) plus manual `/cmd_vel` teleoperation only, with no odometry, mapping, or navigation involved. Confirmed the motor and steering command path worked correctly end-to-end before any more complex subsystem was layered on top.

Phase 2 – SLAM. Cartographer mapping (Section 2.4) added on top of Phase 1, confirmed to require no odometry extension at all given the `use_odometry: false` configuration described in Section 2.4. A real, growing occupancy grid was built by driving the vehicle around a bounded space under teleoperation and saved via `nav2_map_server`’s `map_saver_cli` – this saved map is the one used for all subsequent localisation and navigation testing.

Phase 3 – Navigation. The full Nav2 stack (AMCL, planning, MPPI control) added on top of Phase 2’s saved map, requiring the wheel-odometry extension of Section 4.4 (deferred from Phase 2, where it had not been necessary) and the steering-trim (Section 4.5) and throttle calibration (Section 5.5) work, since dead-reckoning odometry and AMCL localisation are only as accurate as the vehicle model producing them.

The full stack’s initial bring-up – motor bridge, LiDAR node, relay node and the ported Nav2 launch configuration – came up cleanly on the first attempt against the saved Phase 2 map.



Figure 6.2: The Phase 2 occupancy grid, built from real LiDAR data while driving the physical QCar under teleoperation and saved via `map_saver_cli` – the same map used for all subsequent hardware localisation and navigation testing. RViz’s own navigation panel (left) reports a goal reached with zero recoveries.

Both Nav2 lifecycle managers reported their managed nodes active, all seven configured MPPI critics (the stock six plus `EarlyCommitCritic`) loaded matching the intended configuration and the costmap auto-sized correctly to the saved map’s dimensions, with no crash loop. A real navigation goal was deliberately not sent during this first bring-up. Dead-reckoning odometry, along with everything built on top of it including AMCL and planning, is only as trustworthy as the still-unresolved steering-trim calibration at that point in the project’s timeline. This calibration (Section 4.5) was completed before the first real goal was attempted. Figure 6.3 shows the same launch configuration and lifecycle sequence (Section 6.1) exercised end-to-end on its still-maintained simulation branch. It captures, in one session, the terminal log’s lifecycle transitions (top right), the RViz plan and live navigation feedback panel (left) and the Gazebo-side view of the vehicle executing the goal (bottom right). This is the same bring-up recipe the hardware branch above shares, shown here with every stage of the stack visible at once.

6.3.1 First Real-Hardware Goals in Left- and Right-Turn Trials

With Phase 3’s bring-up clean and the steering-trim calibration in place, the first real navigation goals sent to the physical vehicle were deliberately simple: a single corridor turn each, rather than a full reversal manoeuvre. Two trials were run, one requiring a left turn and one a right turn, each sending the vehicle forward out of the corridor junction and into a stop against the side wall of the corresponding side corridor (Section 6.4 covers the reversal manoeuvres that followed). Figures 6.4–6.7 show the RViz path and the physical outcome for each.

Both trials succeeded, with the vehicle reaching its goal and stopping against the intended wall in each case. This was the first confirmation that the calibrated bridge (Chapter 4) and Phase 3 stack could execute a real, AMCL-localised turn on hardware at all, before the more demanding reversal manoeuvres of the next section were attempted.

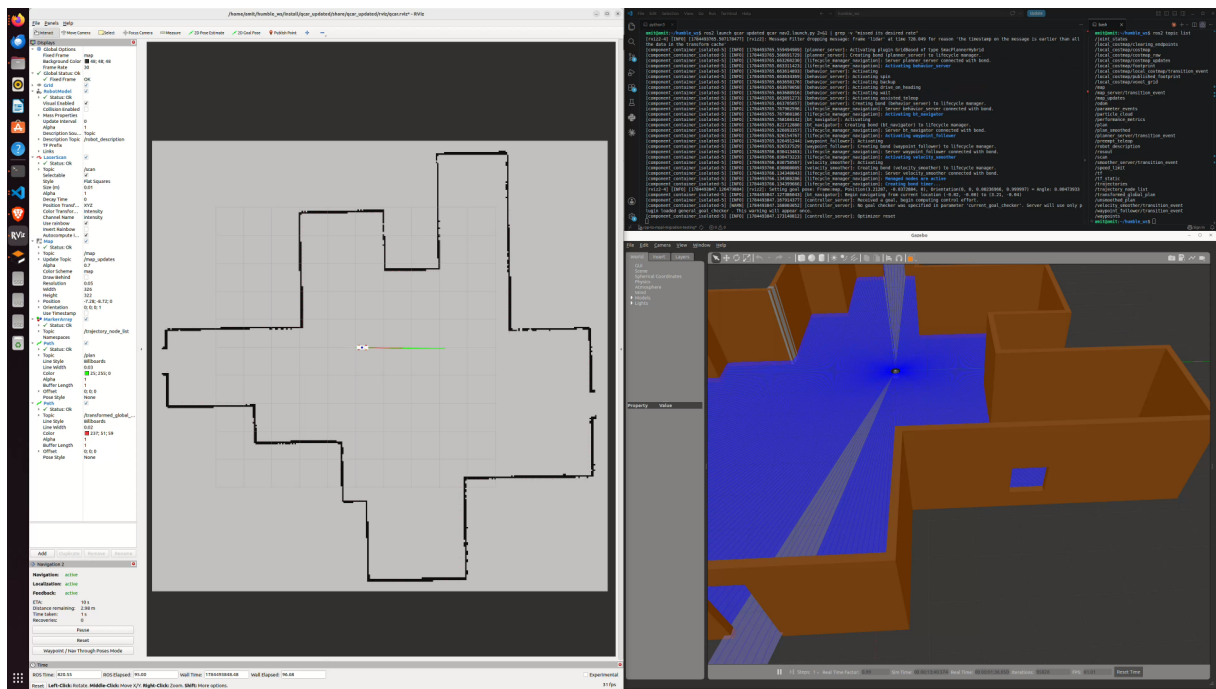


Figure 6.3: The Nav2 stack running end-to-end on the simulation branch of the same launch configuration: bring-up log (top right, showing each server and lifecycle manager activating in turn), RViz with the global plan and live navigation feedback panel (left) and the simulated vehicle executing the goal (bottom right).

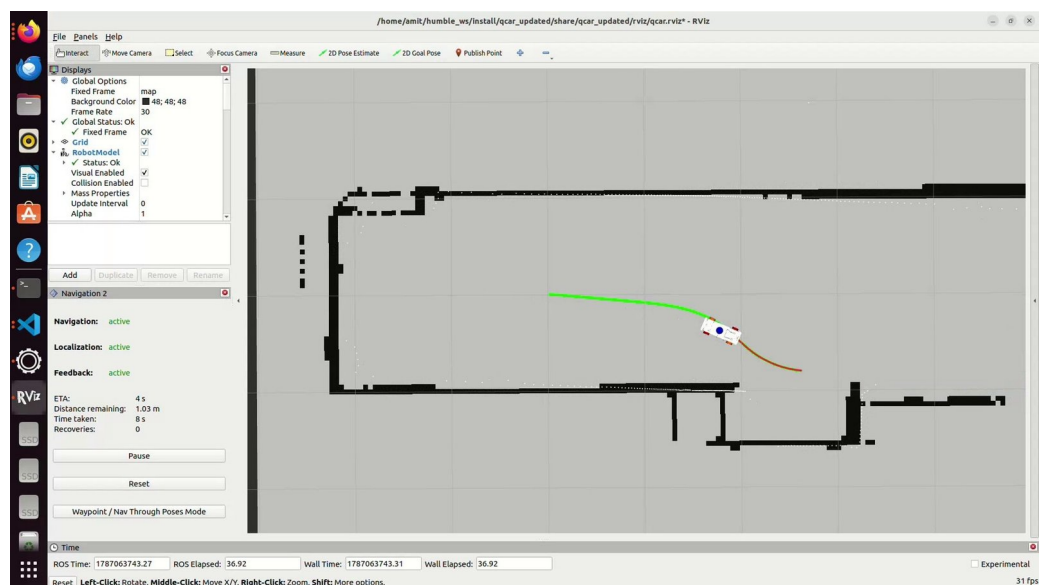


Figure 6.4: RViz during the right-turn trial: the global plan (green) curves the vehicle out of the corridor junction towards the goal, with the navigation feedback panel (left) reporting distance remaining and elapsed time as the turn is executed.



Figure 6.5: The physical QCar at the end of the right-turn trial, stopped in front of the wall at the far side of the junction.

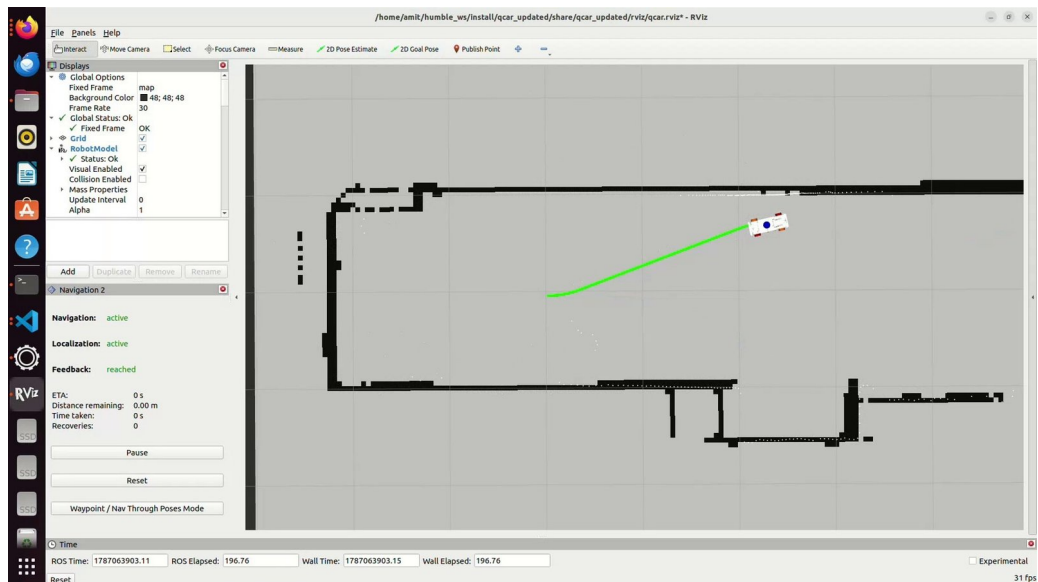


Figure 6.6: RViz during the left-turn trial: the navigation feedback panel reports *Feedback: reached* with zero distance remaining at the end of the curved approach.

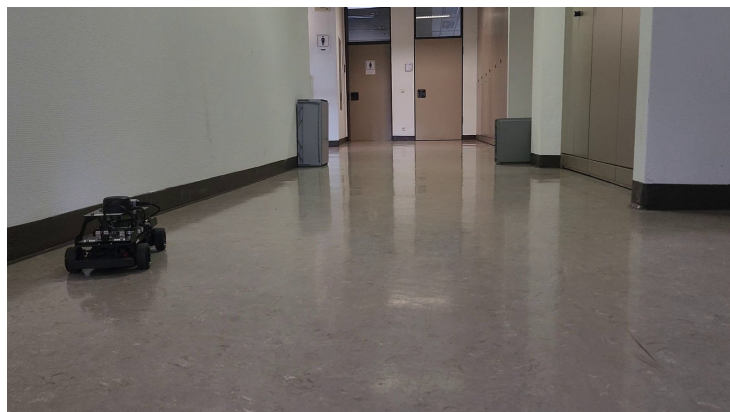


Figure 6.7: The physical QCar at the end of the left-turn trial, stopped against the side wall of the corridor junction.

6.4 Diagnosing a K-Turn Freeze at Reversal Cusps with a Custom Smoother

Beyond the display-latency and calibration issues of the preceding sections, real-hardware testing in tight spaces surfaced a distinct and more serious navigation problem. A goal requiring the vehicle to turn around produced, in my own words at the time, “a chaotic curve, then reverse with opposite curve, then forward curve.” The result was a visibly hesitant, multi-second freeze at the direction change, rather than a clean, continuous K-turn. This section documents the diagnosis, several fix attempts that failed instructively and a custom smoother plugin that resolved it. The smoother was validated in both simulation and on the physical vehicle.

6.4.1 Reproduction

To investigate safely and repeatably, a narrow dead-end corridor (approximately 2.4 m wide, 2 m deep) was added to the simulation map by re-mapping the Gazebo world of Chapter 3 and saving a fresh occupancy grid via `map_saver_cli`. A `NavigateToPose` goal was then sent deep inside it with a reversed final heading, a target that forces a K-turn in a space too tight to complete it in one continuous arc. Two logging tools were built for this investigation. One recorded every `/plan` message (not only the first, so that every replan issued during a recovery cycle gets its own sequence-numbered trace), `/odom` and `/cmd_vel` to timestamped CSVs. A companion analysis script then computed total manoeuvre duration, position-stagnation windows (freeze detection) and true steering-direction-reversal counts directly from those CSVs. Figure 6.8 shows the reproduction goal’s planned path in RViz: the global plan (green) folding back on itself at the cusp vertex, with the short executed segment (orange/red) that manages to complete before the freeze sets in.

6.4.2 A Diagnosis Where the Planner Is Correct and the Controller Is Not

Reconstructing the vehicle’s true chassis heading through the manoeuvre (motion direction minus 180° after the cusp, verified against an independent bicycle-model simulation of a three-point turn) confirmed that `SmacPlannerHybrid` was producing a kinematically correct global plan. Heading rotates smoothly and monotonically through the single Reeds–Shepp cusp the plan contains, exactly the “flip steering side to continue rotating the same way” geometry a real K-turn requires. The freeze is not a planning defect at all – it is entirely in *execution*.

The root cause lies in `nav2_mppi_controller`’s path-handling logic (`path_handler.cpp`, Humble branch). MPPI is only ever shown the global plan truncated up to the cusp pose, until a function called `isWithinInversionTolerances()` reports that the vehicle has reached that pose within *both* a position tolerance and a heading tolerance simultaneously. Until then, none of the path beyond the cusp exists as far as the controller’s own path-following critics are concerned, so nothing in the cost function encourages progress through it.

This is structurally the same failure mode as a final-goal reorientation freeze found and fixed earlier in this project’s tuning history, outside the scope of this thesis’s own narrative. A goal requiring a large final heading change would stall in the same way, for the same reason. It was resolved simply by raising `GoalAngleCritic`’s cost weight (from 3.0 to 10.0), so the controller is pushed to complete the final heading precisely. No equivalent encouragement exists for an

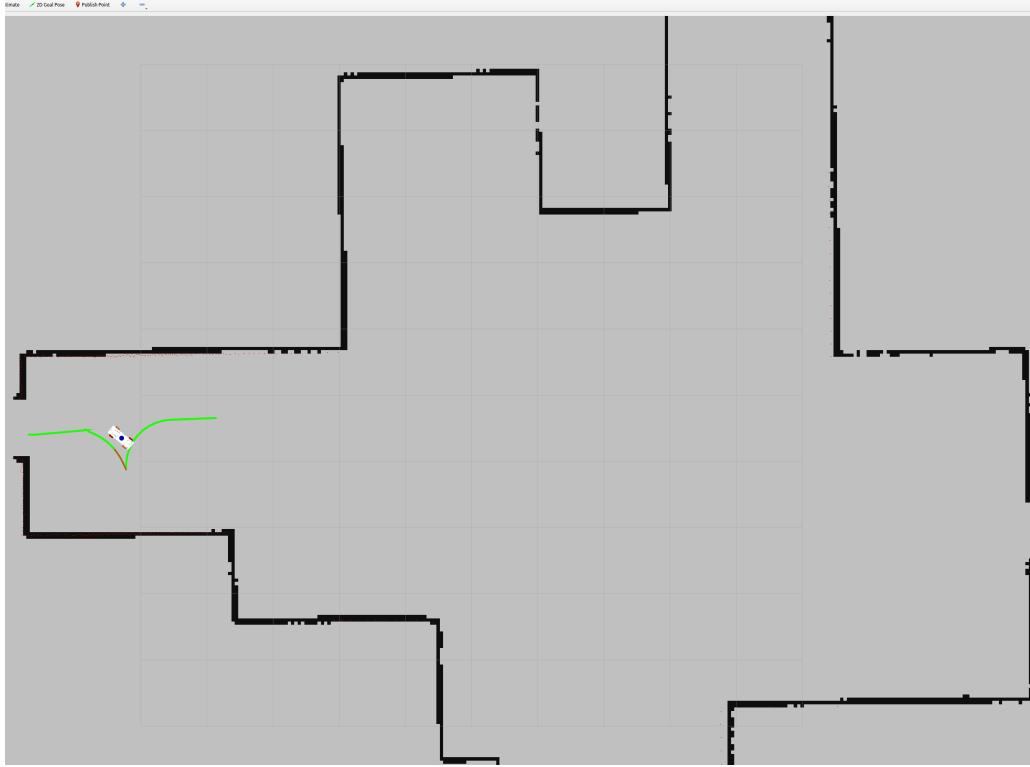


Figure 6.8: The reproduction goal’s cusp in RViz: the global `/plan` (green) reverses direction at a single vertex. The short executed segment (orange/red) shows how little of the reversal leg is actually completed before the freeze.

intermediate cusp pose, which is exactly where this freeze occurs.

6.4.3 Three Plausible Fixes, All Reverted

Three direct attempts to relax this gating condition were each tested against the reproduction and reverted, all pointing to the same underlying lesson:

- **Widening `isWithinInversionTolerances()`’s own position and heading tolerances** (from 0.2 m/0.4 rad to 0.4 m/0.7 rad) made things worse: the controller now reported the cusp “reached” too loosely. The resulting premature reveal of the post-cusp path triggered six or more `progress_checker` aborts before the vehicle had even physically reached the cusp.
- **Faster stuck-detection** (lowering `progress_checker`’s movement-time allowance from 30 s to 8 s) fired aborts every 8 s to 13 s, faster than the manoeuvre could genuinely complete. It once triggered a `BackUp` recovery action that itself failed outright for lack of room in the corridor.
- **A first-version path-smoother** that re-projected the path onto a straight line for a fixed distance after each cusp, then jumped instantly back to the original curve, performed worse still (never completing within a 108 s window). The plugin’s own logic worked exactly as designed, but the hard discontinuity it introduced at the blend boundary was itself a new obstacle for the controller.

All three attempts shared a common thread that became the guiding principle for the eventual fix: *the freeze needs real, uninterrupted time and space to resolve on its own; interrupting it, however that interruption is triggered, consistently made the manoeuvre worse, never better.*

6.4.4 A Reverse-Only, Gradually Blended Smoother

A second design, `CuspStraightenerSmoother`, was built as a `nav2_core::Smoother` plugin – path post-processing rather than a controller-side change. The plugin has two deliberate constraints: it acts *only* on cusps the path enters in reverse, since a front-steered vehicle’s steered axle is self-correcting driving forward but trailing and jackknife-prone in reverse, making a reversing cusp the harder sub-case. Forward-entering cusps and all ordinary forward segments are left completely untouched. The blend itself is also gradual rather than instant, tapering linearly from the straightened projection at the cusp itself back to the original planned curve over a configurable `straight_distance`.

A mutation-order bug and the fix. Directly testing this design against harder, multi-cusp replans (plausible whenever a path is replanned from a robot pose already mid-manoeuve, e.g. during a recovery cycle) surfaced a genuine implementation bug. The original `smooth()` detected every cusp once up front, then processed each one *in place* on the very path array it was simultaneously rewriting. When two cusps fell close together, an earlier cusp’s blend could overwrite position and heading data that a nearby later cusp still needed to read in order to classify itself and compute its own straight-line target – a self-inflicted data race within a single, single-threaded function.

The fix, shown in Listing 6.2, snapshots the incoming path into a read-only `original` vector before any modification begins. Every geometric decision the function makes, including cusp detection, forward/reverse classification and every blend target, reads only from this untouched snapshot. Writes go exclusively to a separate output path being built alongside it.

```

1  const std::vector<geometry_msgs::msg::PoseStamped> original = path.poses;
2
3  std::vector<size_t> cusp_indices;
4  for (size_t i = 1; i + 1 < n; ++i) {
5      const double d1x = original[i].pose.position.x - original[i - 1].pose.position.x;
6      const double d1y = original[i].pose.position.y - original[i - 1].pose.position.y;
7      const double d2x = original[i + 1].pose.position.x - original[i].pose.position.x;
8      const double d2y = original[i + 1].pose.position.y - original[i].pose.position.y;
9      const double len1 = std::hypot(d1x, d1y);
10     const double len2 = std::hypot(d2x, d2y);
11     if (len1 < 1e-6 || len2 < 1e-6) { continue; }
12     const double dot = (d1x * d2x + d1y * d2y) / (len1 * len2);
13     if (dot < 0.0) { cusp_indices.push_back(i); } // direction reverses by > 90 deg
14 }
15 // Classify each cusp (forward- vs reverse-entering) from ‘original’ only,
16 // independent of anything the output-building loop below has written so far.
17 for (const size_t cusp_i : cusp_indices) {
18     const double cusp_yaw = tf2::getYaw(original[cusp_i].pose.orientation);

```

```

19  const double dx = original[culsp_i + 1].pose.position.x - original[culsp_i].pose.
    position.x;
20  const double dy = original[culsp_i + 1].pose.position.y - original[culsp_i].pose.
    position.y;
21  const double along = dx * std::cos(culsp_yaw) + dy * std::sin(culsp_yaw);
22  culsp.push_back({culsp_i, along < 0.0, along}); // along < 0 => reverse-entering
23  }

```

Listing 6.2: Cusp detection and reverse/forward classification, reading only from a read-only snapshot (culsp_straightener_smoother.cpp).

Directly re-tested afterward against a genuine multi-cusp replan (a 47-point path with three culps, two of them at adjacent indices), the fixed version classified and handled both correctly – exactly the scenario the original bug would have corrupted.

Table 6.1 compares three trials each of the untouched baseline, the buggy v2 smoother and the fixed v2 smoother, on the identical corridor reproduction goal with a fresh stack relaunch before every trial.

Table 6.1: K-turn manoeuvre duration and recovery count, corridor reproduction goal (simulation).

Configuration	Trial durations (recoveries)	Mean	Worst case
Baseline (no smoother)	81.3s(1), 81.1s(1), 168.1s(4)	110.2s / 2.0	168.1s
v2 smoother (buggy)	96.6s(2), 101.4s(2), 432.8s(12)	210.3s / 5.3	432.8s
v2 smoother (fixed)	59.3s(0), 97.1s(2), 103.7s(2)	86.7s / 1.3	103.7s

Run-to-run variance is genuinely high in every configuration, consistent with the freeze being a stochastic MPPI sampling effect rather than a deterministic planning defect. But the fixed smoother outperforms the untouched baseline on every measure, while the buggy version is markedly worse than doing nothing at all – underlining how much the mutation-order bug alone was costing.

6.4.5 Extending the Corner, a Further and Larger Improvement

A further refinement addressed the residual freeze directly at its source. The path’s incoming and outgoing legs previously had to meet at one precise geometric vertex, which `isWithinInversionTolerances()` must still converge on exactly, even with the gradual blend of Section 6.4.4 in place. A new `extend_distance` parameter instead pushes the cusp point itself further along the incoming leg’s own heading before the reverse blend begins, so the incoming and outgoing legs deliberately overlap rather than meeting at a single tight point. This gives the controller geometric margin around the transition, instead of an exact target (illustrated in Figure 6.9). Listing 6.3 shows the construction: the extension point is inserted into the output path. The existing distance-based blend is re-anchored to start from it rather than from the original cusp position.

```

1  const double culsp_yaw = tf2::getYaw(original[i].pose.orientation);
2  const double ex = original[i].pose.position.x + std::cos(culsp_yaw) * extend_distance_;
3  const double ey = original[i].pose.position.y + std::sin(culsp_yaw) * extend_distance_;
4

```

```

5 geometry_msgs::msg::PoseStamped ext_pose = original[i];
6 ext_pose.pose.position.x = ex;
7 ext_pose.pose.position.y = ey;
8 output.push_back(ext_pose);
9 // The existing straight_distance_blend (unchanged logic) now runs from
10 // (ex, ey) as its origin, not from the original cusp position - the reverse
11 // leg must travel back past this pushed-out point to rejoin the downstream
12 // curve, giving the controller a margin instead of one exact vertex.

```

Listing 6.3: Pushing the cusp outward before blending, so incoming/outgoing legs overlap (cusp_straightener_smoother.cpp).

With `extend_distance` set to 0.15m alongside the existing 0.21m `straight_distance`, three further trials produced 45.2s, 38.3s and 48.1s – a mean of 43.9s with **zero recoveries in every trial**. This roughly halves the already-improved fixed-v2 mean of 86.7s, on top of its improvement over the 110.2s baseline. This is the final, locked-in configuration.

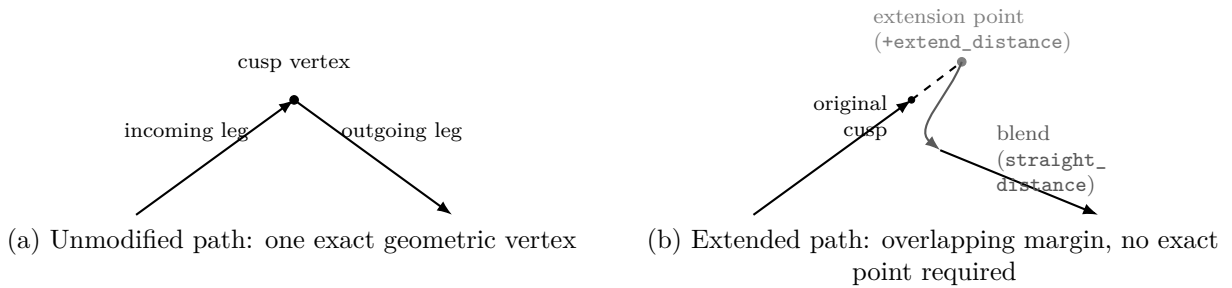


Figure 6.9: The corner-extension mechanism of Listing 6.3. (a) The planner’s raw output meets the incoming and outgoing legs at one precise vertex, which MPPI’s inversion-tolerance gate must satisfy exactly. (b) The smoother pushes the cusp outward along the incoming leg’s own heading before blending back to the outgoing leg, so the two legs deliberately overlap – trading one brittle exact target for a geometric margin the controller can negotiate.

6.4.6 Real-Hardware Validation

The fixed, corner-extending smoother was tested on the physical QCar with two goals I sent manually, each requiring a genuine reversal manoeuvre. A 5.15m traverse completed in 50.1s with zero recoveries across seven replans (cusp counts 1, 1, 1, 1, 8, 8, 0). A second, longer traverse completed in 96.8s, also with zero recoveries, across eight replans (cusp counts 5, 5, 5, 5, 9, 10, 5, 3). Both goals succeeded outright – the strongest available confirmation that the fix generalises beyond the simulated corridor reproduction it was developed against. Figures 6.10 and 6.11 are taken from this real-hardware validation session.

Both goals also produced noticeably higher cusp counts on individual replans (up to 8–10) than any simulation trial had shown (maximum approximately 3). This is an open, unexplained gap between simulated and real replanning behaviour, plausibly `SmacPlannerHybrid` responding to more awkward real goal/environment geometry than the simulated corridor exercised, but not confirmed. It did not prevent either goal from succeeding, so it is recorded here as a genuine open item for future work (Section 8.2) rather than a present limitation of the fix.

One apparent timing anomaly in this trace was investigated and correctly ruled out, rather than mistaken for a bug. The first several `smooth()` calls on the first goal landed within roughly

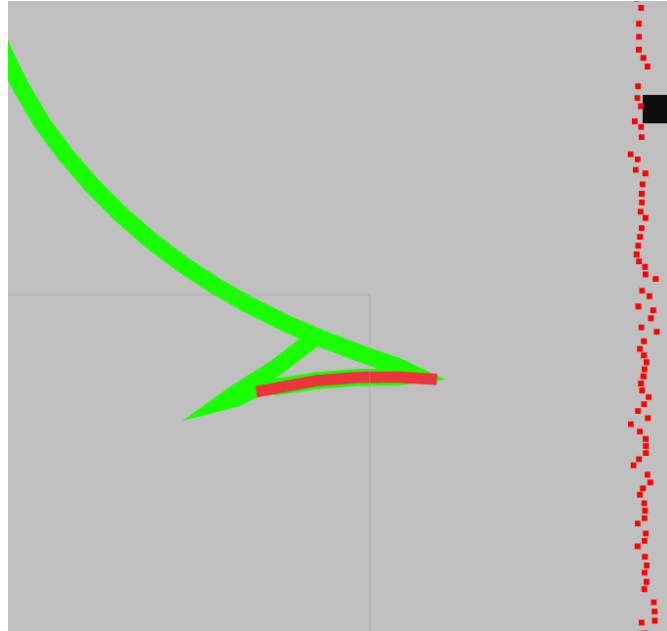


Figure 6.10: A reversal cusp during real-hardware validation: the global plan (green) folding back on itself, with the locally-smoothed path (red) already following the corner-extended geometry of Section 6.4.5 rather than the original sharp vertex.



Figure 6.11: RViz during one of the real-hardware reversal-manoeuvre trials: the navigation feedback panel (left) reports the goal as still active, with ETA and distance-remaining updating live as the vehicle follows the curved approach into the reversal.

one second of each other, which initially looked like an unthrottled replan burst. Re-reading the behaviour tree configuration showed this is by design: `IsPathValid` is already rate-limited to 1 Hz by an existing `RateController`. A `RateController` fires immediately on its first tick before settling into its steady interval. This is exactly the pattern observed, once the also-unconditional, once-only `InitialComputePathToPose` call is accounted for separately. Catching this before implementing a second, redundant rate limiter avoided a change that would have had no effect at all.

6.4.7 A Re-Extension Bug and a Curvature-Continuing Refinement

Hardware testing continued beyond the validation above and surfaced a further, genuine bug in the corner-extension mechanism itself, plus a refinement to it. Both were developed and sim-tested after the hardware-validated result of Section 6.4.6 and are not yet re-confirmed on the physical vehicle at the time of writing. They are reported here with that status made explicit, consistent with this thesis’s practice of not presenting a provisional result as final.

Bug: the extension re-applies to the same cusp on every replan. Nav2 replans roughly once per second while a goal is active. Each replan’s path starts fresh from the robot’s current pose. Because of this, the *same* physical K-turn cusp reappears near the front of the replanned path on every single replan, as the robot approaches and then executes it. Since the smoother has no notion of “already handled this one,” it re-applied the full extend-and-straighten treatment to that same cusp every time. Direct log analysis of a real hardware run confirmed this concretely: one K-turn’s cusp received the treatment on eight separate replans as the robot closed in on it, its index in the (shrinking) replanned path falling from 94 down to 1. By the final replan, the robot was already essentially at the cusp, mid-manoeuve. The smoother was still reshaping the path directly underneath it, rather than leaving an already-committed manoeuvre alone. This is, functionally, a milder recurrence of the same “interrupting an in-progress manoeuvre makes it worse” pattern identified repeatedly elsewhere in this investigation (Section 6.4.3), this time self-inflicted by the smoother rather than by an external replan trigger.

The fix adds a `min_lead_distance` parameter (0.3 m): the arc length from the replanned path’s start to each candidate cusp is computed. The extend-and-straighten treatment is only applied if the cusp is still at least this far ahead of the robot’s current position, leaving an already-reached or already-underway cusp untouched rather than reshaping it again. Two sim trials on the standard K-turn reproduction goal after this fix showed no regression, though the specific skip condition was not directly observed firing in either trial (both completed before any replan’s cusp closed to within the threshold). It is correct by code review and by the logged parameter value at start-up, but not yet directly confirmed exercising its skip branch. This is recorded as an open verification item, not a claimed result.

Refinement: continuing the incoming curve rather than cutting straight. Separately, the extension itself was upgraded from a straight-line push-out (Listing 6.3, hardware-validated above) to one that continues the incoming leg’s own curvature. The motivation is direct: cutting straight right at the corner, even when the vehicle is still visibly curving on its approach to

the cusp, asks the vehicle to straighten abruptly at exactly the point it should be easing into the manoeuvre most naturally. The refined version estimates the incoming leg’s curvature at the cusp from its own recent yaw change, $\kappa = \Delta\theta/\Delta s$ over the last incoming segment. It generates the extension as a short constant-curvature arc with that same curvature, rather than a straight line. This arc is built from several closely spaced sub-poses (0.03 m steps), so the curve is genuinely present in the path’s own geometry, not merely implied by a single endpoint. It degrades automatically to the original straight-line behaviour when the incoming leg is already straight ($\kappa \approx 0$), so it is a strict generalisation of the hardware-validated mechanism rather than a replacement with different behaviour on straight approaches.

```

1 // kappa estimated from the incoming leg's own yaw change (dyaw/ds) at the cusp - 0 if
  the
2 // incoming leg was already straight, giving the original straight-line behaviour
  unchanged.
3 constexpr double kExtensionStep = 0.03;
4 const int extension_steps = std::max(1, static_cast<int>(std::round(extend_distance_ /
  kExtensionStep)));
5 const double step = extend_distance_ / extension_steps;
6 double ex = cx, ey = cy, ext_yaw = cusp_yaw;
7 for (int s = 1; s <= extension_steps; ++s) {
8     const double arc = step * s;
9     double px, py;
10    if (std::abs(kappa) > 1e-6) {
11        const double radius = 1.0 / kappa;
12        px = cx + radius * (std::sin(cusp_yaw + kappa * arc) - std::sin(cusp_yaw));
13        py = cy - radius * (std::cos(cusp_yaw + kappa * arc) - std::cos(cusp_yaw));
14    } else {
15        px = cx + std::cos(cusp_yaw) * arc; // kappa ~ 0: degrades to the straight-line
          case
16        py = cy + std::sin(cusp_yaw) * arc;
17    }
18    ext_yaw = cusp_yaw + kappa * arc;
19    // ... each (px, py, ext_yaw) is appended as its own pose, making the arc explicit.
20 }

```

Listing 6.4: Continuing the incoming leg’s own curvature into the extension, rather than cutting straight (cusp_straightener_smoother.cpp).

A single sim smoke test on the standard reproduction goal completed successfully (≈ 99 s, within the range of prior trials). The new curvature-extension code path was exercised across eight reverse-cusp events over the run’s several replans, with no numerical failure. This is reported as a working, sim-verified refinement, not yet subjected to the same multi-trial statistical comparison or hardware re-validation as the straight-line version it replaces – a concrete, already-identified next step (Section 8.2).

6.4.8 A Related Planner-Side Finding on Near-Wall Goals and a Tolerance Trade-off

A separate, planner-level finding surfaced during the same period of continued tuning, also directly relevant to operating in tight, wall-adjacent spaces. A goal placed only ≈ 0.20 m from the nearest mapped wall – inside the footprint clearance the vehicle’s own $0.44\text{ m} \times 0.18\text{ m}$ collision footprint requires – failed outright on every one of six recovery retries. Each attempt logged `SmacPlannerHybrid` exhausting its entire search iteration budget without ever finding a collision-free plan. This is a goal-placement issue rather than a code defect: no footprint- or inflation-related configuration was implicated. The practical guidance that follows is simply to keep any commanded goal at least 0.3 m to 0.4 m clear of a mapped obstacle, given this vehicle’s footprint.

A tempting-looking fix was tried and deliberately reverted. The reason why is itself a useful finding. `SmacPlannerHybrid`’s own search loop, confirmed directly against its source, backtracks to the closest node it reached within a configurable `tolerance` radius, rather than failing outright whenever the exact goal proves infeasible. Widening `tolerance` (from 0.5 m to 1.0 m) did immediately rescue the failing near-wall goal in a live test.

It introduced a worse, silent failure mode in its place. The substituted pose the planner backtracks to – not the originally requested goal – becomes the plan’s actual endpoint. Nothing downstream in the standard `NavigateToPose` behaviour tree ever rechecks the final pose against the original request. *Any* marginally infeasible goal, not only the intended near-wall case, could now report `Goal succeeded` while the vehicle sat up to a full 1.0 m away from where it was actually asked to go and potentially facing the wrong direction. Nothing in the result would indicate that a substitution had occurred at all.

An honest, visible failure is preferable to a silently mis-reported success, particularly for a system whose results this thesis reports as measured fact. `tolerance` was reverted to its original 0.5 m , accepting that the specific near-wall goal will fail outright again. The correct fix for a genuine target at that location is to ensure real clearance there, not to relax what counts as “success” for every goal system-wide. This mirrors, at the planner level, the same safety-over-convenience judgement Section 5.3 applies at the control level: a fix that resolves the reported symptom by quietly weakening a correctness guarantee elsewhere is not an acceptable trade, even under time pressure.

6.4.9 Two Follow-On Attempts, Reverted

Two further improvements were implemented and hardware-tested the same day. Both were reverted. They are included here because a reverted attempt with a documented, evidence-based reason is exactly the kind of result this thesis’s methodology (Section 3.1) treats as worth reporting, not discarding.

`IsPathValidDeboundedCondition` is a custom behaviour-tree condition requiring two consecutive failed validity checks, rather than stock `IsPathValid`’s single check, before triggering a replan. It improved simulation trials further still (38.2 s , 38.3 s and 40.4 s , mean 39.0 s , zero recoveries) by absorbing occasional single-tick sensor or transform noise that would otherwise trigger an unnecessary replan. On real hardware, a subsequent run showed the vehicle never executing its reverse segment at all. The local costmap reported the path invalid on *every* check,

not intermittently, so the debounced condition correctly (and unhelpfully) confirmed a persistent rather than transient invalidity every 1.5s to 2.6s, aborting the in-progress manoeuvre each time. This is the same “interrupting a manoeuvre makes it worse” lesson from Section 6.4.3, under a new trigger. The most likely root cause, not fully confirmed before the revert, is an unmeasured `qcar_clock_offset` (Section 4.6) on the relay for that session – a fresh measurement immediately afterward found a real ≈ 0.15 s offset. A companion `max_cusps` rejection cap was reverted alongside it to return to one single known-good state rather than debug two independent changes under time pressure, though it was not itself implicated (every replan in the failing run came back at exactly two cusps).

A custom `PathDeviationCritic` pulls sampled trajectories toward a point further along the path once the vehicle drifts more than 0.3m from it, rather than toward the nearest path point. It was tested for three trials (112.8s, 71.7s and 38.4s, mean 74.3s, zero recoveries) against the corner-extending smoother’s own 43.9s mean and reverted. Both the mean and the run-to-run variance were worse, plausibly because the new critic’s pull competed directly against the stock `PathAlignCritic` rather than complementing it.

6.4.10 A Related Operational Caution on Goal Preemption

A separate mechanics finding from this investigation is worth recording as an operational rule rather than a fix. Consider a goal that is deep into a failing/retrying cycle: the planner repeatedly failing to find a plan, say, with the behaviour tree’s recovery retries nearly exhausted. If a new goal is sent while that old one is still active, the new goal does not reliably receive a fresh attempt. It was observed to fail almost instantly, with no real planning attempt of its own, rather than superseding the old goal cleanly. The practical rule is to let a stuck goal finish (fail out) or explicitly cancel it before sending a correction, rather than assuming a new goal always starts with a clean slate.

6.5 Diagnosing an Apparent Display-Localisation Artefact

After the throttle calibration of Section 5.5.3 (but before the buffering-defect fix of Section 5.6), a live Nav2 goal test surfaced a new symptom, described in detail below. When driving toward a goal inside an enclosed space, the entire mapped rectangle appeared to shift in RViz, with the LiDAR scan keeping its internal shape but dragging behind the robot’s true motion before “snapping” back into alignment. The same shift-then-correct pattern recurred, after a delay, once the goal was reported reached.

6.5.1 Ruling Out Simpler Causes

Several candidate causes were checked directly and eliminated. `qcar_clock_offset` was re-measured fresh and re-applied. One procedural caution surfaced along the way: a relay launch command given during this session had omitted the `qcar_clock_offset` argument entirely, silently defaulting it to zero and briefly reintroducing exactly the bug fixed in Section 4.6. It is worth checking for this explicitly before suspecting the offset value itself. `map` $\rightarrow$ `odom` TF was captured directly during active driving and updated in small, smooth, incremental steps,

rather than sudden jumps. The RViz **Fixed Frame** configuration was confirmed correct (`map`). `base` $\rightarrow$ `lidar` TF is static, so it is not a timing bottleneck. Local CPU load was checked and found unremarkable (RViz $\approx 10\%$, the Nav2 container $\approx 29\%$). Network round-trip latency to the QCar was 1 ms to 6 ms, far too small to explain a multi-second visible effect. Direct receipt-versus-capture latency on `/odom` and `/scan` measured ≈ 0.18 s and 0.20 s respectively, small and tightly clustered, ruling out network or relay transit as the source of a large visible lag.

6.5.2 Instrumentation via `monitor_goal_timing.py`

A dedicated instrumentation tool was built to timestamp a single Nav2 goal end to end: goal-sent, real motion start/stop (from `/odom`), RViz-displayed motion start/stop (from the `map` $\rightarrow$ `base` TF) and goal-reached. Two implementation defects were found and fixed while building this tool, both worth recording as general lessons for any future action-status monitoring code on ROS 2:

1. **A QoS durability mismatch.** The actual publisher QoS for `/navigate_to_pose/_action/status` is `RELIABLE + TRANSIENT_LOCAL`. `rclpy`'s default subscription QoS, constructed by passing a plain integer depth, leaves durability at `VOLATILE`. This is technically QoS-compatible by specification (a subscriber may request less than a publisher offers), but it delivered *zero* messages in practice, confirmed twice on real hardware. The fix required constructing an explicit `QoSProfile` matching the publisher exactly, a reminder that formal QoS compatibility rules do not guarantee the practical behaviour a developer expects (Section 2.1). This is exactly the `TRANSIENT_LOCAL`-publisher / `VOLATILE`-subscriber case Table 2.1 marks as formally compatible: durability governs whether *already-published* samples are replayed to a newly matched subscriber. A low-frequency topic like action status may not publish a fresh sample again for a long time, so a `VOLATILE` subscriber that missed the replay is left waiting with no error raised on either side.
2. **Goal-detection logic.** The tool originally waited for status code `ACCEPTED` specifically to mark “goal sent”, but a goal can transition directly from absent to `EXECUTING` between two consecutive status publishes, skipping an independently observable `ACCEPTED` update entirely. This topic's `TRANSIENT_LOCAL` history appears to retain only the latest status per goal, not the full transition sequence. The fix tracks goal UUIDs instead: the first appearance of a previously unseen `goal_id` (after seeding a known-ID set from whatever the topic already retained at monitor startup) marks “goal sent”, regardless of which status value it first carries.

Listing 6.5 shows the explicit `QoSProfile` that fixed the first defect – matching `bt_navigator`'s publisher exactly rather than relying on the default subscription QoS constructed from a plain integer depth.

```

1 STATUS_QOS = QoSProfile(
2     reliability=QoSReliabilityPolicy.RELIABLE,
3     durability=QoSDurabilityPolicy.TRANSIENT_LOCAL,
4     history=QoSHistoryPolicy.KEEP_LAST,
5     depth=10,
6 )

```

```

7 ...
8 self.create_subscription(GoalStatusArray, '/navigate_to_pose/_action/status',
9                           self.status_cb, STATUS_QOS)

```

Listing 6.5: Explicit QoS profile matching `bt_navigator`’s action-status publisher (`scripts/monitor_goal_timing.py`).

Listing 6.6 shows the second fix. Rather than waiting for a specific status code, the callback seeds a `known_goal_ids` set from whatever the topic already retains at startup, so pre-existing, already-completed goals are not mistaken for a new one. It then marks “goal sent” the first time a previously unseen goal UUID appears in any status, independent of which status value that UUID first carries.

```

1 if not self.known_goal_ids:
2     for s in msg.status_list:
3         self.known_goal_ids.add(bytes(s.goal_info.goal_id.uuid))
4     self.get_logger().info(
5         'seeded %d pre-existing goal id(s), waiting for a new one'
6         % len(self.known_goal_ids))
7     return
8
9 for s in msg.status_list:
10     gid = bytes(s.goal_info.goal_id.uuid)
11     if gid not in self.known_goal_ids:
12         self.known_goal_ids.add(gid)
13         if self.goal_sent_t is None:
14             self.goal_sent_t = now
15             self.tracked_goal_id = gid

```

Listing 6.6: Goal-UUID-based “goal sent” detection (`scripts/monitor_goal_timing.py`).

An explicit discovery-wait (polling `get_publishers_info_by_topic` plus a settle margin) was also required before treating the monitor as ready to observe a goal. A subscription created and immediately assumed matched can miss a fast goal’s status entirely, if DDS discovery with the separate `bt_navigator` process has not yet completed.

6.5.3 Measured Result

Table 6.2 presents one complete goal traced with this tool.

Table 6.2: Timing trace of one Nav2 goal (`monitor_goal_timing.py`).

Event	Offset	Pose
Goal sent	–	–
Real start (odom)	+2.145 s after goal sent	(3.344, 1.674) odom frame
RViz start (map→base)	+2.705 s after goal sent	(0.033, –0.031) map frame
Goal reached	+6.660 s after goal sent	–
RViz stop	+2.094 s after goal reached	(1.257, –0.024)
Real stop (odom)	+2.238 s after goal reached	–

This gives a start gap (RViz minus real) of +0.559 s and a stop gap of −0.144 s (RViz reporting “stopped” slightly *before* the real vehicle fully settled – small and plausibly within the detection threshold’s own noise rather than a separate effect). Note that the real-start offset of 2.145 s already includes the HAL read-buffer start-up delay characterised in Section 5.6, so it is not, by itself, a new timing number.

6.5.4 Interpretation and a Necessary Caveat

This measurement supports two separate, unrelated conclusions. The ≈ 0.56 s start-of-goal gap is a modest, constant pipeline latency between a real event and its appearance on screen. Network transit, AMCL’s scan-matching cycle and RViz’s own subscribe/render overhead all stack up to produce it. It is not a TF-jump defect (ruled out in Section 6.5.1). Nor is it a real navigation-performance problem, since Nav2’s own controller consumes `/odom//amcl_pose//tf` directly at the source, without RViz’s additional rendering overhead. The robot’s actual control decisions are working with data less stale than what a human operator sees on screen. The end-of-goal gap, by contrast, is the same stop-latency/coast phenomenon already characterised in Chapter 5. The real vehicle keeps moving for 2.238 s after the goal is reported reached. This matches the pre-fix stop-latency figures in Table 5.2’s predecessor almost exactly, confirmed here via a completely independent measurement method (a live Nav2 goal rather than the open-loop `drive_and_log.py` test).

The necessary caveat and the reason this section is presented in detail rather than as a closed result: **this entire measurement was taken while the `readMode=1` buffering defect of Section 5.6 was still active** in `qcar_bridge.py` – it was fixed the following day. Given that defect’s confirmed effect on every encoder-derived quantity (Table 5.3), the 2.238 s end-of-goal figure here is most likely the same artefact, not a distinct navigation-specific finding. Its magnitude matches the buffering defect’s characterised effect too closely to be coincidental. The 0.559 s start-gap figure is a different measurement (`map`→`base` TF timing rather than `/odom` velocity thresholding) and is not as directly tied to the same artefact. But since it is also downstream of the same buffered `/odom` stream, it cannot be treated as a clean, artefact-free measurement of pure network/AMCL/RViz pipeline latency either.

The diagnostic methodology and tooling built in this section remain valid and reusable: the QoS/goal-detection fixes and the systematic elimination of TF-jump, Fixed-Frame and network-latency causes all still hold. Only the specific numbers in Table 6.2 should be treated as provisional against the `readMode=1` defect, rather than as a clean measurement of the final, fixed system. This project’s hardware access concluded before `monitor_goal_timing.py` could be re-run with the `readMode=0` fix in place, so no corrected figures are available (Section 7.8).

6.6 Final Validation

Following the `readMode=0` fix (Section 5.6), a live Nav2 goal was re-run on the physical vehicle – full onboard bridge, LiDAR node and relay node, with AMCL localised against the existing saved map. My direct, contemporaneous assessment was unambiguous: navigation was “working almost perfectly.” This is a strong qualitative result, consistent with the buffering defect having been the dominant cause of the “skeleton of the map is moving” symptom that opened Section 6.5,

rather than merely a contributing factor to it. `monitor_goal_timing.py` was not re-run for this specific goal, so the precise post-fix start-gap and end-of-goal figures corresponding to Table 6.2 are not available as quantitative numbers in this thesis.

6.7 Summary

This chapter closes the sim-to-real migration begun in Chapter 3. The physical QCar, connected through the bridge of Chapter 4 and calibrated per Chapter 5, was shown to complete real, closed-loop Nav2 navigation goals. This included an Ackermann-appropriate custom MPPI critic and phased integration that isolated each subsystem’s own correctness before combining it with the next.

Section 6.4’s custom smoother resolved a genuine execution-side freeze at reversal cusps, reducing a representative K-turn manoeuvre’s simulated mean duration from 110.2s with recoveries to 43.9s with none. The fix was validated directly on the physical vehicle across two independent goals, with zero recoveries in either. Its two reverted follow-on attempts and the still-open gap between simulated and real replan complexity are reported alongside it rather than omitted. The one remaining significant open item is quantitative rather than functional: this chapter’s display-lag investigation (Section 6.5) was measured under a bridge configuration later found to be affected by the buffering defect of Section 5.6. A qualitative post-fix confirmation exists, but a fresh quantitative re-measurement does not. Chapter 7 draws together the quantitative results available across this and the preceding two chapters.

7 Results

This chapter consolidates the quantitative results produced across Chapters 3–6 and evaluates them directly against the target state defined in Section 1.2: a Nav2 navigation stack that reaches goal poses reliably on the physical QCar, with the sim-to-real gap identified and, where practical, corrected. Results are presented here without re-opening their diagnosis; interpretation and generalisation are deferred to Chapter 8.

Table 7.1 gives the headline figure for each result before the corresponding section restates it in full detail. Every row here is a forward reference, not a new claim.

Table 7.1: Results at a glance.

Metric	Before → after	Section
Throttle speed accuracy	2.6–2.7× error → within 5%	7.1
Steering angle accuracy	≈39% error → ≈5.6%	7.4
Stop latency (mean)	2.42 s → 0.55 s	7.2
Stop extra distance (mean)	0.69 m → 0.12 m	7.2
K-turn duration, sim (mean)	110.2 s/2.0 rec. → 43.9 s/0.0 rec.	7.5
K-turn goals, real hardware	2/2 succeeded, 0 recoveries	7.5
Goal 2, repeated 4×, real hardware	4/4 succeeded, position error 0.129 m to 0.222 m (real variability)	7.6
Goal 3 (near-180° reversal), real hardware	succeeded, position error 0.239 m, yaw error 6.0°	7.6.3
General goal-reaching, real hardware	no pre-planned, varied success-rate campaign run	7.6, 7.8
Map quality / localisation accuracy	4–7% map-scale underestimate; AMCL forward-distance gap 6.7–10.6% (growing), yaw accurate	7.7

7.1 Actuator Calibration Accuracy

Table 7.2 restates the throttle calibration validation of Section 5.5.3 alongside the uncalibrated baseline it replaced.

Table 7.2: Commanded-versus-real cruise speed, before and after calibration.

Commanded speed	Baseline ratio (uncalibrated THROTTLE_GAIN = 1/3)	Calibrated ratio (Eq. 5.2)
0.3 m/s	≈2.6–2.7	1.05
0.4 m/s	– (not separately measured)	1.05
0.5 m/s	– (not separately measured)	1.00

The commanded-to-real speed ratio was reduced from $\approx 2.6\text{--}2.7\times$ to within 5% across the 0.3 m/s to 0.5 m/s range validated. Below ≈ 0.25 m/s, both the calibrated and uncalibrated mappings are unreliable in an open-loop sense, since the vehicle’s static-friction deadband (Section 5.5.2) sits in that range independently of gain calibration. Commanding 0.2 m/s moved the vehicle in only some trials.

7.2 Stop-Distance and Latency

Table 7.3 sets out the `readMode=0` fix’s effect (Section 5.6.4) across all five tested conditions, alongside the residual physical explanation for what remains. As noted in Section 5.6.4, these encoder-derived extra-distance and stop-latency figures were independently cross-checked at each tested distance against tape-measured floor markings and manual stopwatch timing. The two agreed.

Table 7.3: Stop-latency and extra-distance results, before and after the `readMode=0` fix.

Test condition	Start-up delay (before $\rightarrow$ after)	Extra distance (before $\rightarrow$ after)	Stop latency (before $\rightarrow$ after)
2 m @ 0.3 m/s	2.09 s to 0.117 s	0.520 m to 0.114 m	2.37 s to 0.598 s
2 m @ 0.4 m/s	2.14 s to 0.001 s	0.790 m to 0.114 m	2.40 s to 0.501 s
2 m @ 0.5 m/s	2.14 s to 0.116 s	0.923 m to 0.154 m	2.55 s to 0.550 s
1 m @ 0.3 m/s	2.14 s to 0.120 s	0.641 m to 0.118 m	2.44 s to 0.572 s
3 m @ 0.3 m/s	2.08 s to 0.088 s	0.583 m to 0.104 m	2.35 s to 0.509 s

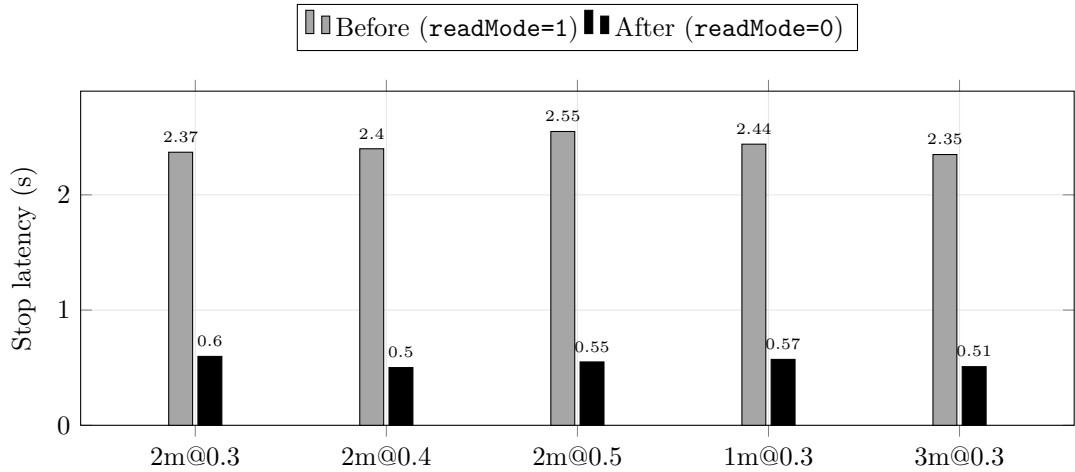


Figure 7.1: Stop latency before and after the `readMode=0` HAL buffering fix (Table 7.3), across all five tested distance/speed combinations. The near-constant $\approx 2.0\text{--}2.1$ s pre-fix latency, largely independent of the commanded distance or speed, is itself evidence for the fixed-buffer-depth mechanism of Figure 5.1 rather than a physical drivetrain effect, which would be expected to scale with speed.

Mean start-up delay across the five post-fix conditions is ≈ 0.09 s (down from ≈ 2.12 s), mean extra distance is ≈ 0.12 m (down from ≈ 0.69 m) and mean stop latency is ≈ 0.55 s (down from ≈ 2.42 s). The post-fix residual is not further reducible with the current open-loop throttle model. Dividing extra distance by stop latency recovers 54 % to 61 % of cruise velocity in four of the five conditions, consistent with an ordinary, roughly linear velocity decay to zero over the measured

latency window (a perfectly linear decay predicts exactly 50%), rather than indicating any further unexplained delay mechanism.

7.3 Sim-versus-Real Overshoot

Table 7.4 presents the sim-versus-real comparison of Section 5.4. Because this comparison was run *before* both the throttle calibration (Section 5.5) and the buffering-defect fix (Section 5.6), it is reported here as the gap that existed at that stage of the project, not as the final residual gap between simulation and hardware. Sections 7.1 and 7.2 above give the corresponding final, post-fix figures, but a matched six-point sim-versus-real sweep was not re-run after both fixes were in place. This project’s hardware access concluded before such a re-run could be scheduled (Section 7.8).

Table 7.4: Sim-versus-real stop-distance overshoot (measured before the fixes of Ch. 5).

	Simulation	Real hardware
Overshoot range (six-point sweep)	0.044 m to 0.067 m	0.58 m to 2.12 m
Isolated 2 m@0.3 m/s test	0.067 m extra	1.90 m extra
Ratio (real / sim), isolated test		$\approx 28\times$
Speed sensitivity	weak	strong ($\propto v^3$, approximately)

7.4 Steering Response Accuracy

Table 7.5 restates the steering-gain validation of Section 4.5.1, isolating the multiplicative gain error from the zero-point trim already covered by Section 7.1’s companion actuator.

Table 7.5: Real wheel deflection versus kinematically commanded deflection, before and after STEERING_GAIN correction.

	Uncorrected (gain= 1)	Corrected (STEERING_GAIN= 1.39)
Residual error vs. kinematic prediction	$\approx 39\%$	$\approx 5.6\%$

As with the throttle mapping, this is a per-unit, empirically fitted correction rather than a property derivable from the vehicle’s nominal kinematic model alone. This is a second, independent confirmation of this thesis’s broader finding that the QCar’s actual actuator response cannot be assumed to match its documented/kinematic specification without direct measurement.

7.5 Navigation Robustness in Tight Spaces

Table 7.6 gives Section 6.4’s K-turn/reversal-cusp results: the representative corridor reproduction goal’s manoeuvre duration and recovery count, across the untouched baseline and the final, corner-extending `CuspStraightenerSmoother` configuration.

This is corroborated, not merely repeated, by real-hardware testing. Two independently sent goals on the physical vehicle, each requiring a genuine reversal manoeuvre, both succeeded with zero recoveries (Section 6.4.6). This is a stronger form of validation than the simulation trial

Table 7.6: K-turn manoeuvre duration and recovery count, before and after the cusp-straightening smoother (simulation, three trials each).

	Baseline (no smoother)	Final config (smoother + corner extension)
Mean duration	110.2 s	43.9 s
Mean recoveries per trial	2.0	0.0
Worst-case duration	168.1 s	48.1 s

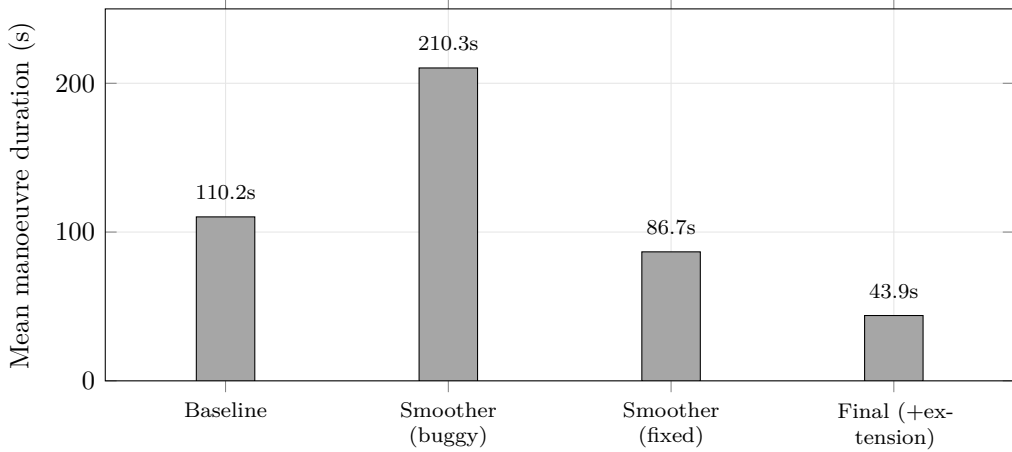


Figure 7.2: Mean K-turn manoeuvre duration across every configuration tested (Table 6.1), including the intermediate, worse-than-baseline result caused by the mutation-order bug of Section 6.4.4 – shown because it happened, not omitted for a cleaner-looking trend. Mean recoveries per trial for the same four configurations, left to right: 2.0, 5.3, 1.3, 0.0.

count alone, since it confirms the fix transfers to real sensor noise, real actuator response and real planner behaviour on unseen goal geometry, not only the specific corridor reproduction it was developed against.

The one qualification recorded against this result is that individual replans on real hardware occasionally showed substantially higher cusp counts (up to 8–10) than any simulation trial produced (maximum ≈ 3). Both hardware goals still succeeded despite this, so it is reported as an open, unexplained gap between simulated and real replanning behaviour, rather than a limitation of the fix itself.

7.6 Navigation Goal-Reaching

Closed-loop `navigate_to_pose` goals requiring both straight driving and real turning were completed successfully on the physical QCar on multiple occasions across this project. This was confirmed both by `bt_navigator`’s own `Goal succeeded` status and, in the final validation of Section 6.6, by my own direct observation that localisation tracked the vehicle’s real motion closely throughout the goal, rather than exhibiting the map-shift artefact of Section 6.5. The one navigation timing measurement available (Table 6.2) was taken under the `pre-readMode=0` bridge configuration and should be read as an upper bound on the true post-fix pipeline latency and stop-coast behaviour during a live goal, not as the final figure, per the caveat in Section 6.5.4.

The one goal class with a genuine, repeated hardware success count is the reversal/K-turn manoeuvre of Section 7.5: 2 of 2 sent goals succeeded with zero recoveries. General straight-line

and gentle-turn goal-reaching, by contrast, is supported only by my own qualitative assessments above and by individual successful **Goal succeeded** events logged during development, not by a planned, repeated multi-goal trial campaign with a computed success rate. This is stated plainly here, rather than implied to be equivalent to the K-turn result and is carried forward as the first item in Section 7.8.

Two further general (non-K-turn) goals were run on physical hardware on 2026-09-07. Each was logged in full: given goal, AMCL-estimated final pose, the planner’s path statistics and commanded-vs-actual velocity over the whole run, not just a qualitative pass/fail observation.

7.6.1 Goal 1, a Straight-Line Approach

The first trial commanded a straight approach: a goal pose of $(1.978, 0.003, 0^\circ)$ in the map frame, 1.871 m away in a straight line from the vehicle’s pose at goal-send time. Physically, this was an approximately 2 m run set up on the lab floor. **SmacPlannerHybrid** planned a 29-waypoint, zero-cusp (no reversal) path of 1.874 m, essentially the straight line itself (curvature ratio $1.00\times$), smoothed by the cusp-straightener smoother. The goal was accepted at epoch 1788779882.264. **Goal succeeded** was reported 8.47 s later with zero recovery behaviours triggered.

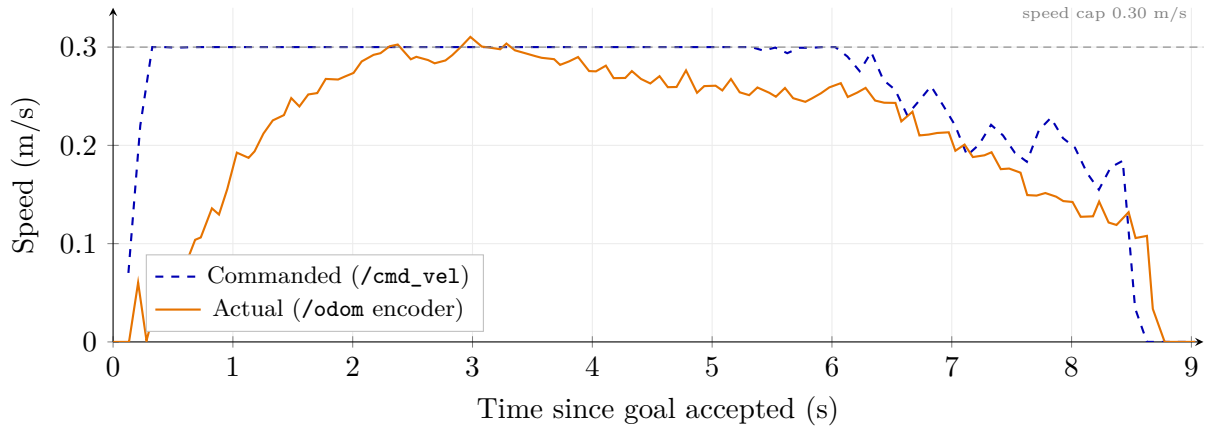


Figure 7.3: Goal 1 (straight approach): commanded MPPI output vs. actual encoder-derived speed. Data logged directly from `/cmd_vel` and `/odom` on the dev PC during the run (50 Hz odometry, ~ 40 Hz `cmd_vel`); commanded velocity is MPPI’s output prior to the QCar bridge’s onboard throttle rate-limiter, actual velocity is wheel-encoder-derived dead reckoning, not ground truth.

Table 7.7: Goal 1: given goal vs. final AMCL-estimated pose (map frame).

	x (m)	y (m)	yaw
Given goal	1.978	0.003	0°
Final pose	1.765	-0.013	not captured
Error	-0.213	-0.016	—

Position error magnitude: 0.214 m (mostly an x -axis undershoot) – outside the configured 0.12 m `xy_goal_tolerance`. The final orientation quaternion was not captured for this run.

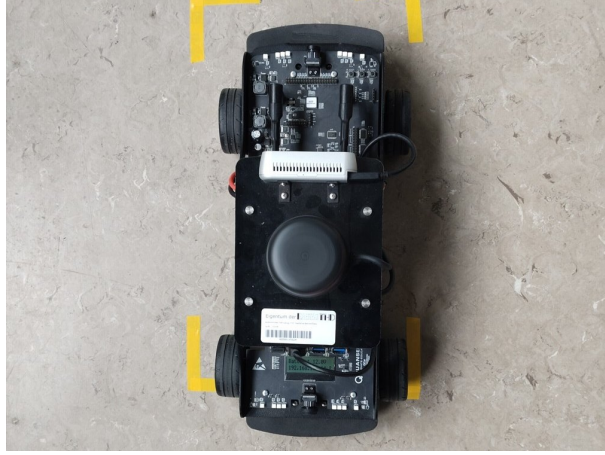


Figure 7.4: The physical QCar at rest at the end of the Goal 1 trial. The yellow tape corners mark the given-goal position, physically measured and placed before the run; the vehicle’s final resting position sits visibly short of centre, consistent with the 0.214 m position error in Table 7.7.

7.6.2 Goal 2, a Curved Approach

The second trial, run on the same map used throughout hardware testing, commanded a goal requiring a genuine turn: $(1.891, -1.775, -90^\circ)$ in the map frame, 2.286 m away in a straight line, but planned as a 3.185 m, 39-waypoint path (curvature ratio $1.39\times$) with zero cusps. The goal was accepted at epoch 1788784376.840. `Goal succeeded` was reported 21.87 s later, again with zero recoveries.

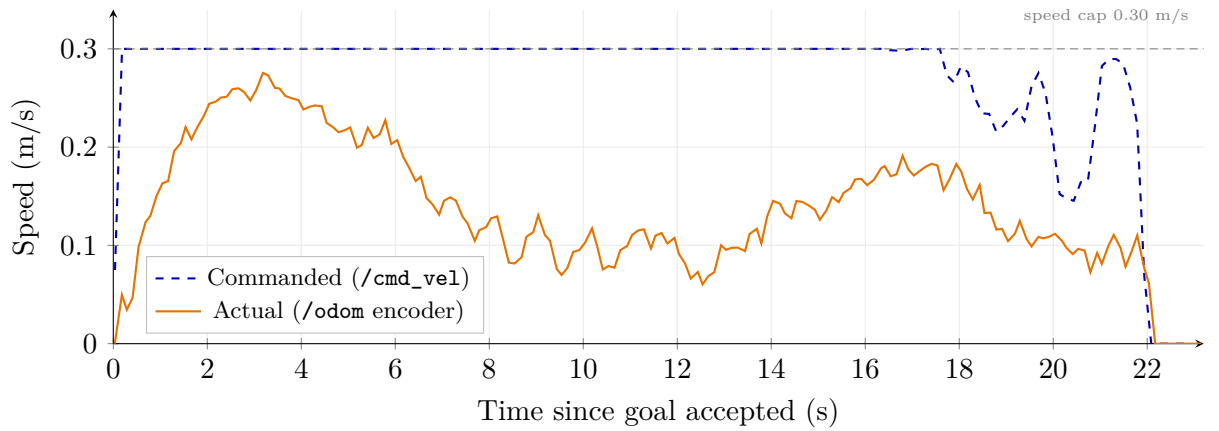


Figure 7.5: Goal 2 (curved approach): commanded MPPI output vs. actual encoder-derived speed. Same logging setup as Figure 7.3. The extended, gradual deceleration around $t \approx 18$ s to 21 s reflects the curved final approach into the goal’s -90° orientation requirement, not a stall or recovery event.

Repeated $3\times$ on 2026-09-10: real run-to-run variability. To check whether Goal 2’s single result above was representative, the identical given goal ($(1.891, -1.775, -90^\circ)$, pulled directly from that run’s own recorded data) was sent three further times on real hardware from the same start position and map, in what turned out to be this project’s final hardware session. All three repeats also reported `Goal succeeded`, but with genuine run-to-run variability. Duration ranged 17.8 s to 60.4 s, some repeats’ planned paths included a reverse-entering cusp that the original run never needed and position error ranged 0.146 m to 0.222 m. No repeat matched the

Table 7.8: Goal 2: given goal vs. final AMCL-estimated pose (map frame).

	x (m)	y (m)	yaw
Given goal	1.891	-1.775	-90°
Final pose	1.905	-1.646	-81.4°
Error	+0.014	+0.128	$+8.6^\circ$

Position error magnitude: 0.129 m – just past the configured 0.12 m `xy_goal_tolerance`. Yaw error 8.6° is comfortably within the 17.2° (0.3 rad) `yaw_goal_tolerance`.



Figure 7.6: The physical QCar at rest at the end of the Goal 2 trial, rotated into its final -90° heading. The yellow tape corners again mark the given-goal position; the vehicle’s final pose is close to, but visibly offset from, the marked box, consistent with the 0.129 m position error and 8.6° yaw error in Table 7.8.

original’s particular combination of fast, clean and accurate. This is a direct, if small-sample, hardware demonstration of the same real run-to-run planning and execution variability this project’s calibration work (Chapters 4–5) already characterises at the actuator level, here visible at the level of whole-path planning and outcome.

Why Goal 1 and Goal 2 exceed their position tolerance. Both Goal 1 and Goal 2 above were reported as `Goal succeeded` despite their final AMCL-estimated pose sitting outside the configured 0.12 m `xy_goal_tolerance` (0.214 m and 0.129 m respectively). This is not a new or unexplained failure mode. It is a hardware confirmation of a tension this project’s own MPPI tuning had already identified between `GoalCritic` (pulling the vehicle to converge exactly on the goal point) and `PathAlignCritic` (favouring a smooth final approach along the planned path), weighted in this project’s configuration to favour the latter. Goal 2’s yaw error (8.6°) remained comfortably inside its own tolerance, so the discrepancy is specific to final position, not orientation.

7.6.3 Goal 3, a Near-180° Reversal Goal

This trial commanded a harder course than Goals 1 and 2: 3.0 m straight ahead of the start position, but with the final orientation reversed 180° from the starting heading. Because an Ackermann-steered vehicle cannot rotate in place, reaching a reversed heading genuinely requires a K-turn/loop-out manoeuvre rather than a spin, directly exercising Section 6.4’s

corner-extending smoother and Section 6.2’s `EarlyCommitCritic`. `SmacPlannerHybrid`’s initial plan (48 waypoints, 1 cusp) was replanned once mid-run, after the live costmap invalidated it, to a final 22-waypoint, 4-cusp path. The goal was accepted at epoch 1789046995.425. `Goal succeeded` was reported 29.3 s later with zero recovery behaviours triggered.

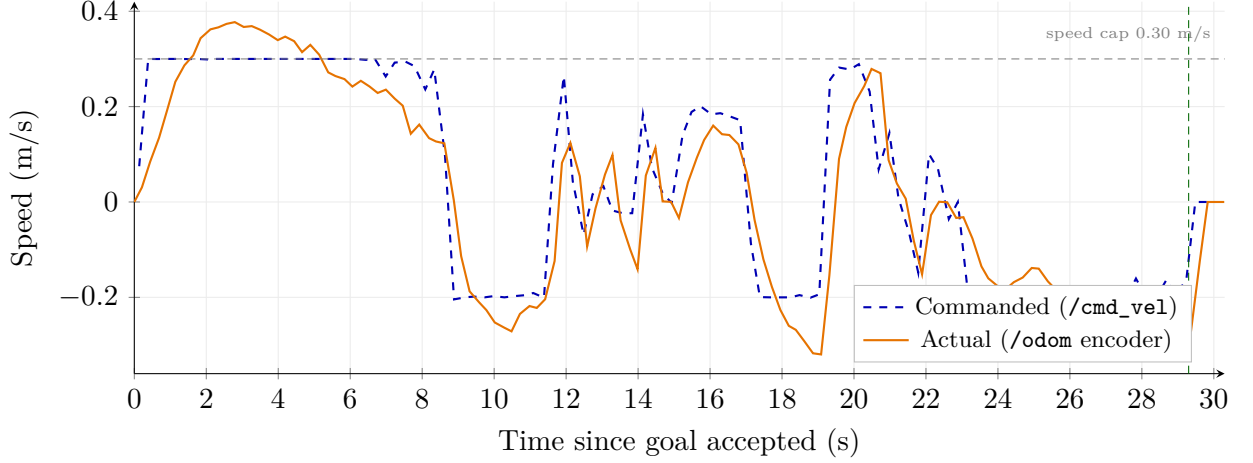


Figure 7.7: Goal 3 (near-180° reversal): commanded MPPI output vs. actual encoder-derived speed. Same logging setup as Figures 7.3–7.5. Negative values are the reversal segment(s) of the manoeuvre; unlike Goals 1 and 2, this path required repeated forward/reverse direction changes to complete the near-180° turn, visible as the multiple sign changes between $t \approx 9$ s and 23 s. The dotted vertical line at 29.3 s marks `Reached the goal!`.

The vehicle covered 5.576 m of real (encoder-integrated) distance to close a 2.662 m straight-line displacement, reaching a peak forward speed of 0.390 m/s and a peak reverse speed of -0.323 m/s across the manoeuvre.

Table 7.9: Goal 3: given goal vs. final AMCL-estimated pose (map frame).

	x (m)	y (m)	yaw
Given goal	3.000	0.000	180.0°
Final pose	2.764	0.039	174.0°
Error	-0.236	+0.039	-6.0°

Position error magnitude: 0.239 m – outside the configured 0.12 m `xy_goal_tolerance`, mostly an x -axis undershoot. Yaw error 6.0° is comfortably within the 17.2° (0.3 rad) `yaw_goal_tolerance`.

7.7 Map Quality and Localisation Accuracy

The two general goals above compare a given goal to AMCL’s own final pose estimate. This is a genuine navigation-accuracy result, but not an independent check of that estimate against physical reality, since both figures come from the same localisation system. This section closes that gap directly with a physical ground-truth measurement, using the same hardware map as the two goals above (`maps/qcar_map_hardware.yaml`) and, for the localisation test, the same straight 2 m goal and start position as Goal 1.



(a) Frame from the handheld hardware recording



(b) Still photo taken after the trial

Figure 7.8: The physical QCar at rest near the end of the Goal 3 trial, close to the yellow tape goal marker. (a) is a frame extracted from a real-hardware video recording of this run, at lower resolution than the tripod-mounted photos used for Goals 1 and 2 since it comes from handheld video rather than a still camera; (b) is a still photo of the same resting vehicle and goal marker, taken immediately afterwards.

7.7.1 Map Quality

Three reference spans were each measured two ways: the map-frame distance between two corners, read via RViz’s `Publish Point` tool on `/clicked_point` and a direct tape measurement of the same real-world span. Table 7.10 presents all three.

Table 7.10: Map-frame distance versus physical tape measurement, three reference spans.

Span	Map distance	Tape distance	Discrepancy
A (long corridor wall, full length)	5.255 m	5.550 m	0.295 m (5.3% under)
B (corridor width)	1.584 m	1.700 m	0.116 m (6.8% under)
C (alcove/doorway width)	1.298 m	1.350 m	0.052 m (3.9% under)

The map consistently *underestimates* real distance by roughly 4–7%, regardless of span length (1.3 m to 5.5 m) or orientation. A one-directional bias of similar magnitude across three such different spans points to a real map-scale error rather than corner-picking imprecision – the map resolution itself (0.05 m/pixel) cannot explain a multi-decimetre discrepancy on the longest span.

7.7.2 Localisation Accuracy Ground-Truth

The same straight-line goal used for Goal 1 (map-frame position (2.0, 0.0), heading 0°) was sent three times via the `NavigateToPose` action client,¹ resetting the vehicle to the same start tape mark each time. AMCL’s final pose was compared against a direct physical measurement: tape distance from the start mark and yaw derived from front/rear-wheel lateral offsets using the wheelbase measured in Section 4.5 ($L = 0.25725$ m), the same principle as that section’s own trim calibration. Table 7.11 presents all three trials.

Two findings follow. AMCL’s yaw estimate matches the independently tape-derived yaw

¹Sent via `ros2 action send_goal`, not the `ros2 topic pub /goal_pose` method used earlier in this project: a `BEST_EFFORT`, no-retransmission topic combined with a short-lived one-shot publisher was found during this session to intermittently drop the goal entirely, with no BT tick and no planner call. The action interface’s handshake accept/reject protocol does not share this failure mode.

Table 7.11: AMCL-reported final pose versus physical ground truth, three trials of the same straight 2 m goal.

Trial	AMCL forward	Tape forward	Gap	AMCL yaw	Tape-derived yaw
1	1.838 m	1.97 m	6.7% under	-1.76°	not measured
2	1.820 m	1.99 m	8.5% under	4.93°	4.45°
3	1.770 m	1.98 m	10.6% under	-2.21°	-2.23°

closely on both trials where the latter was measured (4.93° vs. 4.45° ; -2.21° vs. -2.23°). This is strong evidence that AMCL’s *orientation* estimate is accurate. AMCL also consistently under-reports how far the vehicle actually travelled: the gap grows trial to trial (6.7% $\rightarrow$ 8.5% $\rightarrow$ 10.6%) despite every trial commanding and physically covering essentially the same distance, matching in direction and comparable in magnitude to Test 1’s map-scale finding above.

7.7.3 A LiDAR Range Calibration Curve and a Shared-Bias Hypothesis

The growing, direction-consistent gap between Sections 7.7.1 and 7.7.2 raised a further question, tested ad hoc within the same session: does this bias originate at the LiDAR sensor itself, or somewhere in the SLAM/mapping pipeline? What began as a single 1 m spot-check was expanded, within the same session, into a five-point curve once the first result proved informative. The vehicle was positioned with the LiDAR’s rotation centre at each tape-measured distance from a flat wall. The raw `/scan` range at the beam pointing directly at the wall showed a clean V-shaped minimum across neighbouring beams every time, confirming genuine flat-wall geometry rather than a single noisy reading. Table 7.12 presents all five points.

Table 7.12: Raw LiDAR range versus tape-measured distance from a flat wall, five points.

Tape distance	Raw LiDAR range	Gap	% short
0.5 m	0.490 m	0.010 m	1.9%
1.0 m	0.968 m	0.032 m	3.2%
2.0 m ($\times 2$)	1.884 m / 1.890 m	0.113 m avg	5.7% avg
3.0 m	2.762 m	0.238 m	8.0%

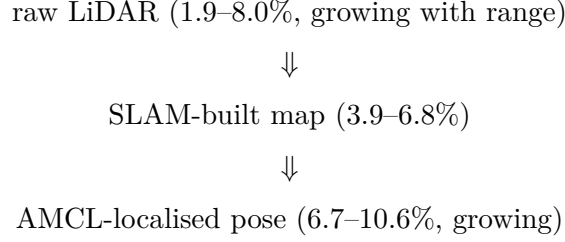
Neither a pure fixed-offset nor a pure fixed-percentage model fits this curve: the percentage error grows roughly linearly with distance (approximately +1.9 percentage points per metre), meaning the absolute gap grows closer to quadratically. A least-squares fit across all five points gives

$$r_{\text{raw}} \approx 0.910 r_{\text{true}} + 0.053 \text{ m}, \quad (7.1)$$

i.e. a combined $\approx 9\%$ range-scale compression plus a small positive fixed offset, not one or the other alone. The two 2 m repeats agree within 6 mm, making this a low-noise, well-characterised curve rather than a single uncertain data point.

Taken together, all three tests point toward one shared cause rather than three independent

effects: a bias that grows as it propagates through the pipeline:



The leading hypothesis is that the LiDAR’s own range-scale nonlinearity (Equation 7.1) is amplified through Cartographer’s pose-graph optimisation into the larger map-scale error, which AMCL then inherits directly, since it localises purely within the map’s own (slightly compressed) coordinate frame rather than against any independent physical reference. This hypothesis is not yet corrected. The session characterised and connected all three findings and produced a real, low-noise correction curve (Equation 7.1), but did not apply it to the LiDAR driver or the map itself. The direct next step is to apply this correction if the driver exposes a range-scale or range-offset parameter, then re-run Sections 7.7.1–7.7.2 to confirm it closes the gap (Chapter 8’s Prospects).

A caveat this reopens for Section 7.6.2. If AMCL under-reports real distance travelled by a margin comparable to Goal 1 and Goal 2’s own position errors (0.214 m and 0.129 m), then the `GoalCritic/PathAlignCritic` tension identified there may not be the sole contributor to those two figures. Some fraction of both may instead reflect the vehicle sitting closer to its true physical goal than AMCL’s own (scale-compressed) estimate reports. Both explanations can be true simultaneously and are not mutually exclusive. This thesis does not have the data to separate their individual contributions for the two specific 2026-09-07 trials, since neither was independently ground-truth-measured at the time. It reports both candidate causes here rather than favouring one without evidence.

7.8 Limitations

The results above are real and, where a number is given, directly measured. But a complete picture of this thesis’s final state also requires stating plainly what was not established. Seven items follow: five are carried forward from the sections above, the sixth is structural to the project as a whole and the seventh is a hardware defect observed throughout testing but never root-caused.

1. **No systematic, pre-planned goal-reaching success-rate campaign across varied goal geometries.** Section 7.6 now has three genuine repeated-trial hardware results – the K-turn goals (2/2, Section 7.5) and Goal 2 repeated four times (4/4 succeeded, real variability in duration, cusps and position error). But these are separate, individually-motivated repeat sessions on specific goals, not one pre-planned campaign spanning a varied, fixed set of goal poses and geometries that would produce a single defensible “X out of Y goals succeeded” headline number for the vehicle as a whole. This project’s hardware

access concluded with the 2026-09-10 session reported throughout this chapter. No further hardware time was available to run such a campaign. This remains an open limitation rather than a planned next step (see Chapter 8’s Prospects for what such a campaign would have needed to cover).

2. **Two results predate the project’s own final fixes.** The sim-versus-real overshoot comparison (Section 7.3) and the navigation-timing figures behind Section 7.6’s caveat were both measured before the `readMode=0` fix (Section 5.6) and, in the sim-versus-real case, before throttle calibration as well. Both are reported as the pre-fix gap that genuinely existed at that point in the project, not silently updated or removed, but neither has been re-measured under the final configuration.
3. **The curvature-continuing cusp refinement is simulation-only.** Section 6.4.7’s most recent smoother refinements are, at the time of writing, verified only in simulation; the hardware-validated 2/2 result above belongs to the earlier, straight-line version of the smoother, not this refinement.
4. **Low-speed closed-loop control was out of scope.** Section 5.5.3 identified sub-0.25 m/s open-loop duty control as unreliable in principle, not as a remaining tuning gap – closing it would require an encoder-feedback velocity loop that this thesis does not implement.
5. **A map-scale and localisation bias was found and characterised, but not root-caused or corrected.** Section 7.7’s ground-truth measurements found the saved map underestimates real distance by 4–7%, AMCL’s reported forward distance under-reports physical travel by a growing 6.7–10.6% and even the raw LiDAR range itself is 1.9–8.0% short against a tape-measured reference, growing with distance (Equation 7.1). This looks like a single bias that grows as it propagates from sensor, through SLAM, to localisation, rather than three independent effects. The leading hypothesis (a LiDAR range-scale bias amplified by Cartographer’s pose-graph optimisation) is not yet confirmed and neither the map nor the LiDAR calibration was corrected in response. This thesis characterises and connects the finding, including a low-noise correction curve, without resolving it.
6. **The scope was navigation, not full autonomy.** This thesis addresses reaching a given goal pose reliably; it does not address perception-driven goal selection, dynamic obstacle avoidance beyond Nav2’s stock costmap layers, or multi-goal mission planning. Each is a substantial extension in its own right, named explicitly in Chapter 8’s Prospects rather than left as an unstated boundary of the work.
7. **The LiDAR scan display still freezes momentarily during sharp turns and sudden forward acceleration.** This symptom recurred throughout hardware testing across this project and remains unresolved at the time of writing. It has not been root-caused. Whether it affects only the RViz visualisation or the underlying `/scan` stream Nav2 itself consumes is not established.

7.9 Applicability of the Results

The throttle deadband/gain values (Equation 5.2) and the steering-trim value (Section 4.5) are per-unit calibration constants, expected to differ, potentially substantially, on a different physical QCar chassis, or even on this same chassis under different tyre wear, floor surface, or battery charge conditions. This was confirmed directly, not only in principle, by a 2026-09-10 hardware session weeks after the calibration above. Three clean, repeated 2 m forward runs at three different commanded speeds all showed a consistent $\approx 10\%$ undershoot against Equation 5.2’s prediction, so `THROTTLE_GAIN` was re-fitted from 0.0667 to 0.077 for that session. Reverse-direction driving showed a further, distinct issue: stop latency and extra distance were consistently $2\text{--}4\times$ worse than forward’s at every tested speed. A single spot-check (1 m @ -0.3 m/s) later in the same session came back statistically identical to forward’s normal numbers, an intermittent effect. A physical inspection (wheels, axle couplings) found no mechanical fault, pointing instead to the open rear differential’s own load-dependent torque redistribution as a real, physically-expected contributor to run-to-run traction variability that no static software gain constant can fully correct for. The *method* used to find these constants is an empirical fixed-duty sweep under real load, rather than an analytically derived or single-point gain estimate. It is expected to transfer directly to any similarly actuated small-scale ground vehicle, independent of the specific numeric results.

Two further findings generalise beyond this platform and this domain. The `readMode` buffering defect (Section 5.6) is specific to the Quanser HAL’s task-based I/O mode. The general failure class it represents is a background acquisition task started at object-construction time, with a read-consumption rate that happens to match its production rate closely enough that a start-up-gap backlog never drains. This is a generic ring-buffer hazard worth checking for in any hardware abstraction layer offering a buffered or task-based I/O mode as an alternative to immediate/pollled reads. The diagnostic principle that resolved it is a general debugging principle, not specific to this project: when an internally self-consistent software abstraction produces a value that contradicts direct physical observation, verify the abstraction’s own *input*, not only its internal consistency. Similarly, the active-braking safety incident (Section 5.3) and its resolution generalise directly to any reactive feedback-control design intended for direct hardware deployment on a system with confirmed actuator lag or inertia, regardless of application domain: trace new reactive control logic through by hand for an overshoot scenario, or validate it in simulation, before it reaches a physical actuator.

8 Summary and Prospects

8.1 Summary

This thesis set out to migrate a Nav2-based navigation stack for an Ackermann-steered vehicle, the Quanser QCar, from a validated Gazebo simulation onto the physical robot and to identify and address the sim-to-real gap encountered along the way (Section 1.2). Each of the four sub-questions posed in the introduction is answered directly by one methodology chapter:

- **Simulation-stack structure** (Chapter 3): the stack was migrated twice. It moved from a hand-rolled bicycle-model inversion to `ros2_control`'s Ackermann controller, deliberately adopting real dead-reckoning odometry rather than simulation ground truth, so the eventual hardware migration would not have to absorb that discrepancy on top of everything else. It then moved to a direct Gazebo drive plugin. As Section 3.4.1 documents, this plugin quietly reverted `/odom` back to ground truth despite the topic name and intent staying the same. This thesis surfaces that discrepancy rather than leaving it uncorrected.
- **Bridging the QCar's onboard compute** (Chapter 4): the vehicle's Jetson TX2 could not run the project's ROS 2 Humble stack natively. Its 32 GB eMMC had no realistic room for a second full ROS 2 installation alongside its factory software. Native ROS 2 communication between the vehicle's onboard ROS 2 Dashing and the development host's ROS 2 Humble was also confirmed, through direct testing rather than assumption, to be impossible at the DDS protocol level. A TCP/JSON bridge was designed and built that presents the physical robot to the rest of the navigation stack as an ordinary ROS 2 robot. LiDAR angle-convention, odometry, steering-trim and inter-machine clock-synchronisation corrections are layered on top, bringing the bridged data to a fidelity usable by AMCL and Nav2.
- **Actuator identification and calibration** (Chapters 4–5): an uncalibrated throttle mapping ($2.6\text{--}2.7\times$ overshoot) was corrected to within 5% accuracy across 0.3 m/s to 0.5 m/s via an empirical duty sweep. A large (≈ 2 s) apparent coasting delay was ultimately traced not to physical drivetrain dynamics but to a ring-buffer artefact in the vendor hardware abstraction layer's task-based I/O mode, reducing start-up delay and stop latency by more than an order of magnitude once corrected. Independently, a real $\approx 39\%$ steering-gain error was characterised through a seven-point sweep and corrected to $\approx 5.6\%$ residual error (Section 4.5.1). This error was separate from the vehicle's zero-point trim and was not fixed by it.
- **Closed-loop validation** (Chapter 6): the bridged, calibrated vehicle was shown to complete real Nav2 goals requiring both straight driving and turning, using a custom Ackermann-appropriate MPPI critic and a phased integration approach that isolated each subsystem before combining it with the next. A separate execution-side freeze at reversal cusps during K-turn manoeuvres in tight spaces was diagnosed to MPPI's own cusp-arrival gating and resolved with a custom path-smoothing plugin. This cut a representative

manoeuvre’s mean duration from 110.2 s (with recoveries) to 43.9 s (with none) in simulation. The fix was validated directly on the physical vehicle across two independent goals, with zero recoveries in either (Section 6.4).

Chapter 7 quantified the resulting gap. Actuator response now tracks commanded speed within 5% (down from a $2.6\text{--}2.7\times$ error). Stop-latency/coast behaviour was reduced from roughly 2.4 s and up to $3\times$ the commanded stop distance, to approximately 0.5 s and ≈ 0.1 m. This residual is fully explained by ordinary linear velocity decay, not by any remaining unexplained mechanism.

8.1.1 Critical Reflection

What worked well in this project was the discipline of measuring directly on hardware, rather than trusting a plausible-sounding explanation. This thesis’s four most significant findings are the protocol-version incompatibility in Section 4.1, the sim-vs-real overshoot ratio in Section 5.4, the buffering defect in Section 5.6 and the reversal-cusp freeze in Section 6.4. Each was, at some point, plausibly attributable to a simpler cause: localisation error, a request-delivery bug, physical coasting, or a tuning-only gain problem. Direct measurement subsequently ruled each of these out.

The cusp-freeze investigation additionally illustrates the same discipline applied to solutions, not only diagnoses. Three plausible fix attempts and two later follow-on refinements were each tested directly against hardware or a controlled reproduction. Each was found wanting by measurement rather than intuition, then reverted. The evidence that ruled it out was recorded, not discarded. What could have gone better is equally clear from Section 5.3: a reactive control design was deployed to physical hardware before it had been adequately traced through or simulated. It produced a genuine safety incident. The incident causing no lasting harm does not make the shortcut it resulted from acceptable practice. This thesis records it, unedited, as a documented counter-example to its own otherwise measurement-first methodology.

A second, more structural limitation is that not every quantitative result in this thesis is a genuinely final, post-fix measurement. Not every claim of success, either, is backed by a repeated-trial statistic rather than a qualitative assessment. Section 7.8 states these gaps in full, in one place, rather than leaving them scattered across individual chapters. Most notably, no systematic goal-reaching success-rate campaign was run. Two results – navigation timing and the sim-versus-real comparison – predate the project’s own final fixes. These are reported as open items rather than papered over. The exact next steps to close each of them are specific and immediately actionable rather than vague, as the Prospects below make concrete.

8.2 Prospects

Each limitation named in Section 7.8 motivates at least one item below. This project’s hardware access concluded with the 2026-09-10 session reported throughout Chapter 7. Every item below is accordingly a direction for continuation work – by this project or a successor – rather than a next step this thesis itself still has the means to take:

- **Re-running the two pre-fix measurements.** `monitor_goal_timing.py` (Section 6.5) and the six-point sim-versus-real sweep (Section 7.3) were both measured before the `readMode=0` fix. The sweep was also measured before throttle calibration. Both would need a further QCar lab session to re-run under the final configuration, closing Section 7.8’s item 2 with genuine post-fix numbers instead of a qualitative confirmation.
- **A planned, repeated goal-reaching evaluation campaign.** Closing Section 7.8’s item 1 directly, a fixed set of goal poses spanning straight-line, gentle-turn and reversal geometries would each be repeated across multiple trials. This would produce the single defensible goal-reaching success-rate figure this thesis currently lacks. This figure would be the natural companion to the K-turn-specific 2/2 hardware result already in hand (Section 7.5).
- **Correcting the map/localisation scale bias.** Section 7.7’s ground-truth benchmarking (closing Section 7.8’s item 5) found a real, direction-consistent bias: map $-4-7\%$, AMCL forward distance $-6.7-10.6\%$ and growing, raw LiDAR range $-1.9-8.0\%$ and growing with distance. It characterised this bias with a low-noise, five-point LiDAR calibration curve (Equation 7.1), but did not apply any correction. If the LiDAR driver exposes a range-scale or range-offset parameter, applying Equation 7.1’s fit and re-running Sections 7.7.1–7.7.2 would show directly whether correcting the sensor closes the larger map- and AMCL-level gaps, or whether a second, independent effect inside Cartographer’s pose-graph optimisation also contributes.
- **Closed-loop low-speed control.** This thesis’s calibration (Section 5.5.3) explicitly identified the sub-0.25 m/s regime as a hard limit of open-loop duty control, not a remaining tuning gap. Reliable low-speed creep – useful for close-quarters manoeuvring or fine final-approach positioning – would require closed-loop velocity feedback from the encoder rather than the fixed duty mapping used throughout this thesis.
- **Deceleration-profile tuning for coast reduction.** The stop-latency residual characterised in Section 7.2 is now small, but not zero and scales with cruise speed. Given that active braking is explicitly ruled out on safety grounds (Section 5.3), a lower-risk avenue is tuning the MPPI `GoalCritic`/deceleration profile in `nav2_params.yaml` so the controller commands a lower cruise speed during final approach to a goal. This was identified but not investigated within this thesis. It would proportionally shrink, not eliminate, the coast without introducing any new reactive control loop.
- **Generalising the diagnostic methodology.** This thesis’s most transferable contribution may be procedural rather than numeric: an internally self-consistent software abstraction can mask a stale or defective input (Section 5.6). The specific discipline of verifying a component’s own upstream input, not only its internal consistency, is a general principle worth testing on other buffered or task-based hardware interfaces beyond the one vendor library examined here.
- **Closing the simulated-versus-real replan-complexity gap.** Section 6.4.6 found that real-hardware K-turn manoeuvres occasionally produce substantially higher per-replan cusp

counts (8–10) than any simulation trial reproduced (maximum ≈ 3), without preventing either tested goal from succeeding. The leading candidate explanations were identified but not isolated and remain open. One is `SmacPlannerHybrid`’s sensitivity to real, more awkward goal/environment geometry. The other is a residual, session-specific clock-offset effect similar to the one implicated in Section 6.4.9’s reverted `IsPathValidDebounced` attempt. Relatedly, a deviation-aware critic weighting scheme was proposed but not implemented, as a controller-side complement to Section 6.4’s path-preprocessing fix. The scheme would ease a sampled trajectory’s pull back toward the exact planned path once it has deviated significantly, favouring a goal-direction target instead of insisting on precise reconvergence.

- **Hardware re-validation of the curvature-continuing smoother refinement.** Section 6.4.7’s two most recent refinements are the `min_lead_distance` fix to the cusp re-extension bug and the curvature-continuing extension itself. At the time of writing, both were verified only in simulation: a single smoke test for the curvature refinement and two trials without directly observing the skip branch fire for `min_lead_distance`. Both are strict generalisations of the already hardware-validated straight-line mechanism and are expected to transfer. This thesis deliberately does not claim hardware confirmation it does not have. Repeating Section 6.4.6’s two-goal real-vehicle test against this refined configuration is the immediate next step.
- **IMU/odometry sensor fusion.** This thesis’s pose estimation between LiDAR updates relies solely on wheel-encoder odometry (Section 2.6); the QCar’s onboard IMU is present and readable through the vendor HAL/PAL library but was never read by this project’s software stack. Fusing the two via an extended or unscented Kalman filter, for example the ROS 2 `robot_localization` package, is a specified but unbuilt extension: IMU and wheel odometry in, a filtered `odom`→`base_link` transform out, feeding AMCL exactly as the current unfiltered odometry does. It was considered during this project but not implemented.
- **Beyond navigation.** This thesis’s scope, stated explicitly in Section 7.8’s final item, is reaching a given goal pose reliably. It does not address how that goal pose is chosen, how the vehicle would react to an obstacle Nav2’s stock costmap layers do not already handle, or sequencing multiple goals into a mission. This thesis’s navigation pipeline (Section 2.6) uses only the 2D LiDAR; no camera-based perception and no IMU data was used or needed for pose-goal navigation. Perception-driven goal selection, dynamic obstacle handling beyond the static/inflation costmap layers used throughout and a mission-level task layer above `navigate_to_pose` are the three most direct extensions of this work into full autonomy. Each is substantial enough to be its own follow-on project rather than a tuning task.
- **Diagnosing the LiDAR scan-rendering freeze.** Section 7.8’s item 7 records a scan display freeze during sharp turns and sudden forward acceleration that recurred throughout hardware testing but was never root-caused. The immediate next step is determining whether the freeze is confined to the RViz visualisation pipeline or also affects the `/scan`

stream Nav2’s own controller consumes, since only the latter would be a genuine navigation-accuracy concern rather than a cosmetic one.

The QCar’s navigation stack, as delivered by this thesis, reaches real closed-loop navigation goals on physical hardware with a sim-to-real gap that is now measured, explained and in most respects substantially reduced – the original goal stated in Section 1.2. What remains open is narrower and better characterised than what this thesis began with: not why the system does not work, but two specific, already-scoped measurements and two specific, already-identified directions for further improvement.

Bibliography

- [1] Anthropic. Claude (sonnet 5), 2026. AI assistant used for drafting and editing support; see the Statement of Authorship for the scope of its use.
- [2] Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008. https://www.cds.caltech.edu/~murray/amwiki/index.php/Second_Edition.
- [3] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, José Neira, Ian Reid, and John J. Leonard. Past, Present, and Future of Simultaneous Localization And Mapping: Toward the Robust-Perception Age. *IEEE Transactions on Robotics*, 32(6):1309–1332, 2016.
- [4] Michele Colledanchise and Petter Ögren. Behavior trees in robotics and AI: An introduction, 2018. arXiv:1709.00084.
- [5] Frank Dellaert, Dieter Fox, Wolfram Burgard, and Sebastian Thrun. Monte carlo localization for mobile robots. In *1999 IEEE International Conference on Robotics and Automation (ICRA)*, 1999.
- [6] Dmitri Dolgov, Sebastian Thrun, Michael Montemerlo, and James Diebel. Practical search techniques in path planning for autonomous driving. In *Proceedings of the First International Symposium on Search Techniques in Artificial Intelligence and Robotics (STAIR)*, 2008.
- [7] Alberto Elfes. Using occupancy grids for mobile robot perception and navigation. *Computer*, 22(6):46–57, 1989.
- [8] Wolfgang Hess, Damon Kohler, Holger Rapp, and Daniel Andor. Real-time loop closure in 2d lidar slam. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016.
- [9] Nathan Koenig and Andrew Howard. Design and use paradigms for gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2149–2154, 2004.
- [10] Steve Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), 2022.
- [11] Steve Macenski, Francisco Martín, Ruffin White, and Jonatan Ginés Clavero. The marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [12] David Mills, Jim Martin, Jack Burbank, and William Kasch. Network time protocol version 4: Protocol and algorithms specification. RFC 5905, 2010. <https://www.rfc-editor.org/rfc/rfc5905>.

- [13] Object Management Group. *The Real-time Publish-Subscribe Protocol (RTPS) DDS Interoperability Wire Protocol Specification, Version 2.2*, 2014. OMG Document formal/2014-09-01.
- [14] Object Management Group. *Data Distribution Service (DDS), Version 1.4*, 2015. OMG Document formal/2015-04-10.
- [15] Jon Postel. User datagram protocol. RFC 768, 1980. <https://www.rfc-editor.org/rfc/rfc768>.
- [16] Jon Postel. Transmission control protocol. RFC 793, 1981. <https://www.rfc-editor.org/rfc/rfc793>.
- [17] Quanser Inc. *QCar System Hardware User Manual*, 2023. Version 1.0, 9 January 2023. Not to be confused with the separate QCar 2 platform and its own documentation.
- [18] Quanser Inc. *QCar Troubleshooting User Manual*, 2023. Version 1.0, 9 January 2023.
- [19] Morgan Quigley, Brian Gerkey, Ken Conley, Josh Faust, Tully Foote, Jeremy Leibs, Eric Berger, Rob Wheeler, and Andrew Ng. Ros: an open-source robot operating system. In *ICRA Workshop on Open Source Software*, 2009.
- [20] Russell Smith. *Open Dynamics Engine User Guide*. ODE Project, 2006. <https://ode.org/ode-latest-userguide.pdf>.
- [21] Jarrod M. Snider. Automatic steering methods for autonomous automobile path tracking. Technical Report CMU-RI-TR-09-08, Robotics Institute, Carnegie Mellon University, 2009.
- [22] Sebastian Thrun. Probabilistic algorithms in robotics. Technical Report CMU-CS-00-126, School of Computer Science, Carnegie Mellon University, 2000.
- [23] Sebastian Thrun, Dieter Fox, Wolfram Burgard, and Frank Dellaert. Robust monte carlo localization for mobile robots. *Artificial Intelligence*, 128(1-2):99–141, 2001.
- [24] Grady Williams, Andrew Aldrich, and Evangelos A. Theodorou. Model predictive path integral control using covariance variable importance sampling, 2015. arXiv:1509.01149.
- [25] Wenshuai Zhao, Jorge Peña Queralta, and Tomi Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 737–744, 2020.

A Appendix

A.1 QCar Hardware Safety Limits

Table A.1 lists the documented hardware safety limits enforced throughout every piece of drive-control code presented in this thesis. The PWM and steering/overcurrent limits are discussed in Section 2.6 [17, 18]. The battery voltage thresholds are documented only in Quanser’s troubleshooting manual [18] and are not otherwise discussed in the main text.

Table A.1: QCar documented hardware safety limits.

Limit	Value
PWM duty cycle saturation	$\pm 30\%$
PWM duty cycle rate limit	100%/s
Battery auto-shutdown	below 10.0 V
Battery low-voltage warning	below 10.5 V
Steering command range	± 0.5 rad
Max. physical steering angle	± 0.5236 rad ($\pm 30^\circ$)
Overcurrent protection	5 A/8 s, 10 A/2 s, or 15 A/0.5 s $\rightarrow$ forced Neutral mode

A.2 Diagnostic and Calibration Tooling Reference

Table A.2 summarises the diagnostic scripts developed and used throughout Chapters 4 –6. Full source is maintained in the project’s version-controlled repository (electronic submission), not reproduced here in full. This follows the guideline’s own recommendation that programming code be supplied on the electronic data carrier rather than printed.

Table A.2: Diagnostic and calibration scripts referenced in this thesis.

Script	Purpose
<code>calibrate_steering.py</code>	Static steering-trim calibration aid (Section 4.5)
<code>drive_and_log.py</code>	Drive to a target distance at a fixed speed, log <code>/odom</code> to CSV, exit automatically (Section 5.5.3)
<code>check_stop_latency.py</code>	As above, plus reports cruise velocity and stop latency directly (Section 5.5.2)
<code>monitor_goal_timing.py</code>	Instruments one Nav2 goal end-to-end: goal-sent, real/displayed motion start and stop, goal-reached (Section 6.5.2)
<code>log_goal.py</code>	Logs every <code>/plan</code> replan (sequence-numbered CSVs), <code>/odom</code> and <code>/cmd_vel</code> for one goal, for offline K-turn/cusp analysis (Section 6.4.1)
<code>CuspStraightenerSmoother</code>	<code>nav2_core::Smoother</code> plugin resolving the reversal-cusp freeze (Section 6.4.4); locked-in production configuration
<code>IsPathValidDebondedCondition</code>	Custom BT.CPP condition node requiring consecutive failed validity checks before replanning; implemented and hardware-tested, currently reverted/deselected (Section 6.4.9)
<code>PathDeviationCritic</code>	Custom MPPI critic pulling deviated trajectories toward a lookahead path point; implemented and tested, currently reverted/deselected (Section 6.4.9)

A.3 Real-Hardware Bring-Up Command Reference

For reproducibility, the full bring-up sequence used throughout Chapters 4–6 is reproduced here.

```
1 # --- On the QCar (two terminals) ---
2 sudo -E env PYTHONPATH=/home/nvidia/Documents/python \
3     python3 ~/ros2_amit/src/onboard_bridge/qcar_bridge.py
4
5 PYTHONPATH=/home/nvidia/Documents/python \
6     python3 ~/ros2_amit/src/onboard_bridge/qcar_lidar_node.py
7
8 # --- On the development PC ---
9 ros2 run qcar_updated qcar_relay_node.py --ros-args \
10     -p qcar_ip:=<qcar-ip> -p qcar_clock_offset:=<measured-offset>
11
12 ros2 launch qcar_updated qcar_nav2.launch.py launch_sim:=false
```

Listing A.1: Real-hardware bring-up (two QCar terminals, two development-PC terminals).

`qcar_clock_offset` (Section 4.6) and the QCar’s DHCP-assigned IP address both drift session to session and must be re-measured / re-checked before each bring-up, not reused from a previous session.

A.3.1 Safe Shutdown Sequence

The shutdown sequence established in Section 4.7 must be followed in order, using `SIGINT` (Ctrl-C) exclusively:

1. Stop `qcar_relay_node.py` (development PC).
2. Stop the visualisation launch process group (development PC).
3. Stop `qcar_lidar_node.py` (QCar).
4. Stop `qcar_bridge.py` (QCar, requires `sudo`).
5. Confirm both TCP ports are released on the QCar: `sudo ss -tlnp | grep -E "5555|5556"` should print nothing.

A.4 Source Code and Data Availability

The complete, version-controlled source code for the simulation stack (Chapter 3), the hardware bridge (Chapter 4) and all calibration/diagnostic tooling (Appendix A.2) referenced throughout this thesis is supplied in the accompanying repository: https://github.com/amitsajeev-byte/qcar_nav_stack.